

Industrial Camera Development Manual

Version 2.4

contents

INTRODUCTION	1
1 OVERVIEW	2
<i>1.1 FILE STRUCTURE.....</i>	<i>2</i>
<i>1.2 DEVELOPMENT ENVIRONMENT.....</i>	<i>3</i>
<i>1.3 MATTERS NEEDING ATTENTION (COMMON PROBLEMS).....</i>	<i>4</i>
<i>1.4 XPLANATION OF COMMONLY USED WORDS IN INDUSTRIAL CAMERAS.....</i>	<i>5</i>
1.4.1 Line scan camera and area scan camera	5
1.4.2 Rolling shutter and global shutter	6
1.4.3 Hardware trigger (external trigger)	7
1.4.4 Optical size (target surface)	8
1.4.5 Frame Memory and Cache	8
2 QUICK DEVELOPMENT GUIDE	10
<i>2.1 OVERVIEW OF CAMERA OPERATION PROCESS.....</i>	<i>10</i>
<i>2.2 DEVELOPMENT ROUTINE.....</i>	<i>16</i>
2.2.1 Basic routine.....	20
2.2.2 Advanced routines	21
2.2.3 MultiCamera routines (use multiple cameras at the same time)	23
2.2.4 OCX routines (ActiveX)	23
2.2.5 MultiExposure routine (a camera with two exposure channels preview, wide dynamic effect)	23

2.2.6 ImageFormat&Saving routines (Gray, RGB24, RGB32 format settings)	25
2.2.7 TriggerAndStrobe routine (trigger and flash signal control)	26
2.2.8 UserDataTest routine (read and write custom data to the camera)	28
2.2.9 SnapshotOnPreview routine (small resolution high-speed preview, large resolution photo)	29
2.2.10 RawTransTest routines (convert offline RAW files to BMP, JPG files)	30
2.2.11 LineScan routine (using line scan mode)	31
2.2.12 ROI routine (how to customize camera image size)	32
2.2.13 GPIO routines (only valid for models with GPIO)	33
2.2.14 SaveFile routine (continuously save each preview image to disk)	34
<i>2.3 DEBUG CAMERA PARAMETERS</i>	35
2.3.1 How to set the exposure time (anti-stroboscopic, no smear, dynamic range improvement)	35
2.3.2 How to get better image color	38
2.3.3 How to improve image clarity	40
2.3.4 Selection of Bayer reduction algorithm (helps to better extract image contours)	41
2.3.5 How to reduce CPU usage	41
2.3.6 Save and load camera parameters	42
3 SDK DATA TYPE DEFINITION	47

3.1 STRUCTURE DEFINITION.....	47
3.2 PARAMETER TYPE DEFINITION.....	54
3.3 INTERFACE RETURN VALUE DEFINITION (ERROR CODE).....	75
4 SDK INTERFACE FUNCTION DESCRIPTION (C/C++ VB DELPHI C# GENERAL)	81
4.1 CAMERASDKINIT.....	81
4.2 CAMERAINIT.....	82
4.3 CAMERAINITEX.....	83
4.4 CAMERAINITEX2.....	85
4.5 CAMERADISPLAYINIT.....	86
4.6 CAMERASETDISPLAYMODE.....	87
4.7 CAMERASETDISPLAYOFFSET.....	88
4.8 CAMERASETCALLBACKFUNCTION.....	89
4.9 CAMERAUNINIT.....	90
4.10 CAMERAPLAY.....	90
4.11 CAMERAPAUSE.....	91
4.12 CAMERASTOP.....	92
4.13 CAMERACREATESETTINGPAGE.....	93
4.14 CAMERACUSTOMIZERESOLUTION.....	94
4.15 CAMERASHOWSETTINGPAGE.....	95
4.16 CAMERASETACTIVSETTINGSUBPAGE.....	96
4.17 CAMERAGETINFORMATION.....	97
4.18 CAMERAIMAGEPROCESS.....	97

4.19 CAMERAIMAGEPROCESSEx.....	98
4.20 CAMERADISPLAYRGB24.....	100
4.21 CAMERASETDISPLAYSIZE.....	100
4.22 CAMERAGETIMAGEBUFFER.....	101
4.23 CAMERAGETIMAGEBUFFEREX.....	102
4.24 CAMERARELEASEIMAGEBUFFER.....	103
4.25 CAMERAGETFRAMESTATISTIC.....	104
4.26 CAMERASETNOISEFILTER.....	105
4.27 CAMERAGETNOISEFILTERSTATE.....	106
4.28 CAMERAINITRECORD.....	106
4.29 CAMERASTOPRECORD.....	107
4.30 CAMERAPUSHFRAME.....	108
4.31 CAMERASAVEIMAGE.....	109
4.32 CAMERASETMIRROR.....	111
4.33 CAMERAGETMIRROR.....	111
4.34 CAMERASETMONOCHROME.....	112
4.35 CAMERAGETMONOCHROME.....	113
4.36 CAMERASETINVERSE.....	114
4.37 CAMERAGETINVERSE.....	115
4.38 CAMERASETIMAGERESOLUTION.....	115
4.39 CAMERAGETIMAGERESOLUTION.....	116
4.40 CAMERAGETMEDIATYPE.....	117

4.41 CAMERASetMEDIATYPE.....	118
4.42 CAMERASetAESTATE.....	119
4.43 CAMERAGetAESTATE.....	120
4.44 CAMERASetAETARGET.....	121
4.45 CAMERAGetAETARGET.....	121
4.46 CAMERASetEXPOSURETIME.....	122
4.47 CAMERAGetEXPOSURETIME.....	123
4.48 CAMERAGetEXPOSURELINETIME.....	124
4.49 CAMERASetANALOGGAIN.....	124
4.50 CAMERAGetANALOGGAIN.....	125
4.51 CAMERASetAEWINDOW.....	126
4.52 CAMERAGetAEWINDOW.....	127
4.53 CAMERASetSHARPNESS.....	128
4.54 CAMERAGetSHARPNESS.....	129
4.55 CAMERASetONCEWB.....	130
4.56 CAMERASetONCEBB.....	130
4.57 CAMERASetLUTMODE.....	131
4.58 CAMERAGetLUTMODE.....	132
4.59 CAMERASelectLUTPRESET.....	133
4.60 CAMERAGetLUTPRESETSEL.....	134
4.61 CAMERASetCUSTOMLUT.....	134
4.62 CAMERAGetCUSTOMLUT.....	135

4.63 CAMERAGetCurrentLUT.....	136
4.64 CAMERASetWbMode.....	137
4.65 CAMERAGetWbMode.....	138
4.66 CAMERAIsWbWinVisible.....	139
4.67 CAMERAGetWbWindow.....	139
4.68 CAMERASetWbWindow.....	140
4.69 CAMERAImageOverlay.....	141
4.70 CAMERASetCrossLine.....	142
4.71 CAMERAGetCrossLine.....	143
4.72 CAMERASetGain.....	144
4.73 CAMERAGetGain.....	145
4.74 CAMERASetGamma.....	146
4.75 CAMERAGetGamma.....	147
4.76 CAMERASetSaturation.....	148
4.77 CAMERAGetSaturation.....	149
4.78 CAMERASetContrast.....	149
4.79 CAMERAGetContrast.....	150
4.80 CAMERASetFrameSpeed.....	151
4.81 CAMERAGetFrameSpeed.....	152
4.82 CAMERASetAntiFlick.....	153
4.83 CAMERAGetAntiFlick.....	153
4.84 CAMERAGetLightFrequency.....	154

4.85 CAMERASetLIGHTFREQUENCY.....	155
4.86 CAMERASetTRANSPACKLEN.....	156
4.87 CAMERAGetTRANSPACKLEN.....	157
4.88 CAMERAWritesN.....	158
4.89 CAMERAReadsN.....	159
4.90 CAMERAsavePARAMETER.....	159
4.91 CAMERALoadPARAMETER.....	160
4.92 CAMERAReadPARAMETERFROMFILE.....	161
4.93 CAMERAGetCurrentPARAMETERGROUP.....	162
4.94 CAMERAEnumerateDEVICE.....	163
4.95 CAMERAGetCAPABILITY.....	164
4.96 CAMERASetTRIGGERCOUNT.....	164
4.97 CAMERAGetTRIGGERCOUNT.....	165
4.98 CAMERAGetTRIGGERDELAYTIME.....	166
4.99 CAMERASetTRIGGERDELAYTIME.....	167
4.100 CAMERASoftTRIGGER.....	168
4.101 CAMERASetTRIGGERMODE.....	168
4.102 CAMERAGetTRIGGERMODE.....	169
4.103 CAMERASnapToBuffer.....	170
4.104 CAMERAGetRESOLUTIONFORSNAP.....	171
4.105 CAMERASetRESOLUTIONFORSNAP.....	171
4.106 CAMERASetPARAMETERTARGET.....	172

4.107 CAMERAGetParameterTarget.....	173
4.108 CAMERASetParameterMask.....	174
4.109 CAMERARstTimeStamp.....	175
4.110 CAMERASaveUserData.....	175
4.111 CAMERALoadUserData.....	176
4.112 CAMERAGetFriendlyName.....	177
4.113 CAMERASetFriendlyName.....	178
4.114 CAMERASdkGetVersionString.....	179
4.115 CAMERAGetEnumInfo.....	179
4.116 CAMERASetIOState.....	180
4.117 CAMERAGetIOState.....	181
4.118 CAMERASetISPOutFormat.....	182
4.119 CAMERAGetISPOutFormat.....	183
4.120 CAMERAGetErrorString.....	183
4.121 CAMERARECONNECT.....	184
4.122 CAMERAConnectTest.....	184
4.123 CAMERASetTriggerDelayTime.....	185
4.124 CAMERAGetTriggerDelayTime.....	186
4.125 CAMERASetStrobeMode.....	186
4.126 CAMERAGetStrobeMode.....	187
4.127 CAMERASetStrobeDelayTime.....	188
4.128 CAMERAGetStrobeDelayTime.....	189

<i>4.129 CAMERASetSTROBEPULSEWIDTH.....</i>	<i>190</i>
<i>4.130 CAMERAGetSTROBEPULSEWIDTH.....</i>	<i>190</i>
<i>4.131 CAMERASetSTROBEPOLARITY.....</i>	<i>191</i>
<i>4.132 CAMERAGetSTROBEPOLARITY.....</i>	<i>192</i>
<i>4.133 CAMERASetEXTTRIGSIGNALTYPE.....</i>	<i>192</i>
<i>4.134 CAMERAGetEXTTRIGSIGNALTYPE.....</i>	<i>193</i>
<i>4.135 CAMERASetEXTTRIGDELAYTIME.....</i>	<i>194</i>
<i>4.136 CAMERAGetEXTTRIGDELAYTIME.....</i>	<i>194</i>
<i>4.137 CAMERASetEXTTRIGJITTERTIME.....</i>	<i>195</i>
<i>4.138 CAMERAGetEXTTRIGJITTERTIME.....</i>	<i>196</i>
<i>4.139 CAMERAGetEXTTRIGCAPABILITY.....</i>	<i>196</i>
<i>4.140 CAMERAALIGNMALLOC.....</i>	<i>197</i>
<i>4.141 CAMERAALIGNFREE.....</i>	<i>198</i>

5 SDK INTERFACE FUNCTIONS ARE EXPLAINED BY FUNCTION CLASSIFICATION 199

<i>5.1 CAMERA INITIALIZATION AND DE-INITIALIZATION.....</i>	<i>199</i>
<i>5.2 SAVE AND LOAD CAMERA PARAMETERS.....</i>	<i>203</i>
<i>5.3 CAMERA FETCHING (ACTIVE FETCHING OR CALLBACK FUNCTION METHOD).....</i>	<i>205</i>
<i>5.4 PREVIEW IMAGE WITH INDICATOR.....</i>	<i>209</i>
<i>5.5 ADJUST CAMERA IMAGE BRIGHTNESS (SET EXPOSURE TIME).....</i>	<i>210</i>
<i>5.6 SWITCH BETWEEN DIFFERENT RESOLUTIONS AND CUSTOM RESOLUTIONS (ROI FUNCTION).....</i>	<i>212</i>
<i>5.7 SET THE CAMERA'S CONTRAST, GAMMA, SATURATION, SHARPNESS AND OTHER ISP PARAMETERS.....</i>	<i>215</i>

5.8 CONVERT COLOR CAMERA TO MONOCHROME CAMERA.....	216
5.9 MULTIPLE CAMERAS ARE USED AT THE SAME TIME, HOW TO ESTABLISH CAMERA CORRESPONDENCE.....	218
5.10 IMAGE FLIP MIRROR AND ROTATE FUNCTION.....	221
5.11 SUPERIMPOSE TEXT ON IMAGES.....	222
5.12 SAVE PICTURE AND VIDEO FUNCTION.....	222
5.13 SET THE IMAGE BIT DEPTH OUTPUT BY THE CAMERA.....	223
5.14 SET THE PIXEL FORMAT OF THE IMAGE (8-BIT GRAYSCALE, 24, 32, 48-BIT COLOR).....	224
5.15 PROCESS THE ORIGINAL RAW IMAGE.....	227
5.16 ETHERNET PORT CAMERA USES API TO DYNAMICALLY SET IP, GATEWAY, AND SUBNET MASK.....	228
5.17 SET THE FRAME RATE OF THE CAMERA.....	228
5.18 USE SOFT TRIGGER OR HARD (EXTERNAL) TRIGGER FUNCTION.....	229
5.19 DEBOUNCE THE SIGNAL IN EXTERNAL TRIGGER MODE.....	230
5.20 EXTERNAL TRIGGER MODE SET TRIGGER DELAY TIME.....	231
5.21 DETECT CAMERA DISCONNECTION AND AUTOMATIC RECONNECTION.....	231
5.22 READING AND WRITING OF CAMERA SERIAL NUMBER.....	232
5.23 READ AND WRITE CUSTOM DATA IN THE CAMERA.....	232
5.24 GET THE STRING DESCRIPTION INFORMATION CORRESPONDING TO THE ERROR CODE.....	233
6 GIGE VISION AND USB3 VISION XML FILE DEFINITION EXPLANATION	234
7 HALCON DEVELOPMENT GUIDE.....	235
7.1 DEVELOPMENT IN HDEVELOP.....	236
7.2 HALCON DEVELOPMENT IN C/C++, VB, C#.....	245

7.3 DEVELOP WITH MULTIPLE CAMERAS AT THE SAME TIME.....	248
8 LABVIEW DEVELOPMENT GUIDE	250
8.1 DEVELOP WITH NI MAX.....	250
8.2 DEVELOPMENT BASED ON DLL FILE CALL METHOD.....	252
8.3 USE MULTIPLE CAMERAS IN LABVIEW.....	253
8.4 THE DISTINCTION OF LABVIEW MULTI-CAMERA.....	254
9 RELEASE THE CAMERA INSTALLATION FILE AFTER INTEGRATION	257
10 TECHNICAL SUPPORT	260

Introduction

This document is provided to users who use our company's industrial cameras for secondary development. It provides a detailed description of the dynamic link library functions used in system development and provides quick programming guidance, In order to be able to integrate the camera into the user's system in a short time.

statement

The copyright, copyright and intellectual property rights of this software belong to our company and are subject to the "Copyright Law of the People's Republic of China", "Computer Software Protection Regulations", "Intellectual Property Protection Regulations" and relevant international copyright treaties, laws, regulations, and other knowledge Protection of property rights laws and treaties. Any unit or individual cannot use, modify, or redistribute any part of this software without our company's authorization, otherwise it will be regarded as an illegal infringement, and our company reserves the right to pursue its responsibility according to law. This clause also applies to text, pictures, tables, etc., to which the company has full rights.

The information and other similar contents described in this document are for your convenience only, and they may be replaced by updated information without notice by the company. It is your responsibility to ensure that the application meets the technical specifications. The company does not make any express or implied, written or oral, statutory or other forms of declarations or guarantees for this information, including but not limited to declarations or guarantees regarding its usage, quality, performance, marketability or applicability for specific purposes.

1 Overview

1.1 File structure

In the secondary development, the library files that need to be used directly are located in the SDK folder of the installation directory, and are divided into two development packages, 32-bit and 64-bit. The 64-bit SDK development files are located in the SDK/X64 folder. Compared with the 32-bit SDK files, the 64-bit SDK file names end with _X64.

The MVCAMSDK.DLL file in the SDK directory is the camera's SDK dynamic link library, which provides all the interface functions of the camera. VC/C++, VB, VB.net, Delphi, C# routines all reference this library file. At the same time, for ease of reference, MVCAMSDK.DLL will be copied to the System32 directory of the system during installation (in the syswow64 directory for WIN64 systems). During your secondary development, you can directly use the file name to access without caring about the path where the MVCAMSDK.DLL file is located. The system will automatically find the file in the System32 directory.

If you need to publish your own installation package, please refer to the method in Chapter 7.

The development routines are located in the following subdirectories of the installation directory:

- Demo/VC++. Based on the routines developed by VC++, the project files of VC6 and VS2010 are provided.
- Demo/VB6. Routines developed based on VB6.
- Demo/VB.net. Routines developed based on VB.net.
- Demo/Delphi6. Routines developed based on Delphi6.
- Demo/C#. A routine developed based on C# (VS2010).
- Demo/VC++/OpenCV. OpenCV example based on VS2010.
- Demo/LabView. Development routine based on ActiveX and DLL interface call.

Among them VC++ provides Basic, BasicEx, Advanced, OCX, MultiCamera, MultiExposure multiple routines, VB6, Delphi6, C# only provides Basic one routine. The two routines of Basic and Advanced cover most of the camera function operations. You can choose different routines according to your development needs to understand how to use our SDK. For the respective characteristics and scope of application of the Basic and Advanced routines, please refer to Chapter 2.

1.2 *Development environment*

SDK is a standard C language interface dynamic link library, which can be loaded by development tools such as C, C++, C#, VB, Labview, and Delphi. Currently, we provide routines based on VC++

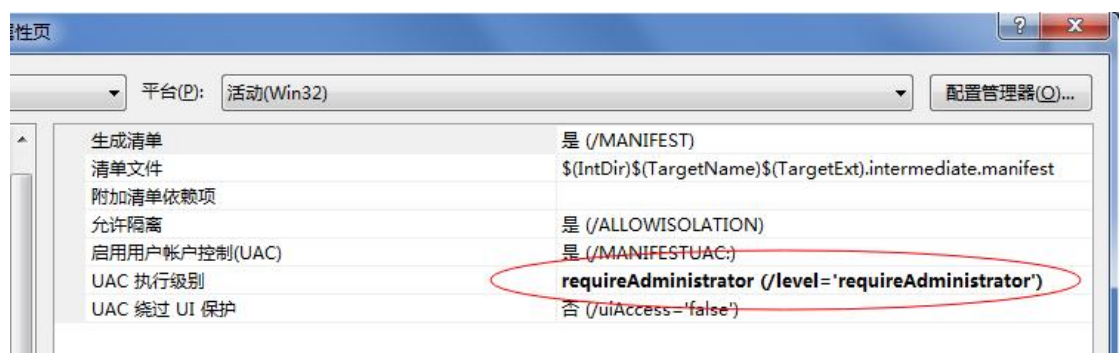
6.0, VS2010, VB6.0, VB.net, Delphi6, C#, Labview (8.6 9.0 2010 2011 2012), Halcon (9, 10, 11, 12), OpenCV (2.4) .

For development tools that use VC6 or above, such as VS2003--VS2013, etc., you can directly open the DSP or DSW file in the DEMO project of VC++6.0, and then follow the prompts to perform the project conversion.

1.3 Matters needing attention (common problems)

- Since the SDK needs to access the registry, under WIN7 and WIN8 systems, if you log in as a non-administrator user, you need to run the camera's DEMO program with administrator privileges to access the camera normally, otherwise the camera initialization will report an error like -13 . The common practice is to log in to the system as an administrator, or to apply for administrator rights in a program you develop. For the MFC project, set the following settings in the project settings -> linker -> manifest

file:



If you use development languages such as DELPHI, C#, VB6, you can also obtain administrator rights through similar methods.

- Switch between installation packages of different versions. We have maintained API interface compatibility for different versions of the SDK, but the camera's kernel driver may need to be manually switched between different software versions. If the camera initialization fails after installing the new version, you need to manually switch to the device manager Update the camera kernel driver once.

1.4 xplanation of commonly used words in industrial cameras

1.4.1 Line scan camera and area scan camera

A line scan camera is a camera that uses a line scan image sensor. The linear image sensor is mainly based on CCD, the data of one line can reach several K or even tens of K, but the height is only a few pixels, and the line frequency is very high, which can reach tens of thousands of lines per second. It is suitable for very high precision and wide format scanning.

Area scan cameras are cameras that use area scan image sensors. Both CMOS and CCD have area scan cameras. The resolution of an area

scan camera is much smaller than that of a line scan camera. However, the picture is the sensing of the whole frame, imaging at one time. The pixel height will be much larger than that of a line scan camera. At the same time, the program development is relatively simple, and a whole image can be obtained at a time without stitching each row of data.

Currently, most of the applications still use area scan cameras. In terms of price, line scan cameras are expensive, often several to several tens of times higher than area scan cameras.

1.4.2 Rolling shutter and global shutter

Rolling shutter and global shutter are mainly for area scan cameras.

Rolling shutter cameras, when the light is exposed, are processed line by line, starting from the first line, scrolling to the last line of light exposure, the exposure start time of each line is different, and the image data is output while exposing; For cameras with global shutters, the entire area array starts and ends at the same time during exposure. After the end, the entire frame of data is read out at once.

Due to the limitation of the rolling shutter principle, the rolling shutter camera is not suitable for shooting high-speed moving pictures. Compared with the global shutter camera, when the rolling shutter camera shoots the moving picture, each line of the image will produce a certain displacement deviation, which will eventually cause the image

Distorted. Here to explain, this kind of distortion is not smearing. Many people mistake this phenomenon as smearing. The smearing is caused by the object moving too fast and the camera's exposure time is set too long. , The smear will cause the image to be blurred, and the rolling shutter will cause the distortion, but the definition of each line of the image will not be affected.

1.4.3 Hardware trigger (external trigger)

In the normal mode, after the camera starts to work, it is continuously collecting images. After collecting this frame, it immediately starts the next frame collection, and so on.

In some industrial applications, it is not necessary for the camera to collect images continuously, but to wait for a specific event to occur before the camera collects a frame of images and obtains the processed result. In this case, you need to use hardware (external) Trigger mode. Industrial cameras that support this working mode will have a dedicated trigger connector on the camera, which is generally an aviation connector ranging from 4 to 12 cores. After entering the trigger mode, the camera will wait for a valid signal. The validity of the signal can be set by software, such as the high and low level mode or the upper and lower edge jump mode. If there is no valid signal, the camera will not output anything Image data. When you find that the program captures timeout,

please check whether the camera is set to trigger working mode on the interface.

1.4.4 Optical size (target surface)

Optical size refers to the size of the camera's photosensitive area. Common sizes 1/4 ", 1/3" 1 / 2.5 "1 / 2.3" 1/2 ", 2/3", it is necessary to select the appropriate lens in accordance with this size, in order to achieve an effect matching .

1.4.5 Frame Memory and Cache

An industrial camera with a cache function means that the camera has a certain ability to cache image data, but does not have the ability to cache the entire frame of image data. Therefore, when the transmission bandwidth is not enough or the transmission line is not reliable and stable, It may cause buffer overflow, and the image frame cannot be reconstructed, resulting in frame loss or no picture at all.

The industrial camera with frame memory function means that the camera has the ability to save the complete image frame inside the camera. Therefore, when the transmission bandwidth is not enough, or the transmission line is not reliable and stable enough, the camera with frame buffer function can still resume the transmission, and the image frame can be reconstructed on the PC side.

Industrial cameras will definitely have a buffer function, but not necessarily a frame memory function. A typical example to illustrate the benefits of a frame memory camera: When 16 cameras are connected at the same time to take pictures in the external trigger mode, after receiving the trigger signal, the 16 cameras start to expose at the same time, and the images are first saved in the camera, and then the PC can read the 16 cameras in any order. There is no need to worry about whether the reading time and bandwidth are sufficient. If there is no frame memory function, the images of these 16 cameras may all be lost. But the camera without frame memory also has its advantages, cost-effective, simple structure, suitable for one computer to connect 1 or 2 cameras.

2 Quick Development Guide

2.1 Overview of camera operation process

For your rapid and accurate development, please be patient and finish reading this chapter.

We recommend that you follow the procedure below to operate the camera (some of the steps are optional and have been marked):

—、Load the dynamic link library file MVCAMSDK.DLL of the SDK.

You can use dynamic or static loading in two ways.

- If you use C/C++ for development, reference the CameraApi.h header file (located in the SDK/DEMO/VC++/include of the installation directory) and MVCAMSDK.lib library file (located in the SDK folder of the installation directory) in the project, Then you can directly reference the interface functions in the SDK in the project, but MVCAMSDK.DLL must be placed in the same directory as your application or under the system32 directory of the system. When placed in other directories, the system must be set Environment variable (PATH).
- If you use VB for development, you can load it through a similar method in our VB6 routines, directly define the

interface function of the SDK and specify the DLL file it refers to. SDK\Demo\VB6\Module\CameraApi.bas module includes all SDK interfaces, and each function interface exported in MVCAMSDK.DLL is mapped into a function that can be called by VB.

- If you use Delphi for development, you can load it through a similar method in our Delphi6 routines, directly define the interface function of the SDK and specify the DLL file it references. The SDK\Demo\Delphi6\Units\CameraApi.pas unit includes all SDK interfaces, and each function interface exported in MVCAMSDK.DLL is mapped into a function that can be called by Delphi.
- If you use C# for development, you can load it in a similar way in our C# routines. The process of loading MVCAMSDK.DLL in C# is similar to that of VB and Delphi. For ease of use, our C# DEMO provides 2 projects, one is MVSDK, this project is specifically used to define the data structure of the development package and load the API function of the development package; The other is the Basic project, which implements a routine that integrates preview, capture, and camera settings. The Basic project indirectly accesses MVCAMSDK.DLL by calling the code in the MVSDK project.

- 二、 Initialize the SDK. After the SDK is loaded, please call the CameraSdkInit function to initialize before using other interfaces.
- 三、 Enumerate devices. Call the CameraEnumerateDevice function to enumerate the devices and get the device list of the camera currently connected to the PC. The list includes the device name (can be modified by yourself), version number, unique serial number, camera model and other information.
- 四、 Initialize the device. According to the camera device enumeration information obtained in the third step, call the CameraInit function to initialize the specified camera and get the camera handle. If you need to open multiple cameras at the same time, use the device names of multiple cameras to call CameraInit multiple times to obtain the handles of multiple cameras. For subsequent operations on the cameras, you need the camera handles obtained at this time to specify the camera object for operation.
- 五、 Let the SDK enter the image acquisition mode. Call the CameraPlay function to let the camera enter the working mode, and the SDK starts to receive images from the camera.
- 六、 Grab the image. The SDK provides two ways to obtain image data. The efficiency of these two methods is the same. The

bottom layer uses a zero-copy mechanism to improve efficiency.

You can choose one according to your development habits.

- Actively call `CameraGetImageBuffer` to obtain a frame of image data. This function will get the address of a buffer used in the SDK to receive image data, and frame header information. At the same time, this function can set a timeout, and if no image is obtained within the specified time (the thread will be suspended), the timeout will be returned.
- In the third step, after initializing the camera, call `CameraSetCallbackFunction` to set a callback function. This method is passive. Only after a valid image data frame is received inside the SDK, the callback function you set will be called to pass the received image data frame.

Note: You can also use the above two methods to obtain images at the same time, but you cannot call `CameraGetImageBuffer` in the callback function set by `CameraSetCallbackFunction` to obtain images again, which will cause deadlock.

七、Process images. The image frame obtained in the previous step is the original format of the camera output. For most models of our company, the original output is in Bayer format or YUV format. These format information will be automatically added to the frame header information. Call `CameraImageProcess` to

obtain image processing Effects, such as color gain adjustment, white balance correction, saturation, LUT transformation, noise reduction, etc., and convert the raw data in YUV or Bayer format to 24BIT bitmap format (RGB888).

- 八、 Superimpose additional content such as crosshairs, auto exposure reference window, white balance reference window, etc. (If you do not need to superimpose information during development, this step can be skipped). Calling the `CameraImageOverlay` function, the crosshairs, auto exposure reference window, and white balance reference window set to the visible state will be superimposed on the input image. The input of `CameraImageOverlay` must be in bitmap format. We recommend that you call `CameraImageOverlay` function after calling `CameraImageProcess` to get the bitmap format.
- 九、 Save the image or display the image (if you do other processing on the image during your development, you do not need to save the image as a file or display it, this step can be skipped).
- If you need to save an image to a file, call the `CameraSaveImage` function to save the image after the sixth, seventh, or eighth steps. The SDK supports four methods: PNG, BMP, JPG, and raw data. If you want to save the original data, you should call the `CameraSaveImage` function after the

sixth step;If you save in BMP, PNG, JPG format, you should call the CameraSaveImage function after the seventh step;If you need to superimpose the crosshairs and the position of the auto exposure and white balance reference window while saving to BMP, PNG, JPG format, then you can call the CameraSaveImage function after the eighth step.

- If you need to display images, there are two ways:
 - a. Realize the image display according to the development environment, such as using OpenGL, DirectDraw, Windows GDI, etc. to realize the image display.
 - b. Use the display interface encapsulated in our SDK to display images. After initializing the camera in the fourth step, call the CameraDisplayInit function to initialize the display interface. This function needs to pass in the handle of the display control (HWND type), which is only suitable for VC/C++, VS, VB, VB.NET, Delphi, C#, etc. Windows GDI development interface tools have certain limitations,But if you meet the conditions of use, we still recommend that you use our packaged display interface.

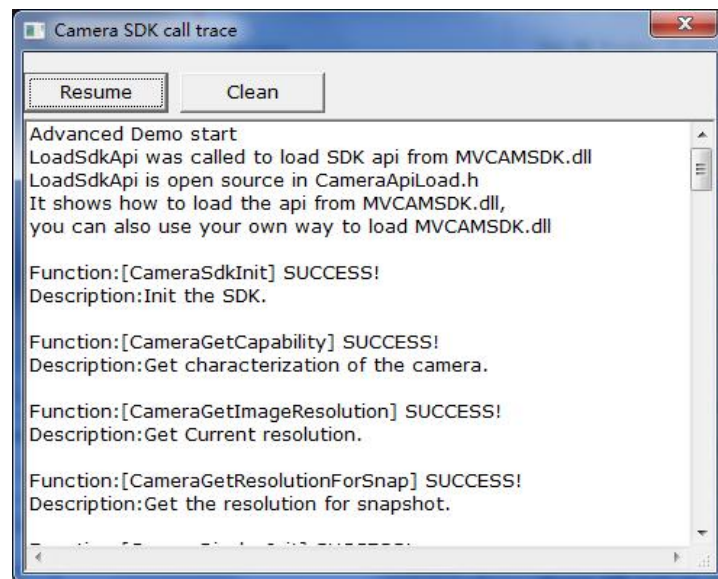
十、 Turn off the camera before exiting the program (**deinitialization is very important, if you directly close the program without deinitializing the camera, the program may report a memory error**).When closing the camera, call the CameraUnInit function.

2.2 Development routine

In order for you to develop and use our SDK more quickly, we have prepared a number of routines for you.The characteristics of each routine are different, and the people suitable for development are also different. You can choose different routines to start with according to your needs.This document describes only the VC routines, and the routine descriptions of other development languages are similar to VC routines.

The development routines are located in the Demo folder of the installation directory. According to the different development languages, they are divided into VC++, VB6, C#, VB.NET, Delphi6 and other directories.Currently, it provides several routines such as Basic, Advanced, MultiCamera, OCX, UserDataTest, TriggerAndStrobe, ImageFormat&Saving, etc. These routines are mainly based on VC++,**Development languages such as C#/Delphi/VB6/VB.NET can refer to the SDK interface called by the VC++ example to realize all functions. The SDK provided by each development language is the**

same.At the same time, in order for you to see the situation of calling the camera SDK interface in the routine more intuitively, we have specially made a tracking information panel called by the camera SDK. After the routine runs, the panel will automatically display and record the entire process. Calling information of SDK interface functions. As shown below:



The features of these routines are as follows:

- The Basic routine uses the SDK interface to create the configuration window of the camera, similar to the device property page in the DirectShow interface. In this way, you basically do not need to develop the UI interface yourself, or even carefully understand the use of the camera-related SDK interface, which can greatly save your development and debugging time. At the same time, our camera parameters are saved in a binary file format. After debugging the parameters

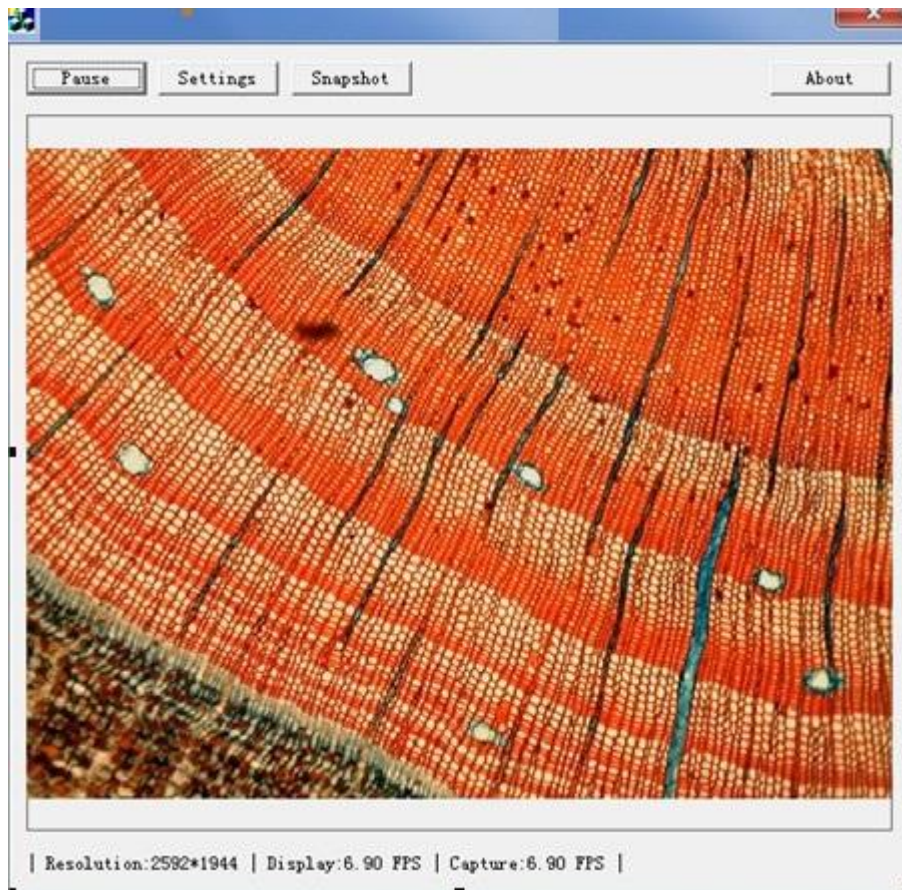
through the software interface on the development machine, the parameters are saved as a file, which can be published to the target machine together, eliminating the need for complicated programming initialization parameters work, **It is strongly recommended that you use this method for development.**

- The Advanced routine covers most of the SDK interfaces, demonstrates the usage conditions and methods of these interface functions, and uses VC++6.0 to make a configuration interface for camera properties. If the Basic routine cannot fully meet your design requirements, you can refer to the Advanced routine to proceed.
- The MultiCamera example demonstrates how to develop multiple cameras. You can connect 1 to 4 cameras of the same model or different models or even different interfaces (USB2.0 USB3.0 GIGE) for simultaneous preview.
- The OCX example demonstrates how to develop the camera through the ActiveX control provided by us. Similarly, the OXC control method also supports multiple cameras.
- The UserDataTest example demonstrates how to write or read custom data to the camera. These data are stored in the camera and are still valid after power failure. Can be used to read and write ID, application data, and bundled data.

- The TriggerAndStrobe example demonstrates the development of external trigger and flash interface.
- The ImageFormat&Saving example demonstrates how to set the data format of the SDK capture and how to save the image.
- The SnapshotOnPreview routine demonstrates how to capture a full-resolution image when previewing at a high speed at a small resolution.
- The ROI routine demonstrates how to customize the camera image resolution when the preset resolution cannot meet the project requirements. For some models, the camera can support the simultaneous transmission of multiple ROIs. The cropping is done by the camera hardware, and the software-side SDK is automatically stitched into the whole picture. After removing the unnecessary transmission area, the frame rate can be effectively increased. For example, after using this technology for a 5 million pixel USB2.0 camera, the frame rate can be increased to 15 frames when the 5 million pixel is uncompressed.
- GPIO routines, demonstrate how to operate the input and output IO that comes with the camera.

2.2.1 Basic routine

This example is located in the Demo/VC++/Basic folder of the installation directory. After the routine runs, the interface is as shown in the figure below:

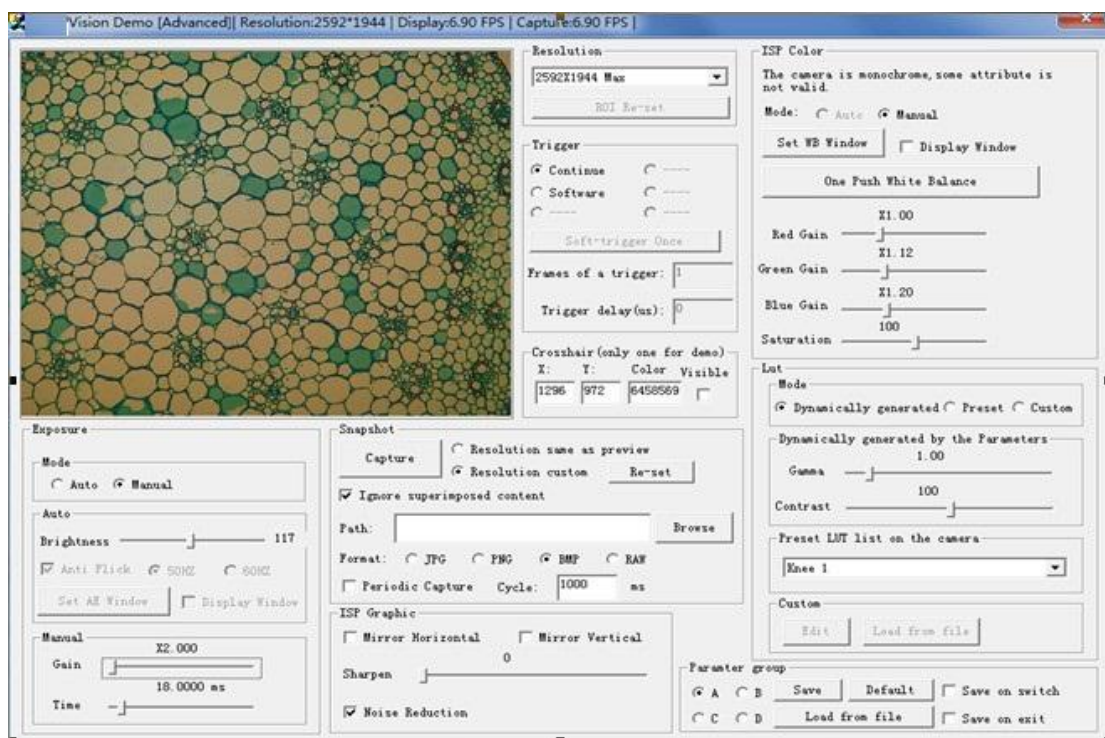


- Pause button. Used to pause camera work.
- Settings button. Click to display the configuration window of the camera. The configuration window is generated by the SDK and made in an unusual style.
- Snapshot button. After clicking, use the SDK's capture interface to obtain a picture and save it to a local file.

- About button. After clicking, the description of the routine will pop up.
- The bottom status bar. From left to right, the current preview resolution, display frame rate, and capture frame rate are displayed in sequence.

2.2.2 Advanced routines

emo/VC++/Advanced folder of the installation directory. After the routine runs, the interface is as shown in the figure below:



- On the title bar of the interface, the resolution, display frame rate and acquisition frame rate are displayed.
- Exposure module, a collection of exposure-related control functions.

- Resolution module, a collection of resolution-related control functions, including custom resolution (ROI).
- trigger module, which integrates the control function of trigger mode. Continue means continuous output mode, and Software means software trigger mode.
- Crosshair module, integrated crosshair control function.
- Snapshot module, which integrates the control functions of the snapshot mode, and demonstrates how to take pictures automatically at regular intervals.
- ISP Graphic module, Set graphics-related control functions, including mirroring, sharpening, noise reduction and other functions.
- ISP Color module,It integrates image color-related control functions, including white balance, RGB three-channel digital gain, and saturation control lights.
- Lut module,The control function of the integrated search conversion function. The SDK supports three search table conversion methods, namely, generating LUT tables by adjusting parameters, using camera preset LUT tables and custom LUT tables.

2.2.3 MultiCamera routines (use multiple cameras at the same time)

This routine is located in the Demo/VC++/MultiCamera folder of the installation directory. It demonstrates how to use the SDK to develop a computer that connects 1 to 4 cameras at the same time and displays the preview images of the 4 cameras. If you need to connect more cameras, please refer to the method in this example to expand, which can support more than 32 cameras at the same time.

2.2.4 OCX routines (ActiveX)

This example is located in the Demo/VC++/OCX folder of the installation directory. It demonstrates how to use the camera's OCX control for application development. OCX mode can also use multiple cameras at the same time. In the example, you can connect 1 or 2 cameras for demonstration. Need to connect more cameras to use at the same time, please refer to the method in the example to expand.

2.2.5 MultiExposure routine (a camera with two exposure channels preview, wide dynamic effect)

This routine is located in the Demo/VC++/MultiExposure folder of the installation directory, and demonstrates how to use one camera to achieve continuous preview of different exposure values and brightness gains under two channels. Applying the same way, you can be extended

to four or more of the preview channel to obtain images of different exposure effect. The picture below is taken using a 1.3 million color camera with a USB2.0 interface. In this double exposure mode, each channel can reach up to 10 frames.

typical application: When shooting metal, glass and other reflective materials, this method can be used to obtain images of different brightness for analysis. After this method is extended, more than 2 exposure effects can be obtained.



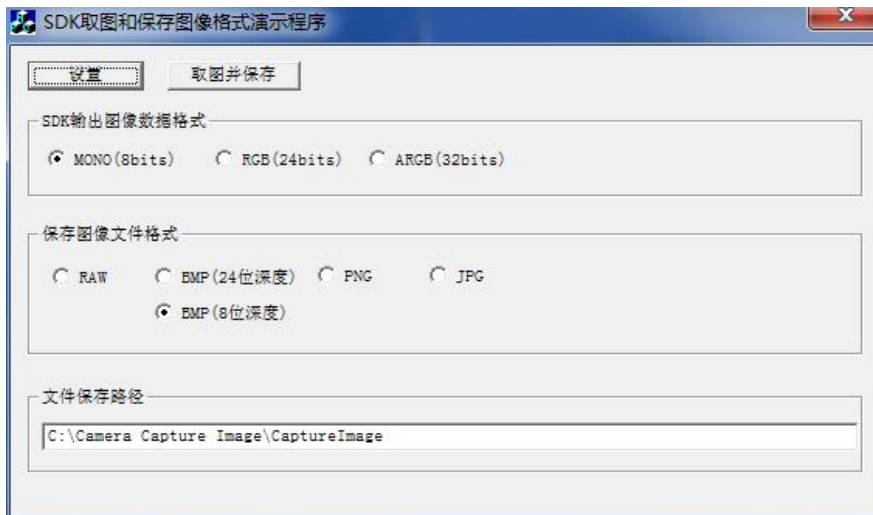
2.2.6 ImageFormat&Saving routines (Gray, RGB24, RGB32 format settings)

This routine is located in the Demo/VC++/ImageFormat&Saving folder of the installation directory, and demonstrates how to set the SDK image processing output format and image file saving operation mode.

At present, our SDK supports 8bit, 24bit, and 32bit image format output, which can meet most applications. It provides 5 image formats to save image files in RAW format, BMP 8, 24-bit depth format, PNG and JPG. The RAW format is the original output format of the camera, which may be 8, 10, 12, and 16bit original data; BMP 8-bit depth image files need to configure the SDK output format to 8bit mode; BMP 24-bit depth, JPG, PNG format save, need The SDK output format is 24bit mode.

typical application: Monochrome cameras output 8-bit grayscale format, which will increase the speed of image processing; color cameras output RGB32 format, the address is aligned by 4 bytes, which is convenient for the visual library to use assembly instructions for hardware acceleration.

The interface after the DEMO runs is shown in the figure below.



2.2.7 TriggerAndStrobe routine (trigger and flash signal control)

The example is located in the Demo/VC++/ TriggerAndStrobe folder of the installation directory. It demonstrates how to set the camera's trigger signal and flash signal. **Note that this example is invalid for models that do not support external triggering.**

At present, our camera supports 4 signal trigger methods, namely rising edge trigger, falling edge trigger, high level trigger, and low level trigger. If a mechanical switch is used, it is recommended to use level trigger mode and set debounce at the same time Time, to filter out the interference signal when the mechanical switch value jumps, if it is an electronic switch, there is no limit; The flash signal supports two modes: full-automatic and semi-automatic. Full-automatic is to automatically generate the signal waveform when the camera is exposed; the

semi-automatic mode is to use the program interface to control the level of the flash signal terminal.

typical application:The flash is output synchronously, and the flash is activated only when the camera is exposed; manual programming controls the pin level of the flash light output to control the switch of the infrared light (normally on or normally closed); use trigger to take pictures with multiple cameras at the same time; use Trigger to take a photo at a specified time, such as when a button is pressed.

The running interface of this routine is shown in the figure below.



2.2.8 UserDataTest routine (read and write custom data to the camera)

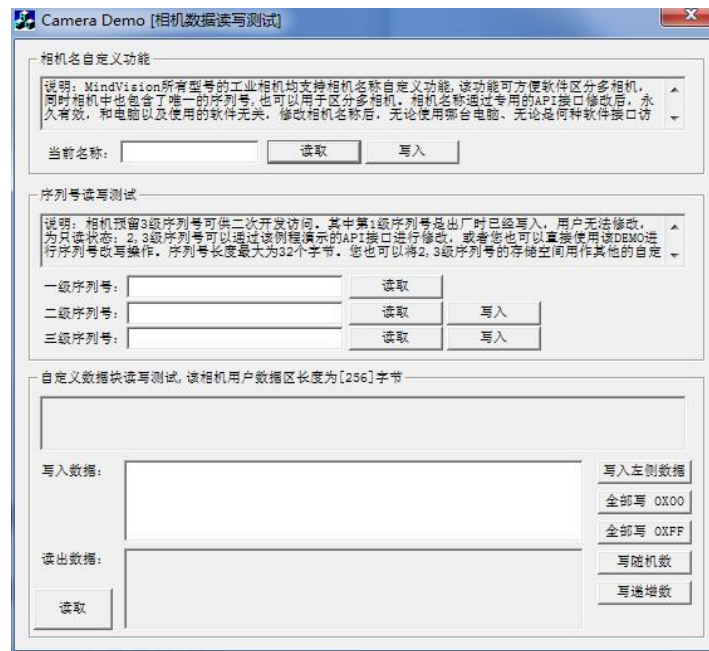
This routine is located in the Demo/VC++/UserDataTest folder of the installation directory, and demonstrates how to write custom data into the camera and read and modify the camera device name.

The modification of the camera name will directly affect the enumeration list of the camera. After the name is modified, when the interface of Directshow, Twain, Halcon, Labview, etc. access the camera, the name will be modified, and it will be fixed in the camera and will be permanently effective. This function is mainly for the convenience of distinguishing when connecting multiple cameras at the same time. After the name of each camera is modified, there is no need to worry about which interface the camera is connected to. Even if the computer host is changed, the name of the camera will not be affected.

Reading and writing of custom data is for the convenience of storing program data in the camera during secondary development; or writing some special data to realize the binding between the program and the camera.

typical application:Read and write camera calibration data; read and write special binding data to realize the association of software and hardware.

The interface of the program is shown in the figure below:



2.2.9 SnapshotOnPreview routine (small resolution high-speed preview, large resolution photo)

This example is located in the Demo/VC++/SnapshotOnPreview folder of the installation directory.

Shows how to use the capture function to capture an image of a specified resolution during continuous preview and save it as a BMP file(The image captured by the capture channel is stored in the memory and can be processed directly or displayed separately. In this example, the image is captured and then saved in the file).

The capture resolution, preview resolution settings, capture image acquisition and preview image acquisition have their own independent interface functions, which do not affect each other; other parameters, such as image brightness, color, etc., are shared between preview and

capture channels. After setting the resolution of the captured image and capturing during continuous preview, the SDK will automatically switch to the resolution. After the capture is successful, it will automatically switch back to the preview resolution.

typical application:, the frame rate can be increased by more than 4 times, and the video fluency is good; large-resolution capture can obtain full-frame images and perform high-precision processing.

The running effect of this routine is shown in the figure below. The preview resolution can be set in "Camera Settings":



2.2.10 RawTransTest routines (convert offline RAW files to BMP, JPG files)

This example is located in the Demo/VC++/RawTransTest folder of the installation directory,

Shows how to use the SDK to convert the saved camera RAW photo files into BMP or JPG files.

When converting RAW files, you need to provide the corresponding camera configuration file (the camera configuration file is in the Camera/Configs folder of the installation directory, which saves the parameter information of the camera when it is running). Through the configuration files of different cameras, RAW photo files taken by different cameras can be accurately restored into BMP, JPG files.

typical application:After single-camera or multi-camera high-speed saving of RAW files to SSD solid state drives, the RAW files are later converted to BMP and JPG for processing, such as panoramic synthesis and other applications.

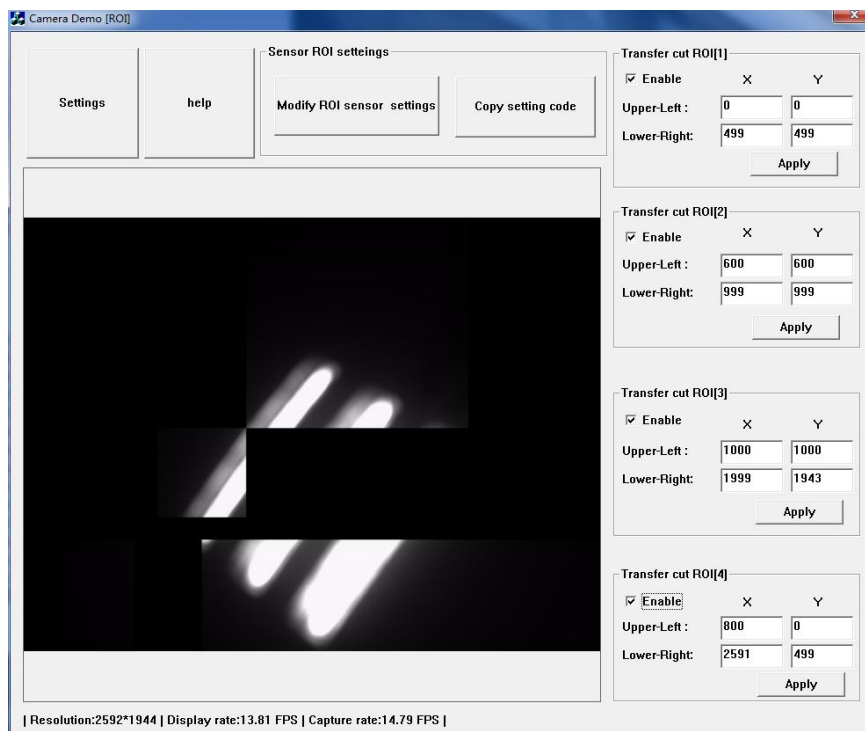
2.2.11 LineScan routine (using line scan mode)

For some types of area scan cameras, such as MV-U130M and MV-UB130M, (1.3 million pixel monochrome), support the output of one line of ROI resolution, such as 1280X1, At this time, the area scan camera can be used as a low-performance line scan camera with a line speed of about 1700 lines per second.

This routine is located in the Demo/VC++/LineScan folder of the installation directory. For cameras that do not support line array ROI

resolution output, an error will be reported when the routine runs. After the routine is run, the image data will be read line by line and displayed after being spliced into a whole picture, so this routine is suitable for shooting cylindrical rolling objects.

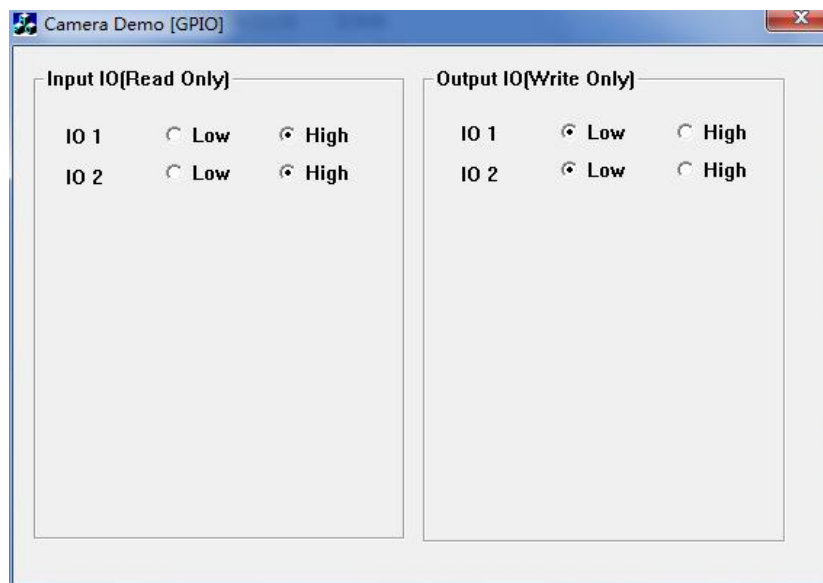
2.2.12 ROI routine (how to customize camera image size)



As shown in the figure above, the ROI example demonstrates a 5 million pixel USB2.0 camera. When the multi-region ROI function is turned on, under the image size of 2592X1944, the frame rate of 14 frames per second can be reached through the USB2.0 interface. The parts that are not involved in the transmission are filled with black, and the four cropped ROI regions can be dynamically modified.

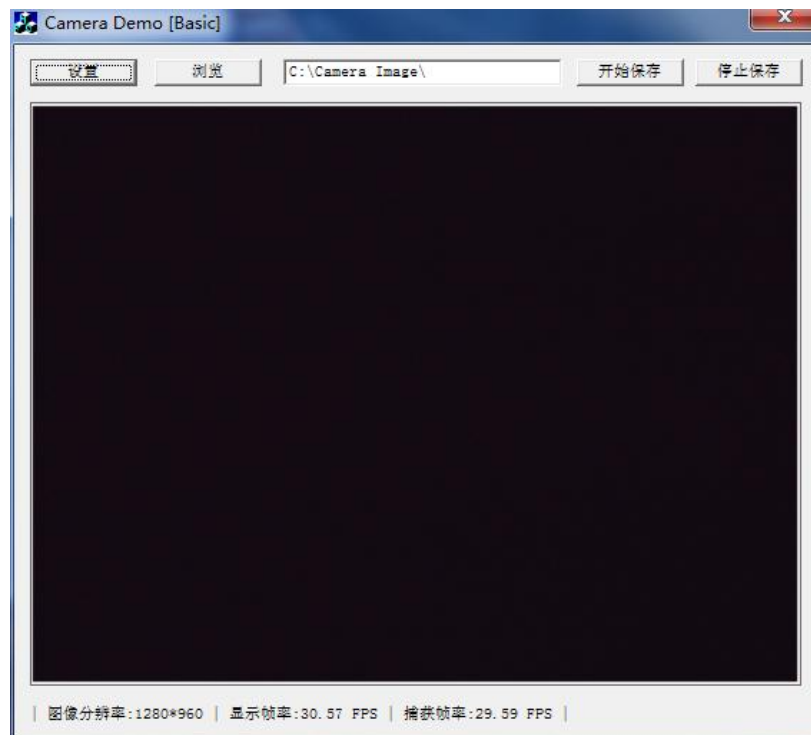
Typical application: When the field of view that needs to be shot is very large and it is dispersed into multiple areas, consider using this function, which can effectively save transmission bandwidth, reduce CPU usage, and increase image transmission frame rate.

2.2.13 GPIO routines (only valid for models with GPIO)



As shown in the figure above, the GPIO example demonstrates how to operate the GPIO port that comes with the camera. When the input IO status changes, the interface will automatically update; when the output IO is operated on the interface, the camera output IO level will change accordingly.

2.2.14 SaveFile routine (continuously save each preview image to disk)



As shown in the figure above, the SaveFile routine demonstrates how to save high-speed images. Through this routine, each frame of the previewed image can be directly saved to the disk, which is mostly used for high-speed, continuous shooting in a short time. Note that when using this routine, the performance of the hard disk of the system must be good enough, and the writing bandwidth of the hard disk must be greater than the output bandwidth of the preview image. Take the MV-GE130GM camera as an example. The camera can output 1280X960 images (1.2 million pixels, 1.2M) 60 frames per second, and the bandwidth is $60 \times 1.2 = 72\text{MB/s}$. If the image is saved as RAW format, the disk will be written A bandwidth of 72MB bytes can meet the

requirements, and every frame output by the camera can be saved. If the image is selected as BMP format, the disk writing bandwidth will need $72\text{MB} \times 3 = 216 \text{ MB/s}$ to save all the images. When the disk write bandwidth is lower than the requirement, some image files cannot be saved. In addition, it is not recommended to select formats other than RAW and BMP when saving at high speed. Images in jpg and png formats require a lot of time to compress and encode, which will cause congestion and cause frame loss, RAW format has not undergone any processing at all, and the conversion time is the smallest. Compared with RAW format, BMP format conversion is slightly more complicated, but the processing time is generally shorter and does not affect performance. Choose RAW format to save, and then cooperate with RawTransTest routine, you can offline convert RAW data into BMP, jpg or png format.

2.3 Debug camera parameters

2.3.1 How to set the exposure time (anti-stroboscopic, no smear, dynamic range improvement)

The exposure time of the camera is a very important parameter. In many applications, the setting methods are exquisite. According to the different applications, the exposure time setting is divided into the following aspects:

- **Achieve anti-stroboscopic effect.**When the light source used for illumination in the vision system is not in DC mode, when you preview the video, if it is a rolling shutter camera, you may see light and dark stripes;If it is a global shutter camera, there may be a big difference in the brightness of the two fields before and after, which looks like a flicker.To solve this problem, you first need to know the frequency of the light source. Generally speaking, the light source of AC lighting is 50 Hz or 60 Hz.If the light source is 50 Hz, you need to set the exposure time to an integer multiple of 10 ms (10 ms minimum); if the source is 60 Hz, you need to set the exposure time to an integer multiple of 8.3333 milliseconds.In addition, you can use the automatic exposure method provided by us and enable the anti-stroboscopic function.
- **Achieve no smear taking pictures.**When performing online visual inspection in the pipeline, it is often necessary to take pictures dynamically, that is, the detected object is captured in motion. If the exposure time is set unreasonably, it may cause obvious smear and blur the picture.To solve this problem, you need to set the exposure time to a very small value, generally less than 2 milliseconds, depending on the speed of the object's movement, continuously reduce the exposure time to match.As the exposure time is reduced, the image brightness will decrease, and stronger supplementary light

needs to be provided from the outside. At the same time, because the rolling shutter camera will produce distortion in the vertical direction, it is necessary to use the global shutter camera to capture without shadow.

- **Improved dynamic range.** The so-called dynamic range is the range from "darkest" to "brightest" contained in the image. The greater the dynamic range, the richer the levels that can be represented. In actual imaging, it often appears that when you see the bright part, the dark part is too dark to be recognized; when you see the dark part, the bright part has been exposed and cannot be recognized. At present, there are two ways to solve this problem. One is to choose a camera with a wide dynamic range by hardware; The second is to manually set the exposure time, and then adjust the gamma value and contrast to achieve the goal. This method is software to increase the dynamic range. The specific operation method is to manually set the exposure time until the brightest place can be seen clearly, then slowly adjust the gamma value to low, and slowly adjust the contrast to high. During the adjustment, observe the changes in the image until you get Better results.
- **The reference area setting for automatic exposure.** By default, the reference area of the automatic exposure algorithm is 1/2 of the middle of the image. You can call the relevant SDK function on the

software interface or secondary development to modify the automatic exposure reference area. After modification, when the part of the image outside the reference area changes, the auto exposure algorithm will not update the camera exposure value.

2.3.2 How to get better image color

Most industrial automation inspections are not sensitive to the color information of objects, so monochrome cameras are used. But in the fields of microscope imaging, color classification and recognition, the color reproduction of the camera is extremely important. When adjusting the color parameters of the camera, the following aspects need to be paid attention to.

- **White balance/area white balance.** Most of the light sources themselves are not pure white. After being illuminated by these light sources, the measured object may appear color cast. For example, under a yellow light source, after the camera is imaging, the image may be yellowish. In this case, white balance correction is needed. Its function is to restore the original white area in the image (but the color temperature of the light source has been affected by the color temperature of the light source) to pure white under the lighting of various color temperatures. $R = G = B$. The so-called regional white balance means that when

performing white balance operations, you can specify a certain image area as the reference analysis object. By default, the reference range of white balance is the entire image.

- Manual white balance/auto white balance. The white balance operation can be divided into two modes: manual and automatic. To manual white balance, also known as one-time white balance or one-key white balance, means that when the user needs to perform color correction, manually click the button on the software to perform a white balance correction. After the correction coefficient is obtained, the same coefficient is always used for color correction when the next white balance is not performed; Automatic white balance means that the camera analyzes the image in real time and dynamically modifies the correction coefficient. Generally speaking, manual white balance is mainly used in industrial applications to ensure that the correction coefficients after each imaging are exactly the same.
- Manually adjust the color gain of the red, green and blue channels. After the manual white balance operation is completed, the white area is restored to the real white that the computer thinks, that is, the R, G, and B components of the white area are completely equal, but this does not mean that the current color will make people look comfortable. According to the survey,

Europeans prefer warm-toned images (yellowish), while Asians prefer cool-toned images (blue). At this time, you can manually modify the R, G, and B three-channel gains to improve the image color.

- Saturation adjustment. Different people have different preferences for rich colors, which can be improved by adjusting saturation. If you like bright colors, increase the saturation setting; if you like light colors, decrease the saturation setting .

2.3.3 How to improve image clarity

In general, the sharpness of the image is determined by the lens' optical pointer, focus, and software algorithm. If the optical pointer of the lens is lower than the camera's requirements or the focus is not clear, choosing a camera with higher pixels will not help.

- Matching of lens and camera. On the parameter table of the camera, you can see the optical size of the camera, such as 1/2.5" target surface, 2.2 X 2.2 um pixel size. According to these parameters, choose the appropriate lens to achieve the best effect. Specific selection The method is not described here, please refer to the relevant documents in the network.
- Sharpness adjustment. The sharpness value can be increased on the software interface to improve the clarity of the image. Note:

As the sharpness increases, the sharpness will increase significantly, but the image noise will also be highlighted.

2.3.4 Selection of Bayer reduction algorithm (helps to better extract image contours)

For color industrial cameras, in general, the data format transmitted from the camera to the host is not RGB data, but an array color data format called Bayer. This format needs to be restored to RGB color data that can be recognized by computer vision algorithms through a certain algorithm. Different restoration algorithms have different effects and performance. Our SDK provides five Bayer conversion algorithms from A1 to A5 to choose from:

- It is calculated as the A2 algorithm by default. The advantage of this algorithm is that the sharpness is better restored, but false colors are easily generated at the edges of the color boundaries.
- For contour extraction using color cameras, it is recommended to use the A3 and A4 algorithms. The sharpness of these two algorithms is slightly lower than that of the A2 algorithm, but the image edges will be smoother and continuous.

2.3.5 How to reduce CPU usage

The CPU usage can be effectively reduced through the following aspects:

- Use monochrome cameras, monochrome cameras do not need color conversion, so you can save part of the CPU overhead. At the same time, set the image processing format of the SDK to 8bit mode, so that the data obtained is 8-bit grayscale, which can reduce the memory space and memory copy time.
- Use a camera with hardware ISP function, such as GIGE, USB3.0 camera.
- Adjust the exposure time, LUT curve and other parameters, while highlighting the characteristics of the detected object, try to make the parts that do not need to be detected as dark or bright as possible, which can greatly reduce the running time of the vision algorithm.

2.3.6 Save and load camera parameters

Four sets of parameters are provided in the SDK for secondary development or direct use by users, for saving or loading camera parameters. In our software platform, the camera parameters are saved on the disk as a binary file, the save path is /Camera/Configs under the software installation directory, and the extension is a .Config file.

- **Three different parameter loading methods**

There are three ways to access camera parameters: by model, by nickname, and by serial number. The default is to load by model, as shown in the figure below:



Take MV-UB130M (1.3 million monochrome frame memory camera) as an example:

1. When choosing to load by model, the file names of the four groups of parameters are:

MV-UB130M-Group0.Config, MV- UB130M-Group1.Config, MV-UB130M-Group2.Config, MV-

UB130M-Group3.Config, Corresponding to the four groups of parameters A, B, C, and D on the software setting interface

respectively. In this parameter loading mode, all cameras of the same model connected to the computer share these four groups of parameters;

When choosing to load parameters by nickname, the file names of the four groups of parameters are

MV-UB130M#0--Group0.Config, MV- UB130M#0-Group1.Config, MV-UB130M#0-Group2.Config, MV- UB130M#0-Group3.Config,

Corresponding to the four groups of parameters A, B, C, and D on the software setting interface respectively. It is worth noting that the nickname of the camera can be modified and can be modified directly on the software interface. It can also be modified through the software interface in the SDK. After the nickname is modified, the nickname is permanently effective. It is fixed in the camera. After changing the computer or USB slot, the nickname remains unchanged. After changing the nickname, the parameter file name corresponding to the camera will also change. For example, if the camera is renamed to Camera1, the file names of the four sets of parameters are Camera1-Group0.Config, Camera1-Group1.Config, Camera1--Group2.Config, Camera1-Group3.Config. This parameter loading mode is more flexible. When the computer is connected to only one camera, if the device nickname is not changed, the nickname of each camera of the same model is the same, and everyone shares these four sets of parameters. If you want one of them to load a different parameter set, you can name this camera separately to distinguish its device nickname from other cameras of the same model.

When loading by serial number is selected, the file names of the four groups of parameters are

MVUB130M-XXXXXX-XXXXXX-Group0.Config,

MVUB130M-XXXXXX-XXXXXX-Group1.Config,

MVUB130M-XXXXX-XXXXX-Group2.Config,

MVUB130M-XXXXX-XXXXX-Group3.Config,

Among them (XXXXX-XXXXX is the unique serial number of each camera), so in this parameter loading mode, the parameters of each camera are separate and do not affect each other.

In practical applications, you can choose different loading methods according to different needs. Loading by model ensures that all cameras of the same model use the same parameters; By loading by nickname, you can use the same parameters for the cameras of the same model, or you can specify one to use independent parameters; by loading by serial number, each camera uses independent parameters.

- **Modify the default parameters of the camera**

When the user clicks the "Restore Default Parameters" button on the software configuration interface, the camera will perform a parameter reset operation to reset the parameters to a preset state. In order to prevent users from various "abnormal" situations after modifying the parameters without knowing the camera parameters.

When restoring the default parameters, the parameters will be restored to the state at the factory, but if you are doing secondary development, you probably want the customer to restore the default parameters you set when you click the button, instead of the factory parameters. Here, we also provide a method to modify the preset

parameters, so that the end customer can restore the preset parameters (or the first time the camera is turned on after installing the camera program for the first time).The method is very simple: use our demo software, after setting the camera parameters, save it as a separate file, the file name is model name + "-Default.Config",Take MV-UB130M (1.3 million monochrome frame memory camera) as an example, save its parameters as a file "MV-UB130M-Default.Config",You can also directly rename the existing Config file to "MV-UB130M-Default.Config", and then place this file in the /Camera/Configs directory of the installation directory.In this way, the end user will restore the parameters in the "MV-UB130M-Default.Config" file when performing preset parameter restoration.

3 SDK data type definition

3.1 Structure definition

➤ **tSdkCameraDevInfo**

prototype:

typedef struct

```
{
    UINT    uVendorID;           // Vendor ID
    UINT    uProductID;         // Product ID
    char    acVendorName[32];    // Vendor name
    char    acProductSeries[32]; // Product Series
    char    acProductName[32];   // product name
    char    acFriendlyName[64];  // Initialize the device nickname
```

used by the interface

```
    char    acDevFileName[32];   // Driver name
    char    acFirmwareVersion[32]; // version
    char    acSensorType[32];    // sensor type
    char    acPortType[32];      // Interface Type
} tSdkCameraDevInfo;
```

Description: Device information of the camera

➤ **tSdkImageResolution**

prototype:

typedef struct

```
{
    INT      iIndex;             // Index number, [0,N] means the
                                // default resolution (N is the maximum number of default resolutions,
                                // generally not more than 20), 0xFF means custom resolution (ROI)
    char     acDescription[32];  // The description of the resolution.
                                // This information is valid only at the default resolution. Custom
                                // resolution can ignore this information
    UINT     uBinSumMode;        // BIN (sum) mode, the range
                                // cannot exceed uBinSumModeMask in tSdkResolutionRange
```

```

    UINT    uBinAverageMode;    // BBIN (average finding) mode,
the range cannot exceed uBinAverageModeMask in
tSdkResolutionRange
    UINT    uSkipMode;          // Whether the size of SKIP, 0
means that SKIP mode is prohibited, and the range cannot exceed
uSkipModeMask in tSdkResolutionRange
    UINT    uResampleMask;      // Masking of hardware
resampling
    INT     iHOffsetFOV;        // The vertical offset of the
acquisition field of view relative to the upper left corner of the
sensor's maximum field of view
    INT     iVOffsetFOV;        // The horizontal offset of the
acquisition field of view relative to the upper left corner of the
sensor's maximum field of view
    INT     iWidthFOV;          // The width of the acquisition field
of view
    INT     iHeightFOV;         // The height of the acquisition field
of view
    INT     iWidth;             // The width of the final image
output by the camera
    INT     iHeight;            // The height of the final image
output by the camera
    INT     iWidthZoomHd;       // The width of hardware scaling,
the resolution that does not require this operation, this variable is set
to 0.
    INT     iHeightZoomHd;      // The height of the hardware
scaling, the resolution for which this operation is not required, this
variable is set to 0.
    INT     iWidthZoomSw;       // The width of the software
scaling, the resolution that does not require this operation, this
variable is set to 0.
    INT     iHeightZoomSw;      // The height of the software
scaling, the resolution that does not require this operation, this
variable is set to 0.

```

```
} tSdkImageResolution;
```

Description: The resolution description of the camera

➤ **tSdkMediaType**

prototype:

```
typedef struct
{
    INT      iIndex;           //Format type number
    char      acDescription[32]; //Description
    UINT      iMediaType;      //Corresponding image format
    encoding
} tSdkMediaType;
```

Description: The image data format output by the camera

➤ **tSdkIspCapacity**

prototype:

```
typedef struct
{
    BOOL bMonoSensor;           //Indicates whether the camera
                                //of this model is a monochrome camera, if it is a monochrome camera,
                                //the color-related functions cannot be adjusted
    BOOL bWbOnce;               //Indicates whether this model
                                //camera supports manual white balance function
    BOOL bAutoWb;               //Indicates whether this model
                                //camera supports auto white balance function
    BOOL bAutoExposure;         //Indicates whether this model
                                //camera supports auto exposure function
    BOOL bManualExposure;       //Indicates whether this model
                                //camera supports manual exposure function
    BOOL bAntiFlick;            //Indicates whether the camera of
                                //this model supports anti-flicker function
    BOOL bDevicelsp;            //Indicates whether this model
                                //camera supports hardware ISP function
}
```

```

        BOOL bForceUseDevicelsp;//When bDevicelsp and
bForceUseDevicelsp are TRUE at the same time, it means that only
hardware ISP is mandatory and cannot be cancelled.
        BOOL bZoomHD;           //Does the camera hardware
support image zoom output (only zoom out).
    } tSdkIspCapacity;

```

Description: ISP module enable information

➤ **tSdkCameraCapbility**

prototype:

```

typedef struct
{
    tSdkTrigger    *pTriggerDesc;           // Trigger mode
    INT            iTriggerDesc;           // The number of
trigger modes, that is, the size of the pTriggerDesc array

    tSdkImageResolution *pImageSizeDesc;// Default resolution
selection
    INT            iImageSizeDesc; // The number of
default resolutions, that is, the size of the pImageSizeDesc array

    tSdkColorTemperatureDes *pClrTempDesc;// Default color
temperature mode for white balance
    INT            iClrTempDesc;

    tSdkMediaType    *pMediaTypeDesc;     // Camera output
image format
    INT            iMediaTypdeDesc;     // The number of
types of camera output image formats, that is, the size of the
pMediaTypeDesc array.

    tSdkFrameSpeed    *pFrameSpeedDesc; // Adjustable
frame rate type, corresponding to normal high speed and super
three speed settings on the interface

```

INT iFrameSpeedDesc; // The number of frame rate types can be adjusted, that is, the size of the pFrameSpeedDesc array.

tSdkPackLength *pPackLenDesc; // Transmission packet length, generally used for network equipment

INT iPackLenDesc; // The number of transmission packet lengths that can be selected, that is, the size of the pPackLenDesc array.

INT iOutputIoCounts; // Number of programmable output IO

INT iInputIoCounts; // Number of programmable input IO

tSdkPresetLut *pPresetLutDesc; // Camera preset LUT table

INT iPresetLut; // The number of LUT tables preset by the camera, that is, the size of the pPresetLutDesc array

INT iUserDataMaxLen; // Indicates the maximum length of the user data area in the camera. 0 means none.

BOOL bParamInDevice; // Indicates whether the device supports reading and writing parameter groups from the device. 1 means support, 0 does not support.

tSdkAeAlgorithm *pAeAlmSwDesc; // Software automatic exposure algorithm description

int iAeAlmSwDesc; // Number of software automatic exposure algorithms

tSdkAeAlgorithm *pAeAlmHdDesc; // Hardware auto exposure algorithm description, NULL means that hardware auto exposure is not supported

```

        int                iAeAlmHdDesc;    // Number of
hardware auto exposure algorithms, 0 means that hardware auto
exposure is not supported

        tSdkBayerDecodeAlgorithm    *pBayerDecAlmSwDesc; //
Algorithm description of software Bayer conversion to RGB data
        int                iBayerDecAlmSwDesc; // The
number of algorithms that software Bayer converts to RGB data

        tSdkBayerDecodeAlgorithm    *pBayerDecAlmHdDesc; // The
algorithm description of hardware Bayer conversion to RGB data,
NULL means it is not supported
        int                iBayerDecAlmHdDesc; // The
number of algorithms for converting hardware Bayer to RGB data, 0
means not supported
        /* Definition of the adjustment range of image parameters for
dynamic UI construction*/
        tSdkExpose          sExposeDesc;    // Exposure range
value
        tSdkResolutionRange    sResolutionRange; // Resolution
range description
        tRgbGainRange         sRgbGainRange; // Image digital
gain range description
        tSaturationRange      sSaturationRange; // Saturation range
description
        tGammaRange           sGammaRange; // Gamma
range description
        tContrastRange         sContrastRange; // Contrast range
description
        tSharpnessRange       sSharpnessRange; // Sharpening
range description
        tSdkIspCapacity        sIspCapacity; // ISP capability
description
    } tSdkCameraCapability;

```

Description: Define integrated device description information, which can be used to dynamically build UI.

➤ **tSdkFrameHead**

prototype:

```
typedef struct
{
    UINT    uiMediaType;    // Image Format
    UINT    uBytes;         // mage data bytes, Total bytes
    UINT    iHeight;        // Image height
    UINT    iWidth;         // Image width
    BOOL    blsTrigger;     // Indicates whether it is a trigger frame
    UINT    uiTimeStamp;    // The acquisition time of this frame, in
0.1 milliseconds
    UINT    uiExpTime;      // The exposure value of the current
image, in microseconds us
    float    fAnalogGain;   // Analog gain multiple of current
image
    INT      iGamma;        // The gamma setting value of the
frame image is valid only when the LUT mode is dynamic parameter
generation, and it is -1 in the other modes
    INT      iContrast;     // The contrast setting value of the
frame image is only valid when the LUT mode is dynamic parameter
generation, and it is -1 in other modes
    INT      iSaturation;   // The saturation setting value of the
frame image is meaningless for monochrome cameras. It is 0
float
fRgain;    // The red digital gain multiplier of this frame of
image processing, meaningless for monochrome cameras, is 1
    float    fGgain;       // The green digital gain multiple of
this frame of image processing is meaningless for monochrome
cameras, which is 1
    float    fBgain;       // The blue digital gain multiplier of
this frame of image processing is meaningless for monochrome
cameras, which is 1
```

```
}tSdkFrameHead;
```

Description: image frame header information

➤ **tSdkFrame**

prototype:

```
typedef struct sCameraFrame
{
    tSdkFrameHead head;        //Frame header
    BYTE *    pBuffer;        //Data area
}tSdkFrame;
```

Description: image frame description

3.2 Parameter type definition

➤ **emSdkLutMode**

prototype:

```
typedef enum
{
    LUTMODE_PARAM_GEN=0,        //Dynamically generate LUT
table by adjusting parameters
    LUTMODE_PRESET,            //Use preset LUT tables
    LUTMODE_USER_DEF,          //Use user-defined LUT tables
}emSdkLutMode;
```

Description: The way of image look-up table transformation

➤ **emSdkParamTarget**

prototype:

```
typedef enum
{
    PARAM_ON_PC = 0,            //Save to PC
    PARAM_ON_DEVICE = 1,        //Save to camera
}emSdkParamTarget;
```

Description: The object to save the camera parameters

➤ **emSdkRunMode**

prototype:

```
typedef enum
{
    RUNMODE_PLAY=0,           //Normal preview, the captured
image will be displayed.
    RUNMODE_PAUSE,           //pause
    RUNMODE_STOP,            //Stop the camera from
working.
}emSdkRunMode;
```

Description: Camera video stream control

➤ **emSdkDisplayMode**

prototype:

```
typedef enum
{
    DISPLAYMODE_SCALE=0,      //Zoom display mode
    DISPLAYMODE_REAL          //1:1 display mode
}emSdkDisplayMode;
```

Description: The display mode of the SDK internal display interface

➤ **emSdkRecordMode**

prototype:

```
typedef enum
{
    RECORD_STOP = 0,          //stop
    RECORD_START,             //start
    RECORD_PAUSE,             //pause
}emSdkRecordMode;
```

Description: Recording status

➤ **emSdkMirrorDirection**

prototype:

```
typedef enum
```

```

{
    MIRROR_DIRECTION_HORIZONTAL = 0,          //Horizontal
mirroring
    MIRROR_DIRECTION_VERTICAL ,                //Vertical mirror
}emSdkMirrorDirection;

```

Description: Image mirroring operation

➤ **emSdkFrameSpeed**

prototype:

```

typedef enum
{
    FRAME_SPEED_LOW = 0,          //Low speed mode
    FRAME_SPEED_NORMAL,          //Normal mode
    FRAME_SPEED_HIGH,            //High speed mode
    FRAME_SPEED_SUPER,           //Ultra high speed mode
}emSdkFrameSpeed;

```

Description: The frame rate of the camera video

➤ **emSdkFileType**

prototype:

```

typedef enum
{
    FILE_JPG = 1,                //JPG
    FILE_BMP = 2,                //BMP
    FILE_RAW = 4,                //RAW: Bayer format file output by
the camera
    FILE_PNG = 8                 //PNG
}emSdkFileType;

```

Description: Format type of archive

➤ **emSdkLightFrequency**

prototype:

```

typedef enum
{

```

```

    LIGHT_FREQUENCY_50HZ = 0,      //50HZ
    LIGHT_FREQUENCY_60HZ ,        //60HZ
}emSdkLightFrequency;

```

Description: Anti automatic strobe flash exposure

➤ **emSdkParameterMode**

prototype:

```

typedef enum
{
    PARAM_MODE_BY_MODEL = 0,  //Load parameters from the file
    according to the camera model name, such as MV-U300
    PARAM_MODE_BY_NAME,      //Load parameters from the file
    according to the device nickname
    (tSdkCameraDevInfo.acFriendlyName), such as MV-U300, the
    nickname can be customized
    PARAM_MODE_BY_SN,        //Load the parameters from the
    file according to the unique serial number of the device. The serial
    number has been written into the device at the factory, and each
    camera has a different serial number.
    PARAM_MODE_IN_DEVICE     //Load the parameters from the
    solid-state memory of the device. Not all models support reading
    and writing parameter groups from the camera, which is determined
    by tSdkCameraCapbility.bParamInDevice
}emSdkParameterMode;

```

Description: PARAM_MODE_BY_MODEL: All cameras of the same model share ABCD four groups of parameter files. Modify the parameter file of a camera, Modifying the parameter file of a camera will affect the loading of the entire camera parameters of the same model.

PARAM_MODE_BY_NAME: All cameras with the same device name share four groups of parameter files ABCD.

By default, when only one camera of a certain model is connected to the computer, The device names are the same, and if you want a

camera to be able to load different parameter files, you can modify the device name to make it load the specified parameter file.

PARAM_MODE_BY_SN: The camera loads the four groups of ABCD parameter files according to its own unique serial number, The serial number has been solidified in the camera before leaving the factory, and the serial number of each camera is different. In this way, the parameter file of each camera is independent.

You can flexibly use the above methods to load parameters according to your own usage environment. For example, take the MV-U300 as an example, if you want multiple cameras of this model to share 4 sets of parameters on your computer, then use the **PARAM_MODE_BY_MODEL** method; If you want one or several MV-U300 to use its own parameter file and the other MV-U300 to use the same parameter file, then use the **PARAM_MODE_BY_NAME** method; If you want each MV-U300 to use a different parameter file, then use the **PARAM_MODE_BY_SN** method.

The parameter file is stored in the \Camera\Configs directory of the installation directory, with a file with the suffix config.

➤ **emSdkParameterTeam**

prototype:

```
typedef enum
{
    PARAMETER_TEAM_DEFAULT = 0xff,
    PARAMETER_TEAM_A = 0,
    PARAMETER_TEAM_B = 1,
    PARAMETER_TEAM_C = 2,
    PARAMETER_TEAM_D = 3
}emSdkParameterTeam;
```

Note: The configuration parameters of the camera are divided into 4 groups A, B, C, and D for saving.

➤ **emSdkPropSheetMsg**

prototype:

```
typedef enum
{
    SHEET_MSG_LOAD_PARAM_DEFAULT = 0,    //After the
parameter is restored to the default, trigger the message
    SHEET_MSG_LOAD_PARAM_GROUP,          //Load the
specified parameter group and trigger the message
    SHEET_MSG_LOAD_PARAM_FROMFILE,       //After loading
the parameters from the specified file, the message is triggered
    SHEET_MSG_SAVE_PARAM_GROUP           //When the
current parameter group is saved, trigger this message
}emSdkPropSheetMsg;
```

Description: The callback message type of the camera configuration page generated by the SDK

➤ **tSdkWhiteBalanceDes**

prototype:

```
typedef struct
{
    INT    iIndex;           // Mode index number
    char acDescription[32];   // Description
} tSdkWhiteBalanceDes;
```

Description: Description of camera white balance mode

➤ **tSdkFrameSpeed**

prototype:

```
typedef struct
{
    INT    iIndex;           // Frame rate index number
    char acDescription[32];   // Description
} tSdkFrameSpeed;
```

Description: Camera frame rate description information

➤ **tSdkExpose**

prototype:

```
typedef struct
{
    UINT    uiTargetMin;           //Auto exposure brightness target
    minimum
    UINT    uiTargetMax;           //Auto exposure brightness target
    maximum
    UINT    uiAnalogGainMin;       //Minimum value of analog gain
    UINT    uiAnalogGainMax;       //Maximum analog gain
    float    fAnalogGainStep;      //Analog gain step
    UINT    uiExposeTimeMin;       //In manual mode, the minimum
    number of exposure lines
    UINT    uiExposeTimeMax;       //In manual mode, the
    maximum number of exposure lines
} tSdkExpose;
```

Description: Definition of camera exposure function range

➤ **tSdkTrigger**

prototype:

```
typedef struct
{
    INT     iIndex;                //Mode index number
    char     acDescription[32];     //Description of the mode
} tSdkTrigger;
```

Description: Trigger mode description

➤ **tSdkPackLength**

prototype:

```
typedef struct
{
    INT     iIndex;                //Subpacket size index number
    char     acDescription[32];     //Corresponding description
    information
```

```
} tSdkPackLength;
```

Description: description of transmission sub-packet size

➤ **tSdkHardwareIO**

prototype:

```
typedef struct
{
    INT    iIndex;                //IO number
    BOOL   bOutPut;               //IIO attributes, input or output
} tSdkHardwareIO;
```

Description: The description of the programmable IO on the camera

➤ **tSdkPresetLut**

prototype:

```
typedef struct
{
    INT    iIndex;                //Numbering
    char   acDescription[32];      //Description
} tSdkPresetLut;
```

Description: The default LUT table description

➤ **tSdkFrameStatistic**

prototype:

```
typedef struct
{
    INT iTotal;                //The total number of frames currently
    collected (including error frames)
    INT iCapture;              //The number of valid frames currently
    collected
    INT iLost;                  //The current number of dropped frames
} tSdkFrameStatistic;
```

Description: Frame rate statistics

➤ **tGammaRange**

prototype:

```
typedef struct
{
    INT    iMin;           //Minimum
    INT    iMax;           //Max
} tGammaRange;
```

Description: Gamma setting range

➤ **tContrastRange**

prototype:

```
typedef struct
{
    INT iMin;           //Minimum
    INT iMax;           //Max
} tContrastRange;
```

Description: Setting range of contrast

➤ **tRgbGainRange**

prototype:

```
typedef struct
{
    INT iRGainMin;       //Minimum value of red gain
    INT iRGainMax;       //Maximum red gain
    INT iGGainMin;       //Minimum green gain
    INT iGGainMax;       //Maximum green gain
    INT iBGainMin;       //Minimum blue gain
    INT iBGainMax;       //Maximum blue gain
} tRgbGainRange;
```

Description: Setting range of RGB three-channel digital gain

➤ **tSaturationRange**

prototype:

```
typedef struct
```

```

{
    INT iMin;        //Minimum
    INT iMax;        //Max
} tSaturationRange;

```

Description: The range of saturation setting

➤ **tSdkResolutionRange**

prototype:

```

typedef struct
{
    INT iHeightMax;           //The maximum height of the image
    INT iHeightMin;           //Minimum image height
    INT iWidthMax;            //The maximum width of the image
    INT iWidthMin;            //Minimum image width
    UINT uSkipModeMask;       //SKIP mode is shielded. If it is 0, it
means that SKIP is not supported. Bit0 is 1, indicating that SKIP 2x2 is
supported; bit1 is 1, indicating that SKIP 3x3...
    UINT uBinSumModeMask;     //BIN (summation) mode is
shielded, which is 0, which means BIN is not supported. Bit0 is 1,
indicating that BIN 2x2 is supported; bit1 is 1, indicating that BIN 3x3
is supported...
    UINT uBinAverageModeMask; //BIN (averaging) mode is
masked, and it is 0, which means BIN is not supported. Bit0 is 1,
indicating that BIN 2x2 is supported; bit1 is 1, indicating that BIN 3x3
is supported...
    UINT uResampleMask;       //Masking of hardware resampling
} tSdkResolutionRange;

```

Description: The resolution setting range of the camera

➤ **emSdkRefWinType**

prototype:

```

typedef enum
{
    REF_WIN_AUTO_EXPOSURE = 0,

```

```
REF_WIN_WHITE_BALANCE,  
}emSdkRefWinType;
```

Description: Visually select the type of reference window

➤ **tSharpnessRange**

prototype:

```
typedef struct  
{  
    INT iMin;        //Minimum  
    INT iMax;        //Max  
} tSharpnessRange;
```

Description: Sharpening setting range

➤ **SKIP_MASK_xxx macro definition**

prototype:

```
#define MASK_2X2_HD      (1<<0)    //Hardware SKIP, BIN,  
resampling 2X2  
#define MASK_3X3_HD      (1<<1)  
#define MASK_4X4_HD      (1<<2)  
#define MASK_5X5_HD      (1<<3)  
#define MASK_6X6_HD      (1<<4)  
#define MASK_7X7_HD      (1<<5)  
#define MASK_8X8_HD      (1<<6)  
#define MASK_9X9_HD      (1<<7)  
#define MASK_10X10_HD     (1<<8)  
#define MASK_11X11_HD     (1<<9)  
#define MASK_12X12_HD     (1<<10)  
#define MASK_13X13_HD     (1<<11)  
#define MASK_14X14_HD     (1<<12)  
#define MASK_15X15_HD     (1<<13)  
#define MASK_16X16_HD     (1<<14)  
#define MASK_17X17_HD     (1<<15)  
#define MASK_2X2_SW      (1<<16)    //Hardware SKIP, BIN,  
resampling 2X2
```

```

#define MASK_3X3_SW      (1<<17)
#define MASK_4X4_SW      (1<<18)
#define MASK_5X5_SW      (1<<19)
#define MASK_6X6_SW      (1<<20)
#define MASK_7X7_SW      (1<<21)
#define MASK_8X8_SW      (1<<22)
#define MASK_9X9_SW      (1<<23)
#define MASK_10X10_SW     (1<<24)
#define MASK_11X11_SW     (1<<25)
#define MASK_12X12_SW     (1<<26)
#define MASK_13X13_SW     (1<<27)
#define MASK_14X14_SW     (1<<28)
#define MASK_15X15_SW     (1<<29)
#define MASK_16X16_SW     (1<<30)
#define MASK_17X17_SW     (1<<31)

```

Description: The mask value of SKIP, BIN, and RESAMPLE mode in the tSdkResolutionRange structure

➤ **CAMERA_MEDIA_TYPE_MONOxx macro definition**

prototype:

```

#define CAMERA_MEDIA_TYPE_MONO1P
(CAMERA_MEDIA_TYPE_MONO |
CAMERA_MEDIA_TYPE_OCCUPY1BIT | 0x0037)

#define CAMERA_MEDIA_TYPE_MONO2P
(CAMERA_MEDIA_TYPE_MONO |
CAMERA_MEDIA_TYPE_OCCUPY2BIT | 0x0038)

#define CAMERA_MEDIA_TYPE_MONO4P
(CAMERA_MEDIA_TYPE_MONO |
CAMERA_MEDIA_TYPE_OCCUPY4BIT | 0x0039)

```

```

#define CAMERA_MEDIA_TYPE_MONO8
(CAMERA_MEDIA_TYPE_MONO |
CAMERA_MEDIA_TYPE_OCCUPY8BIT | 0x0001)

#define CAMERA_MEDIA_TYPE_MONO8S
(CAMERA_MEDIA_TYPE_MONO |
CAMERA_MEDIA_TYPE_OCCUPY8BIT | 0x0002)

#define CAMERA_MEDIA_TYPE_MONO10
(CAMERA_MEDIA_TYPE_MONO |
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0003)

#define CAMERA_MEDIA_TYPE_MONO10_PACKED
(CAMERA_MEDIA_TYPE_MONO |
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x0004)

#define CAMERA_MEDIA_TYPE_MONO12
(CAMERA_MEDIA_TYPE_MONO |
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0005)

#define CAMERA_MEDIA_TYPE_MONO12_PACKED
(CAMERA_MEDIA_TYPE_MONO |
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x0006)

#define CAMERA_MEDIA_TYPE_MONO14
(CAMERA_MEDIA_TYPE_MONO |
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0025)

#define CAMERA_MEDIA_TYPE_MONO16
(CAMERA_MEDIA_TYPE_MONO |
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0007)

```

Description: image format definition

➤ **CAMERA_MEDIA_TYPE_BAYxxx macro definition**

prototype:

```
#define CAMERA_MEDIA_TYPE_BAYGR8  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY8BIT | 0x0008)  
  
#define CAMERA_MEDIA_TYPE_BAYRG8  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY8BIT | 0x0009)  
  
#define CAMERA_MEDIA_TYPE_BAYGB8  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY8BIT | 0x000A)  
  
#define CAMERA_MEDIA_TYPE_BAYBG8  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY8BIT | 0x000B)  
  
#define CAMERA_MEDIA_TYPE_BAYGR10_MIPI  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY10BIT | 0x0026)  
  
#define CAMERA_MEDIA_TYPE_BAYRG10_MIPI  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY10BIT | 0x0027)  
  
#define CAMERA_MEDIA_TYPE_BAYGB10_MIPI  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY10BIT | 0x0028)  
  
#define CAMERA_MEDIA_TYPE_BAYBG10_MIPI  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY10BIT | 0x0029)
```

```
#define CAMERA_MEDIA_TYPE_BAYGR10  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x000C)  
  
#define CAMERA_MEDIA_TYPE_BAYRG10  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x000D)  
  
#define CAMERA_MEDIA_TYPE_BAYGB10  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x000E)  
  
#define CAMERA_MEDIA_TYPE_BAYBG10  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x000F)  
  
#define CAMERA_MEDIA_TYPE_BAYGR12  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0010)  
  
#define CAMERA_MEDIA_TYPE_BAYRG12  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0011)  
  
#define CAMERA_MEDIA_TYPE_BAYGB12  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0012)  
  
#define CAMERA_MEDIA_TYPE_BAYBG12  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0013)
```

```
#define CAMERA_MEDIA_TYPE_BAYGR10_PACKED  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x0026)
```

```
#define CAMERA_MEDIA_TYPE_BAYRG10_PACKED  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x0027)
```

```
#define CAMERA_MEDIA_TYPE_BAYGB10_PACKED  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x0028)
```

```
#define CAMERA_MEDIA_TYPE_BAYBG10_PACKED  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x0029)
```

```
#define CAMERA_MEDIA_TYPE_BAYGR12_PACKED  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x002A)
```

```
#define CAMERA_MEDIA_TYPE_BAYRG12_PACKED  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x002B)
```

```
#define CAMERA_MEDIA_TYPE_BAYGB12_PACKED  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x002C)
```

```
#define CAMERA_MEDIA_TYPE_BAYBG12_PACKED  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x002D)
```

```
#define CAMERA_MEDIA_TYPE_BAYGR16  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x002E)
```

```
#define CAMERA_MEDIA_TYPE_BAYRG16  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x002F)
```

```
#define CAMERA_MEDIA_TYPE_BAYGB16  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0030)
```

```
#define CAMERA_MEDIA_TYPE_BAYBG16  
(CAMERA_MEDIA_TYPE_MONO |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0031)
```

Description: Bayer format encoding definition

➤ **CAMERA_MEDIA_TYPE_RGB8xx macro definition**

prototype:

```
#define CAMERA_MEDIA_TYPE_RGB8  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY24BIT | 0x0014)
```

```
#define CAMERA_MEDIA_TYPE_BGR8  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY24BIT | 0x0015)
```

```
#define CAMERA_MEDIA_TYPE_RGBA8  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY32BIT | 0x0016)
```

```
#define CAMERA_MEDIA_TYPE_BGRA8  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY32BIT | 0x0017)
```

```
#define CAMERA_MEDIA_TYPE_RGB10
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY48BIT | 0x0018)

#define CAMERA_MEDIA_TYPE_BGR10
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY48BIT | 0x0019)

#define CAMERA_MEDIA_TYPE_RGB12
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY48BIT | 0x001A)

#define CAMERA_MEDIA_TYPE_BGR12
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY48BIT | 0x001B)

#define CAMERA_MEDIA_TYPE_RGB16
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY48BIT | 0x0033)

#define CAMERA_MEDIA_TYPE_RGB10V1_PACKED
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY32BIT | 0x001C)

#define CAMERA_MEDIA_TYPE_RGB10P32
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY32BIT | 0x001D)

#define CAMERA_MEDIA_TYPE_RGB12V1_PACKED
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY36BIT | 0x0034)
```

```
#define CAMERA_MEDIA_TYPE_RGB565P  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0035)
```

```
#define CAMERA_MEDIA_TYPE_BGR565P  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0036)
```

Description: RGB format encoding definition

➤ **CAMERA_MEDIA_TYPE_YUV411_8_xxxx macro definition**

prototype:

```
#define CAMERA_MEDIA_TYPE_YUV411_8_UYYVYY  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x001E)
```

```
#define CAMERA_MEDIA_TYPE_YUV422_8_UYVY  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x001F)
```

```
#define CAMERA_MEDIA_TYPE_YUV422_8  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0032)
```

```
#define CAMERA_MEDIA_TYPE_YUV8_UYV  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY24BIT | 0x0020)
```

```
#define CAMERA_MEDIA_TYPE_YCBCR8_CBYCR  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY24BIT | 0x003A)
```

```
#define CAMERA_MEDIA_TYPE_YCBCR422_8  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x003B)  
  
#define CAMERA_MEDIA_TYPE_YCBCR422_8_CBYCRY  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0043)  
  
#define CAMERA_MEDIA_TYPE_YCBCR411_8_CBYYCRY  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x003C)  
  
#define CAMERA_MEDIA_TYPE_YCBCR601_8_CBYCR  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY24BIT | 0x003D)  
  
#define CAMERA_MEDIA_TYPE_YCBCR601_422_8  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x003E)  
  
#define CAMERA_MEDIA_TYPE_YCBCR601_422_8_CBYCRY  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0044)  
  
#define CAMERA_MEDIA_TYPE_YCBCR601_411_8_CBYYCRY  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x003F)  
  
#define CAMERA_MEDIA_TYPE_YCBCR709_8_CBYCR  
(CAMERA_MEDIA_TYPE_COLOR |  
CAMERA_MEDIA_TYPE_OCCUPY24BIT | 0x0040)
```

```
#define CAMERA_MEDIA_TYPE_YCBCR709_422_8
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0041)

#define CAMERA_MEDIA_TYPE_YCBCR709_422_8_CBYCRY
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY16BIT | 0x0045)

#define CAMERA_MEDIA_TYPE_YCBCR709_411_8_CBYYCRY
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY12BIT | 0x0042)
```

Description: YUV format code definition.

➤ **CAMERA_MEDIA_TYPE_RGBXX_PLANAR macro definition**

prototype:

```
#define CAMERA_MEDIA_TYPE_RGB8_PLANAR
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY24BIT | 0x0021)

#define CAMERA_MEDIA_TYPE_RGB10_PLANAR
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY48BIT | 0x0022)

#define CAMERA_MEDIA_TYPE_RGB12_PLANAR
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY48BIT | 0x0023)

#define CAMERA_MEDIA_TYPE_RGB16_PLANAR
(CAMERA_MEDIA_TYPE_COLOR |
CAMERA_MEDIA_TYPE_OCCUPY48BIT | 0x0024)
```

Description: RGB planar format coding definition.

3.3 Interface return value definition (error code)

```
#define CAMERA_STATUS_SUCCESS 0
// Successful operation
#define CAMERA_STATUS_FAILED -1
// operation failed
#define CAMERA_STATUS_INTERNAL_ERROR -2
// internal error
#define CAMERA_STATUS_UNKNOW -3
// unknown mistake
#define CAMERA_STATUS_NOT_SUPPORTED -4
// Doesn't support this feature
#define CAMERA_STATUS_NOT_INITIALIZED -5
// Initialization is not complete
#define CAMERA_STATUS_PARAMETER_INVALID -6
// Invalid parameter
#define CAMERA_STATUS_PARAMETER_OUT_OF_BOUND -7
// Parameter out of bounds
#define CAMERA_STATUS_UNENABLED -8
// Not enabled
#define CAMERA_STATUS_USER_CANCEL -9
// The user manually canceled, for example, click cancel on the roi
panel to return
#define CAMERA_STATUS_PATH_NOT_FOUND -10
// The corresponding path was not found in the registry
#define CAMERA_STATUS_SIZE_DISMATCH -11
// The length of the obtained image data does not match the defined
size
#define CAMERA_STATUS_TIME_OUT -12
// Timeout error
#define CAMERA_STATUS_IO_ERROR -13
// Hardware IO error
#define CAMERA_STATUS_COMM_ERROR -14
// Communication error
#define CAMERA_STATUS_BUS_ERROR -15
```

```

// Bus error
#define CAMERA_STATUS_NO_DEVICE_FOUND -16
// No device found
#define CAMERA_STATUS_NO_LOGIC_DEVICE_FOUND -17
// No logical device found
#define CAMERA_STATUS_DEVICE_IS_OPENED -18
// Device is already on
#define CAMERA_STATUS_DEVICE_IS_CLOSED -19
// Device is turned off
#define CAMERA_STATUS_DEVICE_VEDIO_CLOSED -20
// If the device video is not opened, when the function related to
recording is called, if the camera video is not opened, this error will
be returned.
#define CAMERA_STATUS_NO_MEMORY -21
// Not enough system memory
#define CAMERA_STATUS_FILE_CREATE_FAILED -22
// File creation failed
#define CAMERA_STATUS_FILE_INVALID -23
// Invalid file format
#define CAMERA_STATUS_WRITE_PROTECTED -24
// Write protection, not writable
#define CAMERA_STATUS_GRAB_FAILED -25
// Data collection failed
#define CAMERA_STATUS_LOST_DATA -26
// Data loss, incomplete
#define CAMERA_STATUS_EOF_ERROR -27
// End of frame not received
#define CAMERA_STATUS_BUSY -28
// Busy (the last operation is still in progress), this operation cannot
be performed
#define CAMERA_STATUS_WAIT -29
// Need to wait (the conditions for the operation are not established),
you can try again
#define CAMERA_STATUS_IN_PROCESS -30

```

```

// In progress, has been operated
#define CAMERA_STATUS_IIC_ERROR -31
// IIC transmission error
#define CAMERA_STATUS_SPI_ERROR -32
// SPI transmission error
#define CAMERA_STATUS_USB_CONTROL_ERROR -33
// USB control transmission error
#define CAMERA_STATUS_USB_BULK_ERROR -34
//USB BULK transmission error
#define CAMERA_STATUS_SOCKET_INIT_ERROR -35
// Failed to initialize the network transmission kit
#define CAMERA_STATUS_GIGE_FILTER_INIT_ERROR -36
// The network camera kernel filter driver failed to initialize, please
check whether the driver is installed correctly, or reinstall it.
#define CAMERA_STATUS_NET_SEND_ERROR -37
// Network data sending error
#define CAMERA_STATUS_DEVICE_LOST -38
// Lost connection with web camera, heartbeat detection timed out
#define CAMERA_STATUS_DATA_RECV_LESS -39
// The number of bytes received is less than requested
#define CAMERA_STATUS_FUNCTION_LOAD_FAILED -40
// Failed to load program from file
#define CAMERA_STATUS_CRITICAL_FILE_LOST -41
// The files necessary for running the program are missing.
#define CAMERA_STATUS_SENSOR_ID_DISMATCH -42
// The firmware and program do not match because the wrong
firmware was downloaded
#define CAMERA_STATUS_OUT_OF_RANGE -43
// Parameter out of valid range
#define CAMERA_STATUS_REGISTRY_ERROR -44
// Installer registration error. Please reinstall the program, or run the
installation directorySetup/Installer.exe
#define CAMERA_STATUS_ACCESS_DENY -45

```

```

// No Access. When the specified camera is already occupied by
other programs, and then apply for access to the camera, it will
return to this state. (A camera cannot be accessed by multiple
programs at the same time)
#define CAMERA_STATUS_CAMERA_NEED_RESET          -46
// It means that the camera needs to be reset before it can be used
normally. At this time, please power off the camera and restart it, or
restart the operating system before it can be used normally.
#define CAMERA_STATUS_ISP_MODULE_NOT_INITIALIZED -47
// ISP module is not initialized
#define CAMERA_STATUS_ISP_DATA_CRC_ERROR         -48
// Data verification error
#define CAMERA_STATUS_MV_TEST_FAILED             -49
// Data test failed
#define CAMERA_AIA_PACKET_RESEND                 0x0100
//The frame needs to be retransmitted

#define CAMERA_AIA_NOT_IMPLEMENTED                0x8001
//Commands not supported by the device

#define CAMERA_AIA_INVALID_PARAMETER              0x8002
//Illegal command parameter

#define CAMERA_AIA_INVALID_ADDRESS                0x8003
//Inaccessible address

#define CAMERA_AIA_WRITE_PROTECT                  0x8004
//The accessed object is not writable

#define CAMERA_AIA_BAD_ALIGNMENT                  0x8005
//The accessed address is not aligned as required

#define CAMERA_AIA_ACCESS_DENIED                  0x8006
//No access

```

```

#define CAMERA_AIA_BUSY 0x8007
//Command is being processed

#define CAMERA_AIA_DEPRECATED 0x8008
//0x8008-0x0800B 0x800F This instruction is obsolete

#define CAMERA_AIA_PACKET_UNAVAILABLE 0x800C
//Invalid package

#define CAMERA_AIA_DATA_OVERRUN 0x800D
//Data overflow, usually because more data is received than needed

#define CAMERA_AIA_INVALID_HEADER 0x800E
//Some areas in the packet header do not match the protocol

#define CAMERA_AIA_PACKET_NOT_YET_AVAILABLE 0x8010
//Image packet data is not ready yet, it is mostly used in trigger
mode, application access timeout

#define
CAMERA_AIA_PACKET_AND_PREV_REMOVED_FROM_MEMORY
0x8011
//The subcontract to be accessed no longer exists. Mostly used for
retransmission, the data is no longer in the buffer

#define CAMERA_AIA_PACKET_REMOVED_FROM_MEMORY 0x8012
//CAMERA_AIA_PACKET_AND_PREV_REMOVED_FROM_MEMORY

#define CAMERA_AIA_NO_REF_TIME 0x0813
//There is no reference clock source. Mostly used for time
synchronization command execution

#define CAMERA_AIA_PACKET_TEMPORARILY_UNAVAILABLE x0814

```

```
//Due to the channel bandwidth problem, the current subcontracting  
is temporarily unavailable and needs to be accessed later
```

```
#define CAMERA_AIA_OVERFLOW                                0x0815
```

```
//Data overflow on the device side, usually the queue is full
```

```
#define CAMERA_AIA_ACTION_LATE                             x0816
```

```
//The command execution has exceeded the effective specified time
```

```
#define CAMERA_AIA_ERROR                                    0x8FFF
```

```
//error
```

4 SDK interface function description (C/C++ VB Delphi C# general)

This chapter describes the prototype and description of each interface function in the SDK. For C/C++, VB, Delphi, it can be called directly according to the function name in this manual. In C#, the encapsulated transfer class MVSDK.MvApi is used for calling. For example, MVSDK.MvApi.CameraSdkInit(1) is initialized, and the function calls of other interfaces only need to be prefixed with MVSDK.MvApi.

4.1 CameraSdkInit

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSdkInit  
(  
    int          iLanguageSel  
);
```

Function Description: SDK initialization

Parameter Description: iLanguageSel: Interface language selection, 1: Chinese, 0: English. Others are not currently supported.

return value: When successful, it returns

CAMERA_STATUS_SUCCESS =0; otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: Before calling any other SDK interface, this interface must be called for initialization. This function only needs to be called once during the entire process.

Example:

```
/* Initialize SDK to Chinese interface */  
CameraSdkInit(1);
```

4.2 CameraInit

prototype:

```
MVSDK_API CameraSdkStatus  
CameraInit  
(  
    tSdkCameraDevInfo*    pCameraInfo,  
    int                   emParamLoadMode,  
    int                   emTeam,  
    CameraHandle*         pCameraHandle  
);
```

Function Description: Camera initialization.

Parameter Description:

pCameraInfo: The enumeration description information of the camera is obtained by CameraEnumerateDevice.

emParamLoadMode: The parameter loading method used when the camera is initialized. -1 means use the parameter loading method when exiting last time. The remaining values refer to the emSdkParameterMode type definition.

emTeam: The parameter group used during initialization. -1 means to load the parameter group when it was closed last time.

*pCameraHandle: The handle indicator of the camera. After the initialization is successful, the indicator returns the effective handle of the camera. When calling other camera-related operation interfaces, the handle needs to be passed in, which is mainly used to distinguish between multiple cameras.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: After successful initialization before calling any other camera-related user interface.

Example:

```
/* Initialize the camera and load the parameter settings when it was
turned off last time */
tSdkCameraDevInfo  sCameraList[10];
INT                iCameraNums = 10;
INT                status;
if (CameraEnumerateDevice(sCameraList,&iCameraNums) ==
CAMERA_STATUS_SUCCESS && iCameraNums > 0)
{
    status = CameraInit(&sCameraList[0],-1,-1,&m_hCamera);
    if (status == CAMERA_STATUS_SUCCESS)
    {
        //Camera initialized successfully}
    }
}
```

4.3 CameraInitEx

prototype:

```
MVSDK_API CameraSdkStatus
CameraInitEx
(
    int                iDeviceIndex,
    int                iParamLoadMode,
    int                emTeam,
    CameraHandle*      pCameraHandle
);
```

Function Description: Camera initialization.

Parameter Description:

iDeviceIndex The index number of the camera, starting from 0.
iParamLoadMode The parameter loading method used when the camera is initialized. -1 means use the parameter loading method when exiting last time.

 PARAM_MODE_BY_MODEL means load by model

 PARAM_MODE_BY_SN means loading by serial number

 PARAM_MODE_BY_NAME means load by nickname

 For details, please refer to the definition of emSdkParameterMode in CameraDefine.h.

emTeam The parameter group used during initialization. -1 means to load the parameter group when it was closed last time.

pCameraHandle The handle pointer of the camera. After the initialization is successful, the pointer returns the effective handle of the camera. When calling other camera-related operation interfaces, the handle needs to be passed in, which is mainly used to distinguish between multiple cameras.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: After the initialization is successful, any other camera-related operation interfaces can be called.

Example:

```
/* Initialize the camera and load the parameter settings when it was  
turned off last time */  
tSdkCameraDevInfo  sCameraList[10];  
INT                iCameraNums = 10;  
INT                status;  
if ((iCameraNums  =  
CameraEnumerateDeviceEx(sCameraList,&iCameraNums) )> 0)
```

```

{
    status = CameraInitEx(0,-1,-1,&m_hCamera);
    if (status == CAMERA_STATUS_SUCCESS)
    {
        //Camera initialized successfully
    }
}

```

4.4 CameraInitEx2

prototype:

```

MVSDK_API CameraSdkStatus
CameraInitEx2
(
    char*          CameraName,
    CameraHandle*  pCameraHandle
);

```

Function Description: Camera initialization.

Parameter Description:

CameraName The name of the camera, a string, ends with 0 characters.

pCameraHandle The handle pointer of the camera. After the initialization is successful, the pointer returns the effective handle of the camera. When calling other camera-related operation interfaces, the handle needs to be passed in, which is mainly used to distinguish between multiple cameras.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: After the initialization is successful, any other camera-related operation interfaces can be called.

Example:

```
/* To initialize the camera whose name is Camera1, you need to use
our software to change the camera name to Camera1 in order to
initialize successfully */
tSdkCameraDevInfo  sCameraList[10];
INT                iCameraNums = 10;
INT                status;
if ((iCameraNums ==
CameraEnumerateDeviceEx(sCameraList,&iCameraNums) )> 0)
{
    status = CameraInitEx2("Camera1",-1,-1,&m_hCamera);
    if (status == CAMERA_STATUS_SUCCESS)
    {
        //Camera initialized successfully
    }
}
```

4.5 CameraDisplayInit

prototype:

```
MVSDK_API CameraSdkStatus
CameraDisplayInit
(
    CameraHandle    hCamera,
    HWND            hWndDisplay
);
```

Function Description: Initialize the display module inside the SDK.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

hWndDisplay: The handle of the display window, generally the m_hWnd member of the window.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: This function must be called to initialize before calling CameraDisplayRGB24. If you use your own method for image display in the secondary development (not calling CameraDisplayRGB24), you do not need to call this function.

Example:

```
/* Initialize the display interface, use the SDK packaged display interface*/  
CameraDisplayInit(m_hCamera,m_cPreview.GetSafeHwnd())
```

4.6 CameraSetDisplayMode

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetDisplayMode  
(  
    CameraHandle    hCamera,  
    int              mode  
);
```

Function Description: Set the display mode.

Parameter Description:

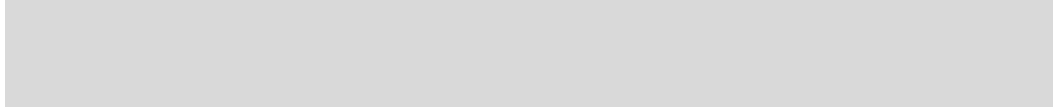
hCamera: The handle of the camera is obtained by the CameraInit function.

mode: Display mode, DISPLAYMODE_SCALE or DISPLAYMODE_REAL, see the definition of emSdkDisplayMode in CameraDefine.h for details

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: You must call CameraDisplayInit for initialization before calling this function.

Example:



4.7 CameraSetDisplayOffset

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetDisplayOffset
(
    CameraHandle    hCamera,
    int              iOffsetX,
    int              iOffsetY
);
```

Function Description: Set the start offset value of the display.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

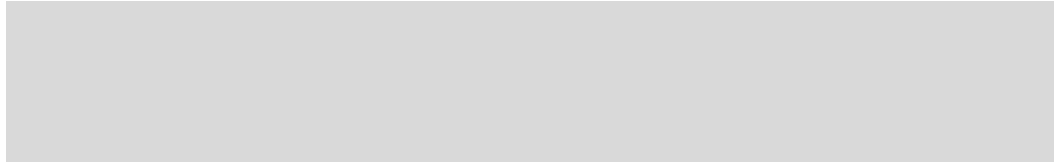
iOffsetX: The X coordinate of the offset.

iOffsetY: The Y coordinate of the offset.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: Only valid when the display mode is DISPLAYMODE_REAL. For example, the size of the display control is 320X240, and the size of the image is 640X480, then when iOffsetX = 160, iOffsetY = 120, the displayed area is the center of the image 320X240. You must call CameraDisplayInit for initialization before calling this function.

Example:



4.8 CameraSetCallbackFunction

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetCallbackFunction
(
    CameraHandle          hCamera,
    CAMERA_SNAP_PROC      pCallbackFunction,
    PVOID                 pContext,
    CAMERA_SNAP_PROC      *pCallbackOld
);
```

Function Description: Set the callback function for image capture.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

pCallbackFunction: Callback function pointer.

pContext: The additional parameter of the callback function. The additional parameter will be passed when the callback function is called. It can be NULL. It is mostly used for multiple cameras to carry additional information.

*pCallbackOld: Used to save the current callback function. Can be NULL.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:

```
/* */  
CameraSetCallbackFunction(m_hCamera,GrabImageCallback,(PVOID)  
this,NULL)
```

4.9 CameraUnInit

prototype:

```
MVSDK_API CameraSdkStatus  
CameraUnInit  
(  
    CameraHandle    hCamera  
);
```

Function Description: Anti-initialize the camera. Free up resources.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```
/* Anti-initialize the camera */  
CameraUnInit(m_hCamera)
```

4.10 CameraPlay

prototype:

```
MVSDK_API CameraSdkStatus  
CameraPlay  
(
```

```

    CameraHandle    hCamera
);

```

Function Description: Let the SDK enter working mode and start receiving images sent from the camera.

Parameter Description: hCamera: The handle of the camera is obtained by the CameraInit function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: If the current camera is in trigger mode, the image will not be updated until the trigger frame is received.

Example:

```

/* The camera starts to capture images */
Cameraplay(m_hCamera);

```

4.11 CameraPause

prototype:

```

MVSDK_API CameraSdkStatus
CameraPause
(
    CameraHandle    hCamera
);

```

Function Description: Let the SDK enter the pause mode and not receive the image data from the camera. At the same time, it will also send a command to the camera to pause output and release the transmission bandwidth.

Parameter Description: hCamera: The handle of the camera is obtained by the CameraInit function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: In the pause mode, the camera parameters can be configured and take effect immediately.

Example:

```
/* The camera starts to capture images */  
CameraPause (m_hCamera);
```

4.12 CameraStop

prototype:

```
MVSDK_API CameraSdkStatus  
CameraStop  
(  
    CameraHandle    hCamera  
);
```

Function Description: To make the SDK enter the stop state, this function is usually called during deinitialization.

Parameter Description: hCamera: The handle of the camera is obtained by the CameraInit function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: This function is called, camera parameters can no longer be configured.

Example:

```
/* The camera starts to capture images */  
CameraStop (m_hCamera);
```

4.13 CameraCreateSettingPage

prototype:

```
MVSDK_API CameraSdkStatus
CameraCreateSettingPage
(
    CameraHandle          hCamera,
    HWND                  hParent,
    Char                   *pWinText,
    CAMERA_PAGE_MSG_PROC  pCallbackFunc,
    PVOID                  pCallbackCtx,
    UINT                   uReserved
);
```

Function Description: Create a property configuration window for the camera.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

hParent: The handle of the main window of the application. Can be NULL.

*pWinText: String pointer, the title bar of the window display.

pCallbackFunc: The callback function of the window message. When the corresponding event occurs, the function pointed to by pCallbackFunc will be called. For example, when the parameter is switched, when pCallbackFunc is called back, the message type is indicated at the entry parameter. This can facilitate the synchronization between the interface developed by you and the UI generated by us. This parameter can be NULL.

pCallbackCtx: Additional parameters of the callback function. Can be NULL. pCallbackCtx will be passed in as one of the parameters when pCallbackFunc is called back. You can use this parameter to make some flexible judgments.

uReserved: Reserved. Must be set to 0.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: When calling this function, the SDK will help you create a configuration window for the camera, which saves you the time to re-develop the camera configuration interface. It is strongly recommended that you use this function to let the SDK create a configuration window for you.

Example:



4.14 CameraCustomizeResolution

prototype:

```
MVSDK_API CameraSdkStatus  
CameraCustomizeResolution  
(  
    CameraHandle          hCamera,  
    tSdkImageResolution   *pImageCustom  
);
```

Function Description: Open custom resolution panel, and by a visual way to configure a custom resolution.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function. Camera name

*pImageCustom: Pointer, return to the custom resolution.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```
/* User-defined resolution */  
CameraCustomizeResolution(m_hCamera,&slImageSize)
```

4.15 CameraShowSettingPage

prototype:

```
MVSDK_API CameraSdkStatus  
CameraShowSettingPage  
(  
    CameraHandle    hCamera,  
    BOOL            bShow  
);
```

Function Description: Set the display status of the camera properties configuration window.

Parameter Description:

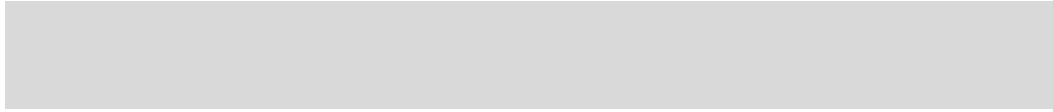
hCamera: The handle of the camera is obtained by the CameraInit function.

bShow: TRUE, display. FALSE, hidden.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: You must call CameraCreateSettingPage to successfully create the camera property configuration window before calling this function to display.

Example:



4.16 CameraSetActiveSettingSubPage

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetActiveSettingSubPage  
(  
    CameraHandle    hCamera,  
    INT             index  
);
```

Function Description: Set the startup page of the camera configuration window.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

index: The index number of the subpage.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: The camera configuration window consists of multiple sub-pages. This function can set which sub-page is currently active and displayed at the forefront.

Example:



4.17 CameraGetInformation

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetInformation  
(  
    CameraHandle    hCamera,  
    char            **pbuffer  
);
```

Function Description: Get the description of the camera

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

**pbuffer: Pointer to the camera description pointer.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.18 CameraImageProcess

prototype:

```
MVSDK_API CameraSdkStatus  
CameraImageProcess  
(  
    CameraHandle    hCamera,  
    BYTE            *pbyIn,  
    BYTE            *pbyOut,  
    tSdkFrameHead   *pFrInfo  
);
```

Function Description: Process the obtained original output image data of the camera, superimpose the processing effects such as saturation, color gain and correction, and noise reduction, and finally get the image data in RGB888 format.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

*pbyIn: The buffer address of the input image data, cannot be NULL.

*pbyOut: The buffer address of the processed image output cannot be NULL.

*pFrInfo: Enter the frame header information of the image. After the processing is completed, the image format uiMediaType in the frame header information will change accordingly.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```
/* */
CameralImageProcess(m_hCamera,
pbyBuffer,m_pblmgBuffer,&tFrameHead);
```

4.19 CameralImageProcessEx

prototype:

```
MVSDK_API CameraSdkStatus
CameralImageProcessEx
(
    CameraHandle    hCamera,
    BYTE*           pbyIn,
    BYTE*           pbyOut,
```

```

    tSdkFrameHead*    pFrInfo,
    UINT              uOutFormat,
    UINT              uReserved
);

```

Function Description: Process the obtained original output image data of the camera, superimpose the processing effects such as saturation, color gain and correction, and noise reduction, and finally get the image data in RGB888 format.

Parameter Description:

hCamera The handle of the camera is obtained by the CameraInit function.

pbyIn The buffer address of the input image data cannot be NULL.

pbyOut The buffer address of the processed image output cannot be NULL.

pFrInfo Enter the frame header information of the image, after processing, the frame header information

uOutFormat The output format of the processed image can be one of CAMERA_MEDIA_TYPE_MONO8, CAMERA_MEDIA_TYPE_RGB, CAMERA_MEDIA_TYPE_BGR, CAMERA_MEDIA_TYPE_RGBA8, CAMERA_MEDIA_TYPE_RGBA8.

pbyIn The corresponding buffer size must match the format specified by uOutFormat.

uReserved Reserved parameter, must be set to 0

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

4.20 CameraDisplayRGB24

prototype:

```
MVSDK_API CameraSdkStatus
CameraDisplayRGB24
(
    CameraHandle    hCamera,
    BYTE            *pbyRGB24,
    tSdkFrameHead   *pFrInfo
);
```

Function Description: Display the image.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pbyRGB24: Image data buffer, RGB888 format.

*pFrInfo: The frame header information of the image.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: You must call CameraDisplayInit for initialization before calling this function.

Example:

```
/*Call the display function to display the image*/
CameraDisplayRGB24(pThis->m_hCamera, pThis->m_pFrameBuffer,
pFrameHead);
```

4.21 CameraSetDisplaySize

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetDisplaySize
(
    CameraHandle    hCamera,
```

```

        INT          iWidth,
        INT          iHeight
    );

```

Function Description:Set the display size of the control.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iWidth: width

iHeight: height

return value:When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: You must call CameraDisplayInit for initialization before calling this function.

Example:

```

/* */
CameraSetDisplaySize(m_hCamera,rect.right - rect.left,rect.bottom -
rect.top)

```

4.22 CameraGetImageBuffer

prototype:

```

MVSDK_API CameraSdkStatus
CameraGetImageBuffer
(
    CameraHandle      hCamera,
    tSdkFrameHead*    pFrameInfo,
    BYTE**            pbyBuffer,
    UINT              wTimes
);

```

Function Description: Obtain a frame of image data.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pFrameInfo: The frame header information pointer of the image.

**pbyBuffer: Pointer to the buffer pointer of the image data. Since a zero-copy mechanism is used to improve efficiency, an index pointing to an indicator is used here.

wTimes: The timeout period for capturing images. The unit is milliseconds. If the image is not obtained within wTimes, the function will return timeout information.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: In order to improve efficiency, the SDK adopts a zero-copy mechanism when capturing images.

CameraGetImageBuffer actually obtains a buffer address in the kernel. After the function is successfully called, CameraReleaseImageBuffer must be called to release the buffer obtained by CameraGetImageBuffer so that the kernel can continue to use the buffer.

Example:

```
/* */  
CameraGetImageBuffer(pThis->m_hCamera,&sFrameInfo,&pbyBuffer,1000);
```

4.23 CameraGetImageBufferEx

prototype:

```

MVSDK_API CameraSdkStatus
CameraGetImageBufferEx
(
    CameraHandle      hCamera,
    INT*              piWidth,
    INT*              piHeight,
    UINT              wTimes
);

```

Function Description: Obtain a frame of image data. Different from the CameraGetImageBuffer function, this function gets the image data in RGB format. **There is no need to call CameraImageProcess function and CameraReleaseImageBuffer function later.**

Parameter Description:

hCamera The handle of the camera is obtained by the CameraInit function.

piWidth Int pointers, Returns the width of the image

piHeight Int pointers, Returns the height of the image **wTimes**: the timeout period for capturing the image. The unit is milliseconds. If the image is not obtained within wTimes, the function will return timeout information.

return value: On success, **it returns the first address of the image data buffer**. On failure, it returns NULL or 0.

4.24 CameraReleaseImageBuffer

prototype:

```

MVSDK_API CameraSdkStatus
CameraReleaseImageBuffer
(
    CameraHandle      hCamera,
    BYTE              *pbyBuffer
);

```

Function Description: Release the buffer obtained by CameraGetImageBuffer.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pbyBuffer: The buffer address obtained by CameraGetImageBuffer.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```
/* */  
CameraReleaseImageBuffer(pThis->m_hCamera,pbyBuffer);
```

4.25 CameraGetFrameStatistic

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetFrameStatistic  
(  
    CameraHandle      hCamera,  
    tSdkFrameStatistic *psFrameStatistic  
);
```

Function Description: Get statistical information about the frame rate received by the camera, including error frames and dropped frames.

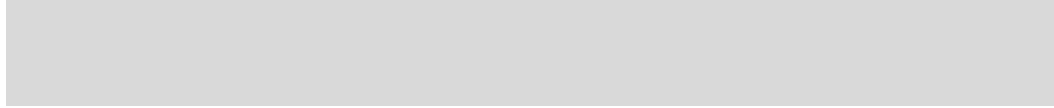
Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*psFrameStatistic: Pointer, returns statistical information.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.26 CameraSetNoiseFilter

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetNoiseFilter  
(  
    CameraHandle    hCamera,  
    BOOL            bEnable  
);
```

Function Description: Set the enable state of the image noise reduction module.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

bEnable: TRUE, enable; FALSE, disable.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.27 CameraGetNoiseFilterState

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetNoiseFilterState  
(  
    CameraHandle    hCamera,  
    BOOL            *pEnable  
);
```

Function Description: Get the enabled state of the image noise reduction module.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pEnable: Indicator, return status. TRUE, to enable.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:

4.28 CameraInitRecord

prototype:

```
MVSDK_API CameraSdkStatus  
CameraInitRecord  
(  
    CameraHandle    hCamera,  
    int             iFormat,  
    char            *pcSavePath,
```

```

        BOOL            b2GLimit,
        DWORD           dwQuality,
        int             iFrameRate
    );

```

Function Description: Initializing a video.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iFormat: The video format currently only supports uncompressed and MSCV. 0: No compression; 1: MSCV mode compression.

*pcSavePath: Path to save video files.

b2GLimit: If it is TRUE, the file is automatically divided when it is larger than 2G.

dwQuality: The larger the quality factor of the video, the better the quality. Range 1 to 100.

iFrameRate: The frame rate of the recording. It is recommended to set the frame rate higher than the actual acquisition frame rate, so that frames will not be missed.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```

// Example code for CameraStartRecord

```

4.29 CameraStopRecord

prototype:

```

MVSDK_API CameraSdkStatus

```

```

CameraStopRecord
(
    CameraHandle    hCamera
);

```

Function Description: End this video.

Parameter Description: hCamera: The handle of the camera is obtained by the CameraInit function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.30 CameraPushFrame

prototype:

```

MVSDK_API CameraSdkStatus
CameraPushFrame
(
    CameraHandle    hCamera,
    BYTE            *pbyImageBuffer,
    tSdkFrameHead   *pFrInfo
);

```

Function Description: Store one frame of data in the video stream.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pbyImageBuffer: The image data buffer must be in RGB format.

*pFrInfo: Image frame header information.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: CameraInitRecord must be called to call this function. After CameraStopRecord is called, this function cannot be called again. Since our frame header information carries the timestamp information of the image collection, the recording can be accurately time synchronized without being affected by the unstable frame rate.

Example:



4.31 CameraSaveImage

prototype:

MVSDK_API CameraSdkStatus

CameraSaveImage

```
(  
    CameraHandle      hCamera,  
    LPCTSTR          lpszFileName,  
    BYTE              *pbyImageBuffer,  
    tSdkFrameHead     *pFrInfo,  
    BYTE              byFileType,  
    BYTE              byQuality  
);
```

Function Description: Save the image buffer data into a picture file.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

lpzFileName: The full path of the image file.

*pbyImageBuffer: The data buffer of the image.

*pFrInfo: The frame header information of the image.

byFileType: The format in which the image is saved. For the value range, see the type definition of emSdkFileType in CameraDefine.h. Currently it supports four formats: BMP, JPG, PNG, and RAW. Where RAW represents the original data output by the camera, saving RAW format files requires that pbyImageBuffer and pFrInfo are the data obtained by CameraGetImageBuffer, and have not been converted into BMP format by CameraImageProcess; Conversely, if you want to save in BMP, JPG or PNG format, pbyImageBuffer and pFrInfo are RGB format data processed by CameraImageProcess. Specific usage can refer to Advanced routines.

byQuality: The quality factor of image saving. This parameter is valid only when saving in JPG format, ranging from 1 to 100. The remaining formats can be written as 0.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:



4.32 CameraSetMirror

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetMirror  
(  
    CameraHandle    hCamera,  
    INT             iDir,  
    BOOL            bEnable  
);
```

Function Description: Set the image mirroring operation. The mirror operation is divided into horizontal and vertical directions.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iDir: Indicates the direction of the mirror image to be obtained. 0: horizontal direction; 1: vertical direction.

bEnable: TRUE, enable mirroring; FALSE, disable mirroring

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.33 CameraGetMirror

prototype:

```
MVSDK_API CameraSdkStatus
```

```

CameraGetMirror
(
    CameraHandle    hCamera,
    INT             iDir,
    BOOL            *pbEnable
);

```

Function Description: Obtain the mirror state of the image.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iDir: Indicates the direction of the mirror image to be obtained. 0: horizontal direction; 1: vertical direction.

*pbEnable:

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:

```

// Example code for CameraGetMirror

```

4.34 CameraSetMonochrome

prototype:

```

MVSDK_API CameraSdkStatus
CameraSetMonochrome
(
    CameraHandle    hCamera,
    BOOL            bEnable
);

```

Function Description: Set the enable of the color to monochrome function.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

bEnable: TRUE means to convert the color image to monochrome.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.35 CameraGetMonochrome

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetMonochrome
(
    CameraHandle    hCamera,
    BOOL            *pbEnable
);
```

Function Description: Get the enable status of the color conversion to monochrome function.

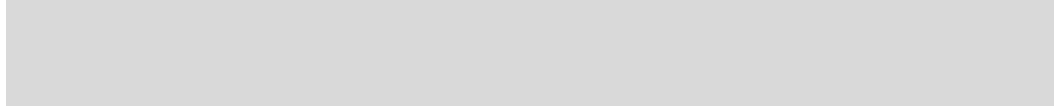
Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

*pbEnable: pointer. Returning TRUE indicates that the function of converting color images to monochrome images is enabled.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.36 CameraSetInverse

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetInverse  
(  
    CameraHandle    hCamera,  
    BOOL            bEnable  
);
```

Function Description: Set the enable of color image color flip function. (negative)

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

bEnable: TRUE means that the image color flip function is turned on, and the effect similar to the film negative can be obtained.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:



4.37 CameraGetInverse

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetInverse  
(  
    CameraHandle    hCamera,  
    BOOL            *pbEnable  
);
```

Function Description: Get the enable state of the image color inversion function.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pbEnable: Pointer, return to the function enable state.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:

4.38 CameraSetImageResolution

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetImageResolution  
(  
    CameraHandle    hCamera,  
    tSdkImageResolution *pImageResolution  
);
```

Function Description: Set the resolution of the preview.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pImageResolution: Structure pointer, used to set the current resolution.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.39 CameraGetImageResolution

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetImageResolution
(
    CameraHandle          hCamera,
    tSdkImageResolution   *psCurlImageResolution
);
```

Function Description: Get the resolution of the current preview.

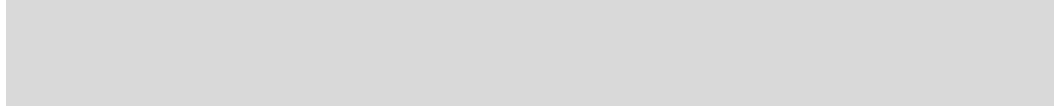
Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*psCurlImageResolution: Structure pointer, used to return the current resolution.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.40 CameraGetMediaType

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetMediaType
(
    CameraHandle    hCamera,
    INT             *piMediaType
);
```

Function Description: Get the format index number of the raw data currently output by the camera.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*piMediaType: Pointer, used to return the index number of the current format type. The properties of the camera are obtained by CameraGetCapability. In the pMediaTypeDesc member of the tSdkCameraCapability structure, the format supported by the camera is saved in the form of an array. The index number pointed to by piMediaType is the index number of the

array.pMediaTypeDesc[*piMediaType].iMediaType represents the encoding of the current format. Please refer to the [Image Format Definition] section in CameraDefine.h for the encoding.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.41 CameraSetMediaType

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetMediaType
(
    CameraHandle    hCamera,
    INT             iMediaType
);
```

Function Description: Set the format index number of the raw data currently output by the camera.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*piMediaType: Pointer, used to return the index number of the current format type. The properties of the camera are obtained by CameraGetCapability. In the pMediaTypeDesc member of the

tSdkCameraCapability structure, the format supported by the camera is saved in the form of an array. The index number pointed to by piMediaType is the index number of the array.pMediaTypeDesc[*piMediaType].iMediaFormat represents the encoding of the current format. Please refer to the [Image Format Definition] section in CameraDefine.h for the encoding.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.42 CameraSetAeState

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetAeState
(
    CameraHandle    hCamera,
    BOOL            bState
);
```

Function Description: Set the camera exposure mode. Automatic or manual.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

bState: TRUE, enable automatic exposure; FALSE, stop automatic exposure.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.43 CameraGetAeState

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetAeState  
(  
    CameraHandle    hCamera,  
    BOOL            *pbAeState  
);
```

Function Description: Get the current exposure mode of the camera.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

*pbAeState: Indicator, used to return to the enabled state of automatic exposure.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.44 CameraSetAeTarget

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetAeTarget  
(  
    CameraHandle    hCamera,  
    INT             iAeTarget  
);
```

Function Description: Set the brightness target value for automatic exposure.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iAeTarget: Brightness target value.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: The setting range is obtained by the CameraGetCapability function.

Example:

4.45 CameraGetAeTarget

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetAeTarget
```

```
(
    CameraHandle    hCamera,
    INT             *piAeTarget
);
```

Function Description: Obtain the brightness target value for automatic exposure.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*piAeTarget: Pointer, return the target value.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.46 CameraSetExposureTime

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetExposureTime
(
    CameraHandle    hCamera,
    double          fExposureTime
);
```

Function Description: Set the exposure time. The unit is microseconds.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

fExposureTime: Exposure time in microseconds.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: For CMOS sensors, the unit of exposure is calculated in line, Therefore, the exposure time cannot be continuously adjusted at the microsecond level. Instead, it will choose the whole line. After calling this function to set the exposure time, it is recommended to adjust CameraGetExposureTime to get the actual set value.

Example:



4.47 CameraGetExposureTime

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetExposureTime
(
    CameraHandle    hCamera,
    double          *pfExposureTime
);
```

Function Description: Get the exposure time. The unit is microseconds.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

pfExposureTime: The pointer returns the exposure time in microseconds.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

4.48 CameraGetExposureLineTime

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetExposureLineTime
(
    CameraHandle    hCamera,
    double          *pfLineTime
);
```

Function Description: Get the exposure time of a line.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pfLineTime: Line exposure time, in microseconds.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.49 CameraSetAnalogGain

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetAnalogGain
```

```
(
    CameraHandle    hCamera,
    INT             usAnalogGain
);
```

Function Description: Set the image simulation gain value of the camera.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

usAnalogGain: The set analog gain value.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: This value is multiplied by sExposeDesc.fAnalogGainStep in the camera property structure obtained by CameraGetCapability to get the actual image signal magnification.

Example:

4.50 CameraGetAnalogGain

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetAnalogGain
(
    CameraHandle    hCamera,
    INT             *pusAnalogGain
);
```

Function Description: Obtain the analog gain value of the image signal.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pusAnalogGai: Pointer, returns the current analog gain value.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:



4.51 CameraSetAeWindow

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetAEWindow
(
    CameraHandle    hCamera,
    int             iHOff,
    int             iVOff,
    int             iWidth,
    int             iHeight
);
```

Function Description: Set the reference window for automatic exposure

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iHOff: The abscissa of the upper left corner of the window

iVOff: The ordinate of the upper left corner of the window

iWidth: The width of the window

iHeight: The height of the window

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: The window is set to the center 1/2 size of each resolution. Can follow the change with the change of resolution; if the window position range determined by iHOff, iVOff, iWidth, iHeight exceeds the current resolution range, the center 1/2 size window will be automatically used

Example:



4.52 CameraGetAEWindow

prototype:

MVSDK_API CameraSdkStatus

CameraGetAEWindow

```
(  
    CameraHandle    hCamera,  
    INT*            piHOff,  
    INT*            piVOff,  
    INT*            piWidth,  
    INT*            piHeight  
);
```

Function Description: Get the position of the automatic exposure reference window.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

piHOff: Pointer, returns the abscissa value of the upper left corner of the window position.

piVOff: Pointer, return the ordinate value of the upper left corner of the window position.

piWidth: : Indicators,Returns the width of the window.

piHeight: Indicators,Returns the height of the window.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:



4.53 CameraSetSharpness

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetSharpness
(
    CameraHandle    hCamera,
    INT             iSharpness
);
```

Function Description: Set the sharpening parameters for image processing.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

iSharpness: Sharpening parameters. The range is obtained by CameraGetCapability, generally [0,100], 0 means that sharpening is turned off.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.54 CameraGetSharpness

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetSharpness
(
    CameraHandle    hCamera,
    INT             *piSharpness
);
```

Function Description: Get the current sharpening setting value.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

* piSharpness: The pointer returns to the current sharpening setting value.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.55 CameraSetOnceWB

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetOnceWB  
(  
    CameraHandle    hCamera  
);
```

Function Description: In manual white balance mode, calling this function will perform white balance once. The effective time is when the next frame of image data is received.

Parameter Description: hCamera: The handle of the camera is obtained by the CameraInit function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.56 CameraSetOnceBB

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetOnceBB  
(
```

```

    CameraHandle    hCamera
);

```

Function Description: Perform a black balance operation.

Parameter Description: hCamera: The handle of the camera is obtained by the CameraInit function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```

// Example code for CameraBlackBalance function

```

4.57 CameraSetLutMode

prototype:

```

MVSDK_API CameraSdkStatus
CameraSetLutMode
(
    CameraHandle    hCamera,
    INT              emLutMode
);

```

Function Description: Set the look-up table conversion mode of the camera LUT mode.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

emLutMode: LUTMODE_PARAM_GEN means that the LUT table is dynamically generated from gamma and contrast parameters.

LUTMODE_PRESET means to use the preset LUT table.

LUTMODE_USER_DEF means to use a user-defined LUT table.

The definition of LUTMODE_PARAM_GEN refers to the emSdkLutMode type in CameraDefine.h.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.58 CameraGetLutMode

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetLutMode
(
    CameraHandle    hCamera,
    INT             *pemLutMode
);
```

Function Description: Obtain the look-up table transformation mode of the camera LUT mode

Parameter Description:

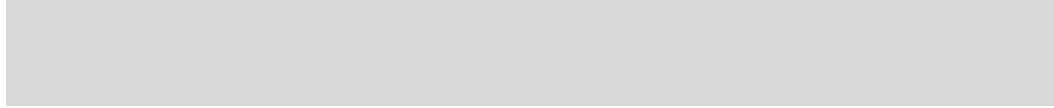
hCamera: The handle of the camera is obtained by the Cameralnit function.

*pemLutMode: Pointer, return to the current LUT mode.

The meaning is the same as the emLutMode parameter in CameraSetLutMode.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.59 CameraSelectLutPreset

prototype:

```
MVSDK_API CameraSdkStatus
CameraSelectLutPreset
(
    CameraHandle    hCamera,
    int             iSel
);
```

Function Description: Select the LUT table in the default LUT mode. You must first use CameraSetLutMode to set the LUT mode to the default mode.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iSel: The index number of the table. The number of tables is obtained by CameraGetCapability.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.60 CameraGetLutPresetSel

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetLutPresetSel  
(  
    CameraHandle    hCamera,  
    int*            piSel  
);
```

Function Description: Get the index number of the LUT table in the default LUT mode.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

piSel: Pointer, returns the index number of the table.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.61 CameraSetCustomLut

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetCustomLut  
(  
    CameraHandle    hCamera,
```

```

        BYTE*          pLut
    );

```

Function Description: Set up a custom LUT table.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

pLut: Pointer, pointing to the address of the LUT table.
The LUT table is 256 bytes in size, representing the mapping values corresponding to the color channels from 0 to 256.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: You must first use CameraSetLutMode to set the LUT mode to a custom mode.

Example:

```

// Example code for CameraSetLutMode

```

4.62 CameraGetCustomLut

prototype:

```

MVSDK_API CameraSdkStatus
CameraGetCustomLut
(
    CameraHandle    hCamera,
    BYTE*          pLut
);

```

Function Description: Get the currently used custom LUT table.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

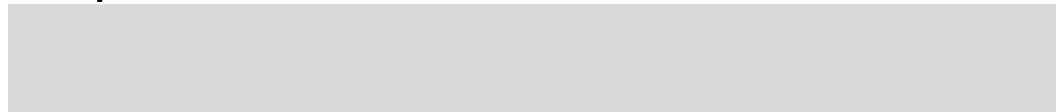
pLut: Pointer, used to return the currently used custom LUT table. The LUT table is 256 bytes in size, representing the mapping values corresponding to the color channels from 0 to 256. The mode value of the custom LUT table is 0 to 255. Represents a 1:1 linear image without changing the original value.

return value: When successful, it returns

CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:



4.63 CameraGetCurrentLut

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetCurrentLut
(
    CameraHandle    hCamera,
    BYTE*           pLut
);
```

Function Description: Obtain the current LUT table of the camera, which can be called in any LUT mode to visually observe the changes of the LUT curve.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

pLut: Pointer, used to return the current LUT table of the camera.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.64 CameraSetWbMode

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetWbMode
(
    CameraHandle    hCamera,
    BOOL            bAuto
);
```

Function Description: Set the camera white balance mode. Divided into manual and automatic two ways.

Parameter Description:

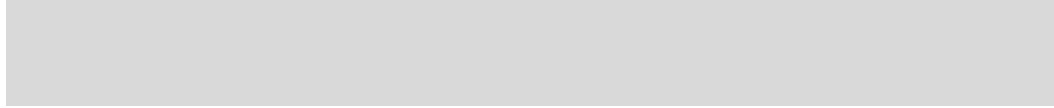
hCamera: The handle of the camera is obtained by the CameraInit function.

bAuto: TRUE, it means the automatic mode is enabled.

FALSE, it means using the manual mode, the white balance is performed once by calling CameraSetOnceWB.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.65 CameraGetWbMode

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetWbMode  
(  
    CameraHandle    hCamera,  
    BOOL            *pbAuto  
);
```

Function Description: Get the current white balance mode.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pbAuto: Pointer, return TRUE means automatic mode, FALSE means manual mode.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.66 CameralsWbWinVisible

prototype:

```
MVSDK_API CameraSdkStatus  
CameralsWbWinVisible  
(  
    CameraHandle    hCamera,  
    BOOL            *pbShow  
);
```

Function Description: Obtain the display status of the white balance window.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pbShow: Pointer, returns TRUE, it means the window is visible.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.67 CameraGetWbWindow

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetWbWindow  
(  
    CameraHandle    hCamera,  
    INT*            PiHOff,  
    INT*            PiVOff,
```

```

        INT*          PiWidth,
        INT*          PiHeight
    );

```

Function Description: Get the position of the white balance reference window.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

PiHOff: Indicators,Returns the abscissa of the upper left corner of the reference window.

PiVOff: Indicators,Returns the ordinate of the upper left corner of the reference window.

PiWidth: Indicators,Returns the width of the reference window.

PiHeight: Indicators,Returns the height of the reference window.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```

// Example code for CameraSetWbWindow

```

4.68 CameraSetWbWindow

prototype:

```

MVSDK_API CameraSdkStatus
CameraSetWbWindow
(
    CameraHandle  hCamera,
    INT           iHOff,

```

```

        INT          iVOff,
        INT          iWidth,
        INT          iHeight
    );

```

Function Description: Set the position of the white balance reference window.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iHOff: The abscissa of the upper left corner of the reference window

iVOff: The ordinate of the upper left corner of the reference window

iWidth: Reference window width

iHeight: The height of the reference window

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:

```

// Example code for CameraWhiteBalanceWindowSet

```

4.69 CameraImageOverlay

prototype:

```

MVSDK_API CameraSdkStatus
CameraImageOverlay
(
    CameraHandle  hCamera,

```

```

    BYTE          *pRgbBuffer,
    tSdkFrameHead *pFrInfo
);

```

Function Description: Superimpose graphics such as crosshairs, white balance reference window, and auto exposure reference window on the input image data. Only the crosshairs and reference windows that are set to be visible can be superimposed.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pRgbBuffer: Image data buffer.

*pFrInfo: The frame header information of the image.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```

// Example code for CameraSetCrossLine

```

4.70 CameraSetCrossLine

prototype:

```

MVSDK_API CameraSdkStatus
CameraSetCrossLine
(
    CameraHandle  hCamera,
    int           iLine,
    INT           x,
    INT           y,
    UINT          color,

```

```

        BOOL        bVisible
    );

```

Function Description: Set the parameters of the specified crosshair.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iLine: Indicates the state of the crosshairs to be set. The range is [0,8], a total of 9 items.

x: The abscissa value of the center position of the crosshair.

y: The ordinate value of the center of the crosshair.

color: The color of the crosshair, the format is (R)|(G<<8)|(B<<16))

bVisible: The display status of the crosshairs. TRUE means display.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: Only the crosshairs set to display state will be superimposed on the image after calling CameraImageOverlay.

Example:

```

// Example code for CameraSetCrossLine

```

4.71 CameraGetCrossLine

prototype:

```

MVSDK_API CameraSdkStatus
CameraGetCrossLine
(

```

```

CameraHandle    hCamera,
int             iLine,
INT             *px,
INT             *py,
UINT            *pcolor,
BOOL            *pbVisible
);

```

Function Description: Get the status of the specified crosshair.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iLine: Indicates the status of which crosshair is to be obtained. The range is [0,8], a total of 9 items.

*px: Pointer, return the abscissa of the center position of the crosshair.

*py: Pointer, return the abscissa of the center position of the crosshair.

*pcolor: Pointer, return the color of the cross line, the format is (R|(G<<8)|(B<<16)).

*pbVisible: If the pointer returns TRUE, it means that the crosshair is visible.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```

// Example code for CameraGetCrosshairStatus
// ...

```

4.72 CameraSetGain

prototype:

```
MVSDK_API CameraSdkStatus
```

```
CameraSetGain
```

```
(  
    CameraHandle    hCamera,  
    int              iRGain,  
    int              iGGain,  
    int              iBGain  
);
```

Function Description: Set the digital gain of the image.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iRGain: The gain value of the red channel.

iGGain: The gain value of the green channel.

iBGain: The gain value of the blue channel.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: The setting range is expressed by the sRgbGainRange member in the camera property structure obtained by CameraGetCapability. The actual magnification is the set value/100.

Example:

4.73 CameraGetGain

prototype:

```
MVSDK_API CameraSdkStatus
```

```
CameraGetGain
```

```
(
    CameraHandle    hCamera,
    int              *piRGain,
    int              *piGGain,
    int              *piBGain
);
```

Function Description: Get the digital gain of image processing.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

*piRGain: The pointer returns the digital gain value of the red channel.

*piGGain: The pointer returns the digital gain value of the green channel.

*piBGain: The pointer returns the digital gain value of the blue channel.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.74 CameraSetGamma

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetGamma
(
    CameraHandle    hCamera,
    int              iGamma
```

```
);
```

Function Description: Set the Gamma value in the dynamic LUT generation mode.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iGamma: The Gamma value to be set.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: The set value will be saved in the SDK immediately, but it will only take effect when the camera is in the LUT mode generated by dynamic parameters. Please refer to the function description part of CameraSetLutMode.

Example:

4.75 CameraGetGamma

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetGamma
(
    CameraHandle    hCamera,
    int              *piGamma
);
```

Function Description: Obtain the Gamma value in the LUT dynamic generation mode. Please refer to the function description of CameraSetGamma function.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

*piGamma: Pointer, returns the current Gamma value.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:

4.76 CameraSetSaturation

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetSaturation
(
    CameraHandle    hCamera,
    INT             iSaturation
);
```

Function Description: Set the saturation of image processing.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

iSaturation: The set saturation value.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: Not valid for monochrome cameras.

Example:



4.77 CameraGetSaturation

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetSaturation
(
    CameraHandle    hCamera,
    INT             *piSaturation
);
```

Function Description: Obtain the saturation of image processing.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*piSaturation: Pointer, returns the saturation value of the current image processing.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.78 CameraSetContrast

prototype:

```
MVSDK_API CameraSdkStatus
```

```

CameraSetContrast
(
    CameraHandle    hCamera,
    INT             iContrast
);

```

Function Description: Set the contrast value in the LUT dynamic generation mode.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iContrast: The set contrast value.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: The set value will be saved in the SDK immediately, but it will only take effect when the camera is in the LUT mode generated by dynamic parameters. Please refer to the function description part of CameraSetLutMode.

Example:

```

// Example code for CameraSetContrast

```

4.79 CameraGetContrast

prototype:

```

MVSDK_API CameraSdkStatus
CameraGetContrast
(
    CameraHandle    hCamera,
    INT             *piContrast
);

```

Function Description: Obtain the contrast value in the LUT dynamic generation mode.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

*piContrast: Pointer, returns the current contrast value.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.80 CameraSetFrameSpeed

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetFrameSpeed
(
    CameraHandle    hCamera,
    INT             iFrameSpeedSel
);
```

Function Description: Set the frame rate of the camera output image.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

iFrameSpeedSel: The index number of the selected frame rate mode, Range from 0 to iFrameSpeedDesc-1 in the information

structure obtained by
CameraGetCapability

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.81 CameraGetFrameSpeed

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetFrameSpeed
(
    CameraHandle    hCamera,
    INT             *piFrameSpeedSel
);
```

Function Description: Select the index number to obtain the frame rate of the camera output image.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*piFrameSpeedSel: Pointer, returns the index number of the selected frame rate mode.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:

4.82 CameraSetAntiFlick

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetAntiFlick  
(  
    CameraHandle    hCamera,  
    BOOL            bEnable  
);
```

Function Description: Set the enabled state of the anti-stroboscopic function during automatic exposure.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

bEnable: TRUE, turn on the anti-stroboscopic function; FALSE, turn off the function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: It is invalid for manual exposure mode.

Example:

4.83 CameraGetAntiFlick

prototype:

```

MVSDK_API CameraSdkStatus
CameraGetAntiFlick
(
    CameraHandle    hCamera,
    BOOL            *pbEnable
);

```

Function Description: Obtain the enabled state of the anti-stroboscopic function during automatic exposure.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pbEnable: Pointer, return to the enable state of the function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```

// Example code for CameraGetAntiFlick

```

4.84 CameraGetLightFrequency

prototype:

```

MVSDK_API CameraSdkStatus
CameraGetLightFrequency
(
    CameraHandle    hCamera,
    int             *piFrequencySel
);

```

Function Description: When auto exposure is obtained, the frequency selection of stroboscopic elimination.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

*piFrequencySel: Pointer, returns the selected index number. 0:50HZ 1:60HZ

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.85 CameraSetLightFrequency

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetLightFrequency
(
    CameraHandle    hCamera,
    int             iFrequencySel
);
```

Function Description: Set the frequency of stroboscopic suppression during automatic exposure.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

iFrequencySel: 0:50HZ , 1:60HZ

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.86 CameraSetTransPackLen

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetTransPackLen
(
    CameraHandle    hCamera,
    INT             iPackSel
);
```

Function Description: Set the packet size of the image data transmitted by the camera. (This method has been abandoned, and the SDK has been changed to auto-negotiation subcontracting. When using GIGE cameras, please enable the jumbo frame transmission function of the network card. After enabling, the SDK will automatically enable maximum subcontracting to optimize performance)

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iPackSel: The index number of the subpacket length selection. The subpacket length can be expressed by the pPackLenDesc member in the acquired camera attribute structure, and the

iPackLenDesc member indicates the maximum number of optional subpackaging modes.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.87 CameraGetTransPackLen

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetTransPackLen
(
    CameraHandle    hCamera,
    INT             *piPackSel
);
```

Function Description: Get the selection index number of the current transmission packet size of the camera.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*piPackSel: Pointer, returns the index number of the currently selected packet size.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.88 CameraWriteSN

prototype:

```
MVSDK_API CameraSdkStatus  
CameraWriteSN  
(  
    CameraHandle    hCamera,  
    BYTE            *pcSN,  
    INT             iLevel  
);
```

Function Description: Set the serial number of the camera. The serial number of our camera is divided into 3 levels. The factory default camera serial number of level 0, Level 1 and Level 2 are reserved for secondary development. The length of each level sequence number is 32 bytes.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

*pcSN: Sequence number buffer.

iLevel: The serial number level to be set can only be 1 or 2.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:

4.89 CameraReadSN

prototype:

```
MVSDK_API CameraSdkStatus  
CameraReadSN  
(  
    CameraHandle    hCamera,  
    BYTE            *pcSN,  
    INT             iLevel  
);
```

Function Description: Read the serial number of the specified level of the camera.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pcSN: Sequence number buffer.

iLevel: The sequence number level to be read. It can only be 1 and 2.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.90 CameraSaveParameter

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSaveParameter  
(  
    CameraHandle    hCamera,
```

```

        INT          iTeam
    );

```

Function Description: Save the current camera parameters to the specified parameter group. The camera provides four groups of spaces A, B, C, and D to save parameters.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iTeam: PARAMETER_TEAM_A Save to group A,
 PARAMETER_TEAM_B Save to group B,
 PARAMETER_TEAM_C Save to group C,
 PARAMETER_TEAM_D Save to group D.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```

// Example code for CameraSaveParameter

```

4.91 CameraLoadParameter

prototype:

```

MVSDK_API CameraSdkStatus
CameraLoadParameter
(
    CameraHandle    hCamera,
    INT             iTeam
);

```

Function Description: Load the parameters of the specified group into the camera.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iTeam: PARAMETER_TEAM_A Save to group A,
PARAMETER_TEAM_B Save to group B ,
PARAMETER_TEAM_C Save to group C,
PARAMETER_TEAM_D Save to group D.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.92 CameraReadParameterFromFile

prototype:

```
MVSDK_API CameraSdkStatus  
CameraReadParameterFromFile  
(  
    CameraHandle    hCamera,  
    char            *sFileName  
);
```

Function Description: Load parameters from the specified parameter file on the PC. The camera parameters of our company are saved on the PC as a file with the suffix of .config, located in the Camera\Configs folder under the installation.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

*sFileName: The full path of the parameter file.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.93 CameraGetCurrentParameterGroup

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetCurrentParameterGroup
(
    CameraHandle    hCamera,
    INT             *piTeam
);
```

Function Description: Get the currently selected parameter group.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

*piTeam: Pointer, return to the currently selected parameter group.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.94 CameraEnumerateDevice

prototype:

```
MVSDK_API CameraSdkStatus  
CameraEnumerateDevice  
(  
    tSdkCameraDevInfo    *pDSCameraList,  
    INT                  *piNums  
);
```

Function Description: Enumerate devices and establish a list of devices.

Parameter Description:

* pDSCameraList: Device list array pointer.

*piNums : The number indicator of the device, the number of elements in the pDSCameraList array is passed in when calling, and the number of devices actually found is saved when the function returns.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: Before calling CameraInit, this function must be called to obtain device information. The value pointed to by piNums must be initialized and should not exceed the number of elements in the pDSCameraList array, otherwise it may cause memory overflow.

Example:

4.95 CameraGetCapability

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetCapability
(
    CameraHandle          hCamera,
    tSdkCameraCapbility *pCameraInfo
);
```

Function Description: Get the characteristic description structure of the camera.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pCameraInfo: Pointer, returns the structure describing the camera's characteristics.

tSdkCameraCapbility is defined in CameraDefine.h.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.96 CameraSetTriggerCount

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetTriggerCount
(
    CameraHandle          hCamera,
```

```

        INT          iCount
    );

```

Function Description: Set the number of trigger frames in trigger mode.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iCount: The number of frames acquired by one trigger.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: It is valid for both software trigger and hardware trigger mode. The default is 1 frame, that is, one frame of image is collected by a trigger signal.

Example:

```

// Example code for CameraSetTriggerCount

```

4.97 CameraGetTriggerCount

prototype:

```

MVSDK_API CameraSdkStatus
CameraGetTriggerCount
(
    CameraHandle    hCamera,
    INT             *piCount
);

```

Function Description: Get the number of frames for a trigger.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*piCount: Pointer, returns the number of frames acquired by one trigger.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.98 CameraGetTriggerDelayTime

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetTriggerDelayTime
(
    CameraHandle    hCamera,
    UINT            *puDelayTimeUs
);
```

Function Description: Get the hard trigger delay time currently set.

Parameter Description:

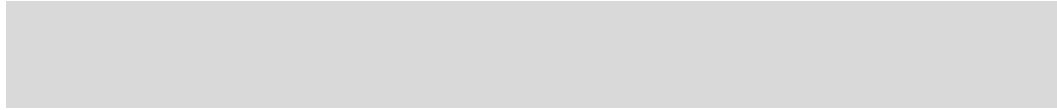
hCamera: The handle of the camera is obtained by the CameraInit function.

*puDelayTimeUs: Pointer, return delay time, Unit microsecond.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: Only some models of cameras support this function.

Example:



4.99 CameraSetTriggerDelayTime

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetTriggerDelayTime  
(  
    CameraHandle    hCamera,  
    UINT            uDelayTimeUs  
);
```

Function Description: Set the trigger delay time in hardware trigger mode, in microseconds.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

uDelayTimeUs: Trigger delay. The unit is microseconds.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Note: Only some models of cameras support this function.

Example:



4.100 CameraSoftTrigger

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSoftTrigger  
(  
    CameraHandle    hCamera  
);
```

Function Description: Perform a soft trigger.

Parameter Description: hCamera: The handle of the camera is obtained by the CameraInit function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.101 CameraSetTriggerMode

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetTriggerMode  
(  
    CameraHandle    hCamera,  
    INT             iTriggerModeSel  
);
```

Function Description: Set the trigger mode of the camera.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iTriggerModeSel: Mode selection index number. The mode that can be set is obtained by the CameraGetCapability function. Please refer to the definition of tSdkCameraCapbility in CameraDefine.h. In general, 0 represents a continuous acquisition mode; 1 means software trigger mode; 2 means hardware trigger mode.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.102 CameraGetTriggerMode

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetTriggerMode
(
    CameraHandle    hCamera,
    INT             *piTriggerMode
);
```

Function Description: Get the trigger mode of the camera.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*piTriggerMode: Pointer, returns the index number of the currently selected camera trigger mode.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.103 CameraSnapToBuffer

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSnapToBuffer  
(  
    CameraHandle      hCamera,  
    tSdkFrameHead     *pFrameInfo,  
    BYTE              **pbyBuffer,  
    UINT              uWaitTimeMs  
);
```

Function Description: Snap an image into the buffer.

Parameter Description:

hCamera: The handle of the camera is obtained by the Cameralnit function.

*pFrameInfo: Pointer, returns the frame header information of the image.

**pbyBuffer: Pointer to pointer, used to return the address of the image buffer.

uWaitTimeMs: The timeout period, in milliseconds.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.104 CameraGetResolutionForSnap

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetResolutionForSnap  
(  
    CameraHandle          hCamera,  
    tSdkImageResolution   *pImageResolution  
);
```

Function Description: Get the resolution selection index number in the snapshot mode.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pImageResolution: Pointer, return to the resolution of the capture mode.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.105 CameraSetResolutionForSnap

prototype:

```

MVSDK_API CameraSdkStatus
CameraSetResolutionForSnap
(
    CameraHandle          hCamera,
    tSdkImageResolution    *pImageResolution
);

```

Function Description: Set the resolution of the camera output image in the capture mode.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*pImageResolution: If pImageResolution->iWidth and pImageResolution->iHeight are both 0, it means that it is set to follow the current preview resolution. The resolution of the captured image will be the same as the preview resolution currently set.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```

// Example code for CameraSetResolutionForSnap

```

4.106 CameraSetParameterTarget

prototype:

```

MVSDK_API CameraSdkStatus
CameraSetParameterTarget
(

```

```

    CameraHandle    hCamera,
    int             iTarget
);

```

Function Description: Set the target object of parameter access.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

iTarget: The object of parameter access. PARAM_ON_PC means to read and write the parameter file from the PC, and PARAM_ON_DEVICE to read and write the parameter file from the camera. Refer to the type definition of emSdkParamTarget in CameraDefine.h.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

```

// Example code for CameraSetParameterTarget

```

4.107 CameraGetParameterTarget

prototype:

```

MVSDK_API CameraSdkStatus
CameraGetParameterTarget
(
    CameraHandle    hCamera,
    int             *piTarget
);

```

Function Description: Get the target object of parameter access.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

*piTarget: Pointer, returns the object of parameter access.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

note:

Example:

4.108 CameraSetParameterMask

prototype:

MVSDK_API CameraSdkStatus

CameraSetParameterMask

```
(  
    CameraHandle    hCamera,  
    UINT            uMask  
);
```

Function Description: Set the shielding of parameter access. When parameter loading and saving, it will decide whether to load or save each module parameter according to the shield.

Parameter Description:

hCamera: The handle of the camera is obtained by the CameraInit function.

uMask: shield. Refer to the emSdkPropSheetMask type definition in CameraDefine.h.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.109 CameraRstTimeStamp

prototype:

```
MVSDK_API CameraSdkStatus  
CameraRstTimeStamp  
(  
    CameraHandle    hCamera  
);
```

Function Description: Reset the time stamp of the image acquisition, starting from 0.

Parameter Description: hCamera: The handle of the camera is obtained by the CameraInit function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.110 CameraSaveUserData

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSaveUserData
```

```
(
    CameraHandle    hCamera,
    UINT            uStartAddr,
    BYTE            *pbData,
    int             ilen
);
```

Function Description: Save user-defined data to the non-volatile memory of the camera. The maximum length of the user data area that may be supported by each model of camera is different. The length information can be obtained from the device characteristics description.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

uStartAddr The starting address starts from 0. **Note that the address must be 64-byte aligned.**

pbData Data buffer pointer, return the data read.

ilen The length of the read data, ilen + uStartAddr must be less than the maximum length of the user area.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.111 CameraLoadUserData

prototype:

```
MVSDK_API CameraSdkStatus
CameraLoadUserData
(
    CameraHandle    hCamera,
    UINT            uStartAddr,
    BYTE            *pbData,
```

```

        int          ilen
    );

```

Function Description: Read user-defined data from the non-volatile memory of the camera. The maximum length of the user data area that may be supported by each model of camera is different. The length information can be obtained from the device characteristics description.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

uStartAddr The starting address starts from 0. **Note that the address must be 64-byte aligned.**

pbData Data buffer pointer, return the data read.

ilen The length of the read data, ilen + uStartAddr must be less than the maximum length of the user area.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.112 CameraGetFriendlyName

prototype:

```

MVSDK_API CameraSdkStatus

```

```

CameraGetFriendlyName

```

```

(
    CameraHandle    hCamera,
    char *          pName
);

```

Function Description: Read the nickname of the camera, the nickname of the camera is automatically generated by default, in the form of model name + #0,1,2, For example, Camera MV-U500#0 and Camera MV-U500#1 indicate two 5 million

pixel cameras connected to the same computer. If you need to set a nickname for a camera separately, you can call the CameraSetFriendlyName function. For example, if the first camera is changed to My Camera 1, then the names of the two MV-U500 are My Camera 1 and Camera MV-U500#1.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

pName Returns the first address of the string buffer of the camera's nickname, the buffer size needs to be greater than 32 bytes.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.113 CameraSetFriendlyName

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetFriendlyName
(
    CameraHandle    hCamera,
    char*           pName
);
```

Function Description: Set the nickname of a certain camera. After the setting is successful, the nickname will be solidified in the camera. This function can easily distinguish multiple cameras.

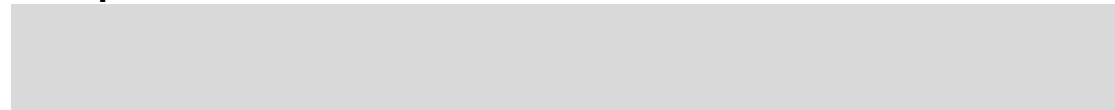
Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

pName Point to the first address of the string buffer of the camera nickname to be set. The nickname must be less than 31 characters. Set the string

"auto" or "automatically generated" to use the default nickname.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.114 CameraSdkGetVersionString

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSdkGetVersionString  
(  
    char*      pVersionString  
);
```

Function Description: Read user-defined data from the non-volatile memory of the camera. The maximum length of the user data area that may be supported by each model of camera is different. The length information can be obtained from the device characteristics description.

Parameter Description: pVersionString Returns the SDK version string. The length of the buffer must be greater than 32 words.

return value: Returns the SDK version description string.

Example:



4.115 CameraGetEnumInfo

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetEnumInfo
```

```
(
    CameraHandle      hCamera,
    tSdkCameraDevInfo* pCameraInfo
);
```

Function Description: Obtain enumeration information of the specified device.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.
 pCameraInfo Pointer, returns the enumeration information of the device.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.116 CameraSetIOState

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetIOState
(
    CameraHandle      hCamera,
    INT               iOutputIOIndex,
    UINT              uState
);
```

Function Description: Set the level state of the specified IO, IO is output type IO, the number of programmable output IO reserved by the camera is determined by iOutputIoCounts in tSdkCameraCapbility.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.
 iOutputIOIndex The index number of IO, starting from 0.

uState The state to be set, 1 is high, 0 is low.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.117 CameraGetIOState

prototype:

MVSDK_API CameraSdkStatus

CameraGetIOState

```
(  
    CameraHandle    hCamera,  
    INT             iInputIOIndex,  
    UINT*           puState  
);
```

Function Description: Set the level status of the specified IO, IO is input type IO, the number of programmable output IO reserved by the camera is determined by iInputIOCounts in tSdkCameraCapbility.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

 iInputIOIndex The index number of IO, starting from 0.

 puState Pointer, return IO status, 1 is high, 0 is low.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.118 CameraSetIspOutFormat

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetIspOutFormat  
(  
    CameraHandle    hCamera,  
    UINT            uFormat  
);
```

Function Description: Set the output format of the image processing of the CameraIspProcess function, stand by CAMERA_MEDIA_TYPE_MONO8 and CAMERA_MEDIA_TYPE_RGB8、CAMERA_MEDIA_TYPE_RGBA8、CAMERA_MEDIA_TYPE_BGR8、CAMERA_MEDIA_TYPE_BGRA8 (defined in CameraDefine.h) five types, Corresponding to 8-bit grayscale images and 24-bit RGB, 32-bit RGBA, 24-bit BGR, and 32-bit BGRA formats respectively. The output format can be selected according to the needs of the vision library during secondary development. For example, BGR format is required under OPENCV, while RGB format is required for QT development.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

uFormat To set the format, it can be
One of CAMERA_MEDIA_TYPE_MONO8 and
CAMERA_MEDIA_TYPE_RGB8、
CAMERA_MEDIA_TYPE_RGBA8、
CAMERA_MEDIA_TYPE_BGR8、
CAMERA_MEDIA_TYPE_BGRA8.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.119 CameraGetIspOutFormat

prototype:

```
MVSDK_API CameraSdkStatus
```

```
CameraGetIspOutFormat
```

```
(  
    CameraHandle    hCamera,  
    UINT*           puFormat  
);
```

Function Description: Obtain the output format of CameraGetImageBuffer function image processing, the format can be set by CameraSetIspOutFormat.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

puFormat Returns the currently set format, its value is One of CAMERA_MEDIA_TYPE_MONO8 and CAMERA_MEDIA_TYPE_RGB8、CAMERA_MEDIA_TYPE_RGBA8、CAMERA_MEDIA_TYPE_BGR8、CAMERA_MEDIA_TYPE_BGRA8.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.120 CameraGetErrorString

prototype:

```
MVSDK_API char*
```

```
CameraGetErrorString
```

```
(
```

```
CameraSdkStatus    iStatusCode  
);
```

Function Description: Get the description string corresponding to the error code.

Parameter Description: iStatusCode error code. (Defined in CameraStatus.h)

return value: When successful, enter the first address of the string corresponding to the error code.

Example:

4.121 CameraReConnect

prototype:

```
MVSDK_API CameraSdkStatus  
CameraReConnect  
(  
    CameraHandle    hCamera,  
);
```

Function Description: Reconnect the device, which is used to reconnect USB and GIGE devices after accidental disconnection.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.122 CameraConnectTest

prototype:

```
MVSDK_API CameraSdkStatus
```

```
CameraConnectTest
```

```
(  
    CameraHandle    hCamera,  
);
```

Function Description: Test the connection status of the camera to detect whether the camera is disconnected.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0), indicating that the connection is not dropped; otherwise, it returns an error code with a non-zero value. Please refer to the definition of error code in CameraStatus.h.

Example:

4.123 CameraSetTriggerDelayTime

prototype:

```
MVSDK_API CameraSdkStatus
```

```
CameraSetTriggerDelayTime
```

```
(  
    CameraHandle    hCamera,  
    UINT            uDelayTimeUs  
);
```

Function Description: Set the trigger delay time in hardware trigger mode, in microseconds. When the hard trigger signal comes, after a specified delay time, start to collect images. Only certain models of cameras support this function. Please check the product manual for details.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

UDelayTimeUs Hard trigger delay. The unit is microseconds.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.124 CameraGetTriggerDelayTime

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetTriggerDelayTime
(
    CameraHandle    hCamera,
    UINT*           puDelayTimeUs
);
```

Function Description: Get the hard trigger delay time currently set.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

 puDelayTimeUs Pointer, return delay time, Unit microsecond.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.125 CameraSetStrobeMode

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetStrobeMode
(
    CameraHandle    hCamera,
```

INT	iMode
-----	-------

);

Function Description: Set the STROBE signal on the IO pin terminal. This signal can be used for flash control or external mechanical shutter control.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function..

iMode When the STROBE_SYNC_WITH_TRIG_AUTO is synchronized with the trigger signal, after the trigger, when the camera performs exposure, the STROBE signal is automatically generated. At this time, the effective polarity can be set (CameraSetStrobePolarity).

When it is STROBE_SYNC_WITH_TRIG_MANUAL, and The trigger signal is synchronized. After the trigger, STROBE delays the specified time (CameraSetStrobeDelayTime), and then continues the specified time pulse (CameraSetStrobePulseWidth), and the effective polarity can be set (CameraSetStrobePolarity).

When it is STROBE_ALWAYS_HIGH, the STROBE signal is always high, ignoring other settings.

When it is STROBE_ALWAYS_LOW, the STROBE signal is always low, ignoring other settings.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.126 CameraGetStrobeMode

prototype:

```
MVSDK_API CameraSdkStatus
```

```
CameraGetStrobeMode
```

```
(  
    CameraHandle    hCamera,  
    INT*            piMode  
);
```

Function Description: r the mode set by the current STROBE signal.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

piMode Pointer, return
 STROBE_SYNC_WITH_TRIG_AUTO,
 STROBE_SYNC_WITH_TRIG_MANUAL,
 STROBE_ALWAYS_HIGH OR
 STROBE_ALWAYS_LOW.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.127 CameraSetStrobeDelayTime

prototype:

```
MVSDK_API CameraSdkStatus
```

```
CameraSetStrobeDelayTime
```

```
(  
    CameraHandle    hCamera,  
    UINT            uDelayTimeUs  
);
```

Function Description: When the STROBE signal is in STROBE_SYNC_WITH_TRIG, set its relative trigger signal delay time through this function.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

uDelayTimeUs Relative to the delay time of the trigger signal, the unit is us. Can be 0, But it cannot be negative.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.128 CameraGetStrobeDelayTime

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetStrobeDelayTime
(
    CameraHandle    hCamera,
    UINT*           upDelayTimeUs
);
```

Function Description: When the STROBE signal is in STROBE_SYNC_WITH_TRIG, the relative trigger signal delay time is obtained through this function.

Parameter Description: hCamera	The handle of the camera is obtained by the CameraInit function.
---------------------------------------	--

upDelayTimeUs Pointer, return delay time, unit US.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.129 CameraSetStrobePulseWidth

prototype:

```
MVSDK_API CameraSdkStatus  
CameraSetStrobePulseWidth  
(  
    CameraHandle    hCamera,  
    UINT            uTimeUs  
);
```

Function Description: When the STROBE signal is in STROBE_SYNC_WITH_TRIG, set its pulse width through this function.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

uTimeUs The width of the pulse, the unit is time us.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.130 CameraGetStrobePulseWidth

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetStrobePulseWidth  
(  
    CameraHandle    hCamera,  
    UINT*           upTimeUs  
);
```

Function Description: When the STROBE signal is in STROBE_SYNC_WITH_TRIG, get its pulse width through this function.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

 upTimeUs The pointer returns to the pulse width. The unit is time us.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.131 CameraSetStrobePolarity

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetStrobePolarity
(
    CameraHandle    hCamera,
    INT              uPolarity
);
```

Function Description: When the STROBE signal is in STROBE_SYNC_WITH_TRIG, set the polarity of its active level through this function. The default is active high. When the trigger signal comes, the STROBE signal is pulled high.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

 iPolarity The polarity of the STROBE signal is 0 for low level and 1 for high level. The default is active high.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.132 CameraGetStrobePolarity

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetStrobePolarity
(
    CameraHandle    hCamera,
    INT*            upPolarity
);
```

Function Description: Get the effective polarity of the current STROBE signal of the camera. The default is active high.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.
 ipPolarity Pointer, returns the current valid polarity of the STROBE signal.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.133 CameraSetExtTrigSignalType

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetExtTrigSignalType
(
    CameraHandle    hCamera,
    INT              iType
);
```

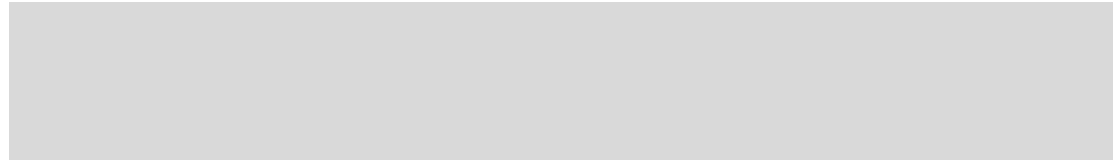
Function Description: Set the type of trigger signal external to the camera. Upper edge, lower edge, or high or low level mode.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

 iType Type of external trigger signal, return value: Refer to the definition of emExtTrigSignal in CameraDefine.h.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.134 CameraGetExtTrigSignalType

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetExtTrigSignalType  
(  
    CameraHandle    hCamera,  
    INT*            ipType  
);
```

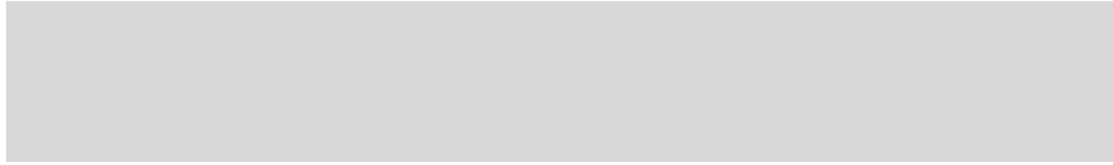
Function Description: Get the type of the current external trigger signal of the camera.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

 ipType Pointer, returns the type of external trigger signal, return value: Refer to the definition of emExtTrigSignal in CameraDefine.h.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.135 CameraSetExtTrigDelayTime

prototype:

```
MVSDK_API CameraSdkStatus
CameraSetExtTrigDelayTime
(
    CameraHandle    hCamera,
    UINT            uDelayTimeUs
);
```

Function Description: Set the delay time of the external trigger signal, the default is 0, and the unit is microseconds.

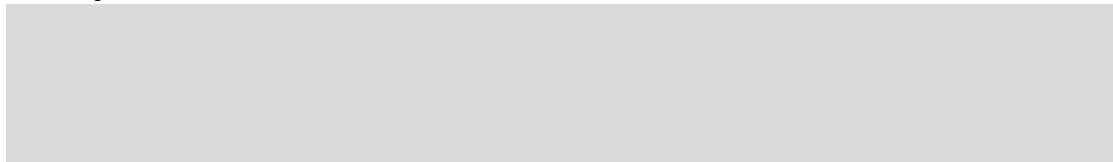
When the set value uDelayTimeUs is not 0, after the camera receives the external trigger signal, it will delay uDelayTimeUs for one microsecond before image capture.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

uDelayTimeUs Delay time, in microseconds, the default is 0.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.136 CameraGetExtTrigDelayTime

prototype:

```
MVSDK_API CameraSdkStatus
CameraGetExtTrigDelayTime
```

```
(
    CameraHandle    hCamera,
    UINT*           upDelayTimeUs
);
```

Function Description: Obtain the set delay time of the external trigger signal, the default is 0, the unit is microseconds.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

upDelayTimeUs Pointer, return delay time.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:

4.137 CameraSetExtTrigJitterTime

prototype:

MVSDK_API CameraSdkStatus

CameraSetExtTrigJitterTime

```
(
    CameraHandle    hCamera,
    UINT            uTimeUs
);
```

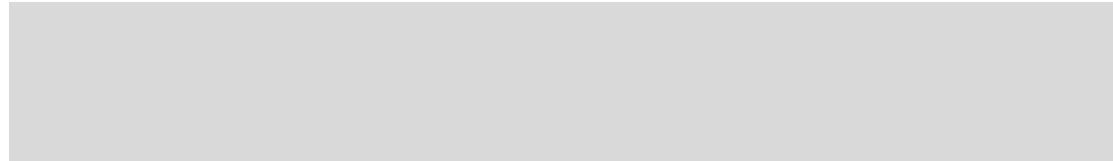
Function Description: et the debounce time of the external trigger signal of the camera. The debounce time will only take effect when the external trigger signal mode is selected as high level or low level trigger. The default is 0, the unit is microseconds, and the maximum is 150 milliseconds.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

uTimeUs The unit is microseconds.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.138 CameraGetExtTrigJitterTime

prototype:

```
MVSDK_API CameraSdkStatus  
CameraGetExtTrigJitterTime  
(  
    CameraHandle    hCamera,  
    UINT*           upTimeUs  
);
```

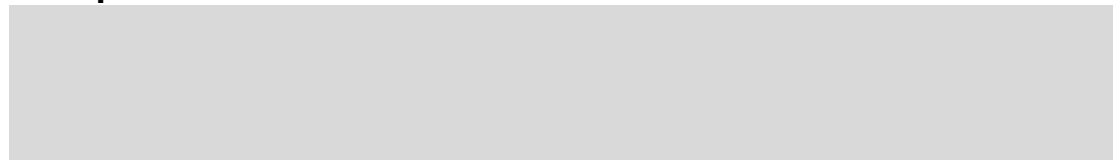
Function Description: Get the set debounce time of the camera external trigger, the default is 0. The unit is microseconds.

Parameter Description: hCamera The handle of the camera is obtained by the CameraInit function.

 upTimeUs The pointer returns to the set debounce time.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

Example:



4.139 CameraGetExtTrigCapability

prototype:

```

MVSDK_API CameraSdkStatus
CameraGetExtTrigCapability
(
    CameraHandle    hCamera,
    UINT*           puCapabilityMask
);

```

Function Description: Get the property mask triggered outside the camera.

Parameter Description: hCamera he handle of the camera is obtained by the CameraInit function.

 puCapabilityMask Pointer, return to the camera external trigger feature mask, the mask refers to the macro definition at the beginning of EXT_TRIG_MASK_ in CameraDefine.h.

return value: When successful, it returns CAMERA_STATUS_SUCCESS (0); otherwise, it returns an error code other than 0. Please refer to the definition of error code in CameraStatus.h.

4.140 CameraAlignMalloc

prototype:

```

MVSDK_API BYTE*
CameraAlignMalloc
(
    int            size,
    int            align
);

```

Function Description: Request an address aligned memory block.

Used to cooperate with hardware acceleration algorithms and need to use aligned memory blocks.

Parameter Description: size memory size
 align Address alignment, if 16-byte alignment is required, set to 16

return value: When it succeeds, it returns the first address of the applied memory. On failure, NULL is returned.

4.141 CameraAlignFree

prototype:

```
MVSDK_API VOID  
CameraAlignFree  
(  
    BYTE*      membuffer  
);
```

Function Description: Release the memory block whose address is aligned by CameraAlignMalloc.

Parameter Description: membuffer The memory address returned by CameraAlignMalloc.

return value: no.

5 SDK interface functions are explained by function classification

In order to facilitate users to quickly find the development method of a certain function, we have summarized some commonly used methods.

5.1 Camera initialization and de-initialization

For the camera workflow, we first enumerate, initialize, and then deinitialize the camera before exiting the program.

- For camera enumeration, we provide two functions, `CameraEnumerateDevice` and `CameraEnumerateDeviceEx`. `CameraEnumerateDevice` will return the number of cameras and detailed description information. The description information is returned as a structure array, such as the name, model, serial number, interface type, etc. of the camera will be listed in detail; `CameraEnumerateDeviceEx` only returns the number of cameras.
- For the initialization of the camera, we provide three functions: `CameraInit`, `CameraInitEx`, and `CameraInitEx2`. `CameraInit` needs to be used in conjunction with `CameraEnumerateDevice`, while `CameraInitEx` and `CameraInitEx2` need to be used in conjunction with `CameraEnumerateDeviceEx`. `CameraInit` needs to pass in the

camera structure information obtained by CameraEnumerateDevice;CameraInitEx only needs to pass in the serial number ID of the camera, for example, to initialize the first camera, pass in 0, the second camera pass in 1, and so on;CameraInitEx2 is the name of the incoming camera. For example, you can change the camera name to "Camera1" in advance through the tool we provide, and when CameraInitEx2 is called, just pass in "Camera1".For the case of multiple cameras working at the same time, this method can effectively establish a one-to-one correspondence, but it should be noted that the name of each camera must be changed to a different one to ensure uniqueness.

- Deinitialization function, CameraUnInit.无 No matter which initialization method is adopted, just call CameraUnInit to de-initialize.

The following is a typical program flow using

CameraEnumerateDevice and CameraInit:

```
tSdkCameraDevInfo sCameraList[10];
```

```
int iCameraNums;
```

```
int status;
```

```

iCameraNums = 10;

//Enumerate cameras, return description information of up to 10 cameras

    if (CameraEnumerateDevice(sCameraList,&iCameraNums) !=

CAMERA_STATUS_SUCCESS || iCameraNums == 0)

    {

        return FALSE;

    }

//If there is only one camera, iCameraNums will be internally modified to 1 by

CameraEnumerateDevice.

    if ((status = CameraInit(&sCameraList[0],-1,-1,&m_hCamera)) !=

CAMERA_STATUS_SUCCESS)

    {

        return FALSE;

    }

//Call before program exit

CameraUnInit(m_hCamera);

```

**The following is a typical program flow using
CameraEnumerateDeviceEx and CameraInitEx:**

```

int iCameraNums;

iCameraNums = CameraEnumerateDeviceEx();

```

```
if (iCameraNums == 0) return FALSE;
```

```
    if ((status = CameraInitEx(0,-1,-1,&m_hCamera)) != CAMERA_STATUS_SUCCESS)

    {

        return FALSE;

    }
```

```
//Call before program exit
```

```
CameraUnInit(m_hCamera);
```

The following is a typical program flow using

CameraEnumerateDeviceEx and CameraInitEx2:

```
int iCameraNums;
```

```
iCameraNums = CameraEnumerateDeviceEx();
```

```
if (iCameraNums == 0) return FALSE;
```

//In this example, the name "Camera1" is used for initialization, so the camera must be renamed "Camera1" with the tool first

```
    if ((status = CameraInitEx2("Camera1",&m_hCamera)) != CAMERA_STATUS_SUCCESS)

    {

        return FALSE;
```

```
}  
  
//Call before program exit  
CameraUnInit(m_hCamera);
```

5.2 Save and load camera parameters

The camera parameters are saved in the computer and exist as a binary file. The interface is divided into four groups: A, B, C, and D. The loading methods are also divided into three methods: by model, by nickname and by serial number, as shown in the following figure:



- CameraSetParameterMode, Used to set the way of parameter saving.
For details, refer to Chapter 4 Function Explanation.
- CameraSaveParameter, Save the parameter to the specified parameter group. For details, refer to Chapter 4 Function Explanation.
- CameraLoadParameter, Load the parameters of the specified group.
For details, refer to Chapter 4 Function Explanation.

- CameraSaveParameterToFile, save the current camera parameters to the specified file. For details, refer to Chapter 4 Function Explanation.
- CameraReadParameterFromFile, Load camera parameters from the specified file. For details, refer to Chapter 4 Function Explanation.

5.3 Camera fetching (active fetching or callback function method)

After the initialization is complete, the camera can start to take pictures. According to the way of taking pictures, our SDK is divided into active and passive (callback functions).

- **Active image reading method:**

After completing the camera initialization, you can use the CameraGetImageBuffer or CameraGetImageBufferEx3 function to actively read the image, The difference between the two is that the CameraGetImageBuffer function gets the original RAW data buffer, and the CameraImageProcess function needs to be used to convert the RAW data into the specified data format.

After using the obtained buffer pointer, call CameraReleaseImageBuffer to release the right to use the buffer, What needs to be explained here is that CameraReleaseImageBuffer only releases the right to use the data

buffer obtained by CameraGetImageBuffer, and does not repeatedly apply and release memory, so there is no need to worry about efficiency;The CameraGetImageBufferEx3 function directly obtains the image data in the specified format. The specific format is determined by the input parameters. It can be 8-bit or 16-bit grayscale, or 24-bit or 32-bit color format data. After the CameraGetImageBufferEx3 function is called, No need to call CameraReleaseImageBuffer.**The common point of the two image reading functions is that the timeout period can be set,**For example, if a timeout of 1000 milliseconds is set, within 1000 milliseconds, if a valid image is not obtained, the function will block and the thread will be suspended until it exceeds 1000 milliseconds or a valid image is read.Therefore, in the design of the software structure, the user can create a special thread to capture images, and then continue to perform image capture, only need to set a reasonable timeout,Or call the function once when you need to collect an image to get an image.

The following is a code example using CameraGetImageBuffer:

```
tSdkFrameHead  sFrameInfo;

BYTE*          pbyBuffer;

CameraSdkStatus status;

/* pbyBuffer automatically applies for the memory inside the SDK and stores the
original RAW data*/
```

```

        if(CameraGetImageBuffer(m_hCamera,&sFrameInfo,&pbyBuffer,1000) ==
CAMERA_STATUS_SUCCESS)

    {

/* m_pFrameBuffer needs to apply for the memory pbyBuffer first. The converted image
data is stored in m_pFrameBuffer. By default, BRG24bit image data will be obtained.*/

        CameraImageProcess(m_hCamera, pbyBuffer, m_pFrameBuffer,&sFrameInfo);

        CameraReleaseImageBuffer(m_hCamera,pbyBuffer);

    }

```

The following is a code example using

CameraGetImageBufferEx3:

```

    int width,height;//Image width and height

    unsigned int uTimeStamp;//Image timestamp

    BYTE*pImageData;

    pImageData = CameraAlignMalloc(1280*1024*3,16);//Assuming a 1.3 megapixel color
camera

    if(CameraGetImageBufferEx3(m_hCamera,

pImageData ,1,&width,&height,&uTimeStamp,1000) == CAMERA_STATUS_SUCCESS)

    {

        /*The image was successfully collected and visually processed... */

        /* pImageData stores the color image data in RGB24 format, width and height will

```

return the width and height of the image, uTimeStamp will return the timestamp of the image, 1000 is the timeout period*

```
}
```

CameraAlignFree(pImageData); //In practical applications, memory is allocated for pImageData once during program initialization, and the memory is released when the program is finally exited. The buffer can be used repeatedly in the middle.

- Passive image reading mode (callback function)

If you need to use the callback function for image reading processing, you need to set the callback function for image reading after the camera is initialized

After CameraInit, call the following function to set the callback.

```
m_pFrameBuffer = CameraAlignMalloc(1280*1024*3,16); //Assuming a 1.3 megapixel
```

color camera

```
CameraSetCallbackFunction(m_hCamera, GrabImageCallback, (PVOID)0, NULL);
```

Where GrabImageCallback is defined as follows:

```
void _stdcall GrabImageCallback(CameraHandle hCamera, BYTE *pFrameBuffer,
```

```
tSdkFrameHead* pFrameHead, PVOID pContext)
```

```

{

    CameraSdkStatus status;

    //Convert RAW data into image data in a specified format. It is converted to image
data in BGR24 format by default.

    status = CameraImageProcess(hCamera, pFrameBuffer,
m_pFrameBuffer,pFrameHead);

    //After the conversion is successful, use the data in m_pFrameBuffer for
subsequent processing.

}

```

5.4 Preview image with indicator

In order to simplify the user's secondary development process, our SDK has encapsulated some interface functions to facilitate image preview, provided that VS-related tools must be used for interface development, such as C#, VB, Delphi, MFC, etc.

After the camera is initialized, call the following code to complete the initialization of the display part.

CameraDisplayInit passes in the hWnd handle of the window. Different development tools have different ways to obtain the hWnd handle of the display control.

CameraSetDisplaySize sets the size of the display area in pixels. Generally set to the actual size of the display control.

CameraSetDisplayMode sets the display mode, there are two display modes: tile and zoom. The default is zoom display. The image will be scaled to a suitable size and displayed on the display controls.

CameraSetDisplayOffset sets the offset value of the display coordinates. When the size of the display control is smaller than the size of the image and CameraSetDisplayMode is set to tile display mode, you can set the offset value to display a certain area of the image through this function.

For the detailed usage of the above several functions, we provide special routines, please refer to ChinaVision\Demo\VC++\CameraDisplay routines.

5.5 Adjust camera image brightness (set exposure time)

When the camera leaves the factory, in order to facilitate the demonstration effect, the default is automatic exposure, that is, the camera will automatically adjust the camera's exposure time and analog gain value according to the brightness of the ambient light to obtain the best image brightness. However, this automatic adjustment mode is not suitable for industrial applications, so some functions need to be called to manually set the exposure and gain to stabilize the image brightness to suit the needs of software algorithms. For example, in some cases,

users may want to overexpose most areas of the image to reduce interference with the image.

- `CameraSetAeState`, set the camera exposure mode, divided into manual and automatic modes.
- `CameraSetExposureTime`, set the exposure time of the camera in microseconds. Note that the longer the exposure time, the lower the camera's frame rate. For example, if the exposure time is set to 500 milliseconds, then the camera can image up to 2 times per second, and the image will look stuck. Therefore, in order to reduce the time of taking pictures, the exposure time should be as low as possible. Of course, if the exposure time is low, the external light source is needed to increase the illumination, otherwise the image will be very dark.
- `CameraSetAnalogGain`, set the analog gain value of the camera, the size of this value will only affect the image brightness, but will not affect the image frame rate. Because it is only a circuit amplification factor, but the increase of this value will increase the image background noise. For applications that pursue image quality, the analog gain should be set to the minimum.

Note that the precondition for manual setting of `CameraSetExposureTime` and `CameraSetAnalogGain` is to set the

**camera to manual exposure mode through CameraSetAeState,
otherwise the setting is invalid!**

5.6 Switch between different resolutions and custom resolutions (ROI function)

The camera's preset resolution is a pre-set ROI in the SDK for ease of use, as shown below:

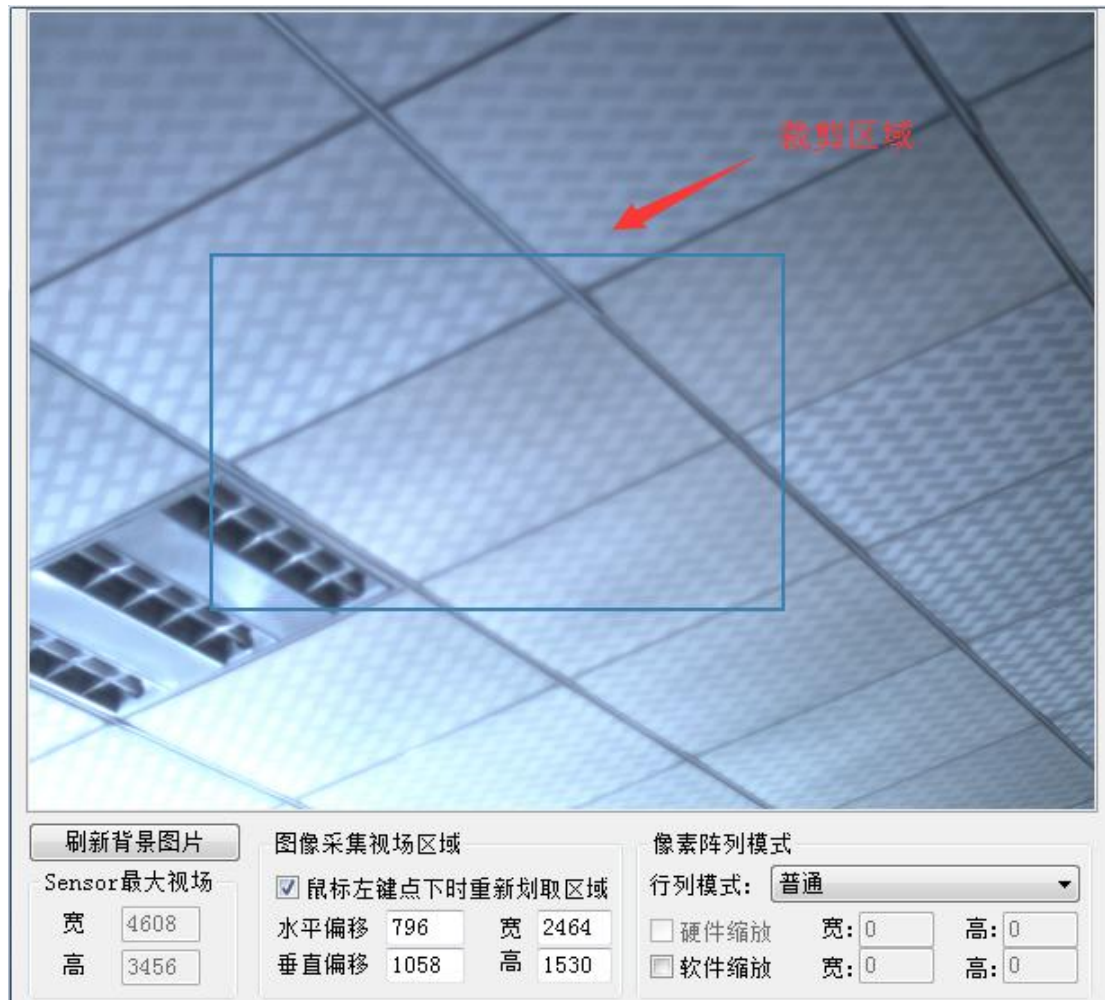


To switch the preset resolution, you only need to set the index of the corresponding resolution, sample code:

```
tSdkImageResolution Resolution = { 0 };  
Resolution.iIndex = Default resolution index;  
CameraSetImageResolution(hCamera, &Resolution);
```

The preset resolution index is shown in the figure, 0: 4608X3456 MAX 1: 4384X3288 ROI and so on

Custom resolution function: Specify a cropping rectangle in the maximum resolution of the camera, so that the camera only outputs the specified rectangular area, which can often increase the frame rate and shorten the processing time of the algorithm, as shown in the following figure:



Note: There are certain restrictions on the offset and width of the cropped rectangle. The horizontal offset and width need to be multiples of 16, and the vertical offset and height can generally be multiples of 4. (Some monochrome cameras can support multiples of 1, while color cameras are at least multiples of 2).

Sample code:

```
tSdkImageResolution sRoiResolution = { 0 };
```

```
// Set to 0xff for custom resolution, set to 0 to N for select preset resolution
```

```
sRoiResolution.iIndex = 0xff;
```

```
// width
```

```
sRoiResolution.iWidth = width;
```

```
sRoiResolution.iWidthFOV = width;
```

```
// height
```

```
sRoiResolution.iHeight = height;
```

```
sRoiResolution.iHeightFOV = height;
```

```
// Field of view offset
sRoiResolution.iHOffsetFOV = offsetx;
sRoiResolution.iVOffsetFOV = offsety;

// ISP software zoom width and height, all 0 means not zoom
sRoiResolution.iWidthZoomSw = 0;
sRoiResolution.iHeightZoomSw = 0;

// BIN SKIP mode setting (requires camera hardware support)
sRoiResolution.uBinAverageMode = 0;
sRoiResolution.uBinSumMode = 0;
sRoiResolution.uResampleMask = 0;
sRoiResolution.uSkipMode = 0;

CameraSetImageResolution(hCamera, &sRoiResolution);
```

Special modes such as BIN (cumulative, average) and SKIP (snap point) require hardware support. Some cameras may not support it. The setting methods of each mode are as follows:

BIN accumulation 2x2:

```
sRoiResolution.uBinSumMode = 1;
sRoiResolution.iWidth=sRoiResolution.iWidthFOV/2;
sRoiResolution.iHeight=sRoiResolution.iHeightFOV/2;
```

BIN cumulative 4x4:

```
sRoiResolution.uBinSumMode = 4;
sRoiResolution.iWidth=sRoiResolution.iWidthFOV/4;
sRoiResolution.iHeight=sRoiResolution.iHeightFOV/4;
```

BIN average 2x2:

```
sRoiResolution.uBinAverageMode = 1;
sRoiResolution.iWidth=sRoiResolution.iWidthFOV/2;
sRoiResolution.iHeight=sRoiResolution.iHeightFOV/2;
```

BIN average 4x4:

```
sRoiResolution.uBinAverageMode = 4;
sRoiResolution.iWidth=sRoiResolution.iWidthFOV/4;
sRoiResolution.iHeight=sRoiResolution.iHeightFOV/4;
```

SKIP points 2x2:

```
sRoiResolution.uSkipMode = 1;
sRoiResolution.iWidth=sRoiResolution.iWidthFOV/2;
```

`sRoiResolution.iHeight=sRoiResolution.iHeightFOV/2;`

SKIP points 4x4:

`sRoiResolution.uSkipMode = 4;`

`sRoiResolution.iWidth=sRoiResolution.iWidthFOV/4;`

`sRoiResolution.iHeight=sRoiResolution.iHeightFOV/4;`

5.7 Set the camera's contrast, gamma, saturation, sharpness and other ISP parameters

- **CameraSetGamma**, set the gamma value of the camera. The smaller the gamma value, the pixels with smaller brightness values will be increased, making the overall brightness positive;The larger the gamma value, the gray value of high-brightness pixels will be reduced, making the overall brightness smaller.The default gamma value is 1.0.
- **CameraSetContrast**, set the contrast value of the camera. The larger the contrast, the darker the black area of the image and the whiter the white area, making the image look very clear in monochrome, which can effectively capture the outline in some visual processing;On the contrary, if the contrast is smaller, the monochrome will be unclear and the look will be hazy. The default contrast is 100, the minimum can be 1, and the maximum can be 200.
- **CameraSetSaturation**, set the saturation.The greater the saturation, the thicker the color; on the contrary, the lighter it is. If the saturation is set to 0, the image will have no color at all, which is equivalent to a

monochrome camera. The default value is 100. The adjustment range is 0 to 200.

- CameraSetGain, set the gain value of the three color channels of R, G and B.
- CameraSetSharpness, set the sharpening level of the image. The higher the sharpness, the better the image clarity, but the greater the noise; on the contrary, the lower the sharpness, the stronger the image is hazy, but the noise is very low and it appears smooth. The default value is 0, which means there is no sharpening enhancement effect.

5.8 Convert color camera to monochrome camera

The CameraSetMonochrome function can convert a color image to monochrome. The effect is equivalent to setting the saturation of the image to 0. Therefore, you can also use the CameraSetSaturation function to set the saturation to 0 to obtain a monochrome grayscale image.

Note: Whether it is a monochrome or color camera, the factory default is to output image data in BGR24 format. For the BGR24 of a monochrome camera, B=G=R. If you only need to process 8-bit grayscale images, after calling the following code after initialization,

the image output by the CameraImageProcesss function will be 8-bit grayscale.

//nitialize the camera

CameraInit....

//Set the output format of the CameraImageProcesss function to

8-bit grayscale

CameraSetIsOutFormat(hCamera, CAMERA_MEDIA_TYPE_MONO8);

//Subsequent picture taking part

```
        if(CameraGetImageBuffer(m_hCamera,&sFrameInfo,&pbyBuffer,1000) ==  
CAMERA_STATUS_SUCCESS)  
        {  
            CameraImageProcess(m_hCamera, pbyBuffer,  
m_pFrameBuffer,&sFrameInfo);// m_pFrameBuffer, in the output buffer address  
pointed to, the 8-bit grayscale image format is saved.  
        }
```

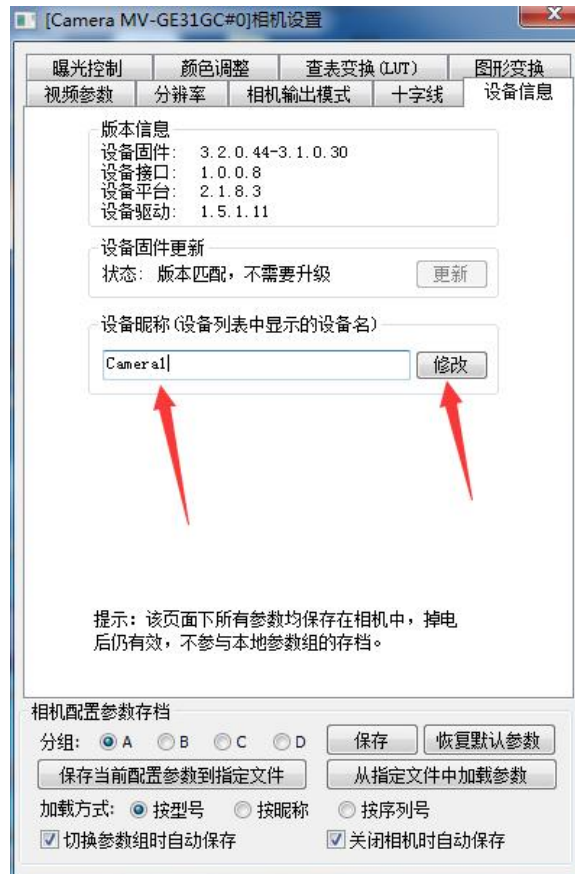
Or directly use the CameraGetImageBufferEx function to get an 8-bit grayscale image.

5.9 Multiple cameras are used at the same time, how to establish camera correspondence

When multiple cameras are used in the vision system at the same time, it is necessary to confirm the one-to-one correspondence between the camera and the inspection station on the software. However, the scanning sequence of the camera is often not fixed, and is related to the power-on sequence of the camera, so some special recognition is required on the software side. We recommend the following two methods for binding:

- Follow the serial number of the camera. Before the camera leaves the factory, it has a unique serial number code. This serial number can be read during the enumeration phase of the camera. Through the `CameraEnumerateDevice` function can scan the number of cameras and detailed description information structure array. The type is `tSdkCameraDevInfo`, and the `acSn` member in `tSdkCameraDevInfo` stores a serial number string of up to 32 bytes. The serial number can be judged by the software to select the corresponding `tSdkCameraDevInfo` array for subsequent `CameraInit` calls.
- Follow the custom name of the camera. By default, the name of the camera changes with the power-on sequence of the camera. For example, the first camera powered on ends with the model name plus #0. For example, MV-GE500M#0 and MV-GE500M#1

respectively represent the names of the two cameras connected to the system. If the power-on sequence of the cameras is changed, then the original No. 0 camera may become the No. 1 camera. Therefore, we provide a modification tool to manually modify the custom name of each camera. The custom name is the same as the serial number and can be enumerated during the enumeration phase, Also through the CameraEnumerateDevice function can scan the number of cameras and detailed description information structure array, The type is tSdkCameraDevInfo, and the acFriendlyName member in tSdkCameraDevInfo saves the custom name of the device up to 32 bytes. Then select the tSdkCameraDevInfo array according to the name to call CameraInit. The way to modify the custom name of the camera is as follows. After entering the device nickname, click the Modify button, and then turn off the power of the camera and restart it. The custom name will take effect and will be permanently fixed inside the camera.



Modify the custom name of the camera

After the camera name is modified, it can also be quickly initialized through the CameraInitEx2 function. Suppose the two cameras are renamed "Camera1" and "Camera2" and the code is as follows:

```
//Scanning camera, if there are 2, iCameraCounts = 2
int iCameraCounts = CameraEnumerateDeviceEx();

//Initialize the camera with the custom name "Camera1". If the camera
with the corresponding name is not found, the initialization will fail.
CameraInitEx2 ("Camera1",&hCamera1);

//Initialize the camera with the custom name "Camera2". If the camera
with the corresponding name is not found, the initialization will fail.
```

CameraInitEx2 ("Camera2",&hCamera2);

5.10 Image flip mirror and rotate function

- Enable image horizontal flip, CameraSetMirror(hCamera,0,1);
- Prohibit the horizontal flip of the image (default),
CameraSetMirror(hCamera,0,0);
- Enable vertical flip of image, CameraSetMirror(hCamera,1,1);
- Prohibit vertical flip of image (default),
CameraSetMirror(hCamera,1,0);
- Image rotation 0 degrees (default), CameraSetRotate(hCamera,0);
- Rotate the image by 90 degrees (default),
CameraSetRotate(hCamera,1);
- Rotate the image by 180 degrees (default),
CameraSetRotate(hCamera,2);
- The image is rotated by 270 degrees (default),
CameraSetRotate(hCamera,3);

Note that after the image is rotated, the width and height of the image will be changed. For example, it was originally 800X600. After rotating by 90 degrees, it will be 600X800. Pay attention when processing, otherwise it will cause display errors and the image will look chaotic.

5.11 Superimpose text on images

In order to facilitate customers to superimpose text on the image for information prompt, we encapsulate a function `CameraDrawText` for quick calling. Regarding the usage of this function, we specially provide a routine, located under `\Demo\VC++\DrawText`. It should be noted that the `CameraDrawText` function superimposes text directly on the image content. In order not to affect the visual processing, it should be called in the final process of the visual processing (call before the image display function).

5.12 Save picture and video function

- Image saving function. Using `CameraSaveImage`, you can save the image as one of RAW, PNG, JPG, BMP24bit, and BMP8bit. A typical process is:
 - 1, Use `CameraGetImageBuffer` function to get RAW image data;
 - 2, Use `CameraImageProcess` function to convert RAW into BGR24 or 8-bit grayscale format. If you want to save RAW format data, skip this step.
 - 3, `CameraReleaseImageBuffer`, release the right to use the RAW data buffer of `CameraGetImageBuffer`.

4, The CameraSaveImage function saves the image data obtained by the CameraImageProcess function as one of JPG, BMP or PNG. If it is RAW data saving, use the RAW data buffer obtained by CameraGetImageBuffer.

The example of image saving is located in

\Demo\VC++\ImageFormat&Saving.

- Video function. CameraInitRecord function initializes a recording. CameraPushFrame encodes a frame of image. CameraStopRecord stops recording and ends the file writing operation. The example of recording is located in \Demo\VC++\Record.

5.13 Set the image bit depth output by the camera

By default, industrial cameras generally transmit 8bit raw format data. In some wide dynamic applications, users can change the bit depth of raw data output by the camera through the CameraSetMediaType function. Currently we support 3 bit depths of 8, 12, and 16 bits. CameraSetMediaType is to select the type of bit depth supported by the camera.

For example, CameraSetMediaType(hCamera,0); select the first type, which is the 8bit format,

CameraSetMediaType(hCamera,1); select the second type, which is 12bit,

CameraSetMediaType(hCamera,2); Choose the third type, which is 16bit.

Note: Not all models support 12 and 16bit output modes. If

CameraSetMediaType returns a non-zero value, it means that the setting has failed.

5.14 Set the pixel format of the image (8-bit grayscale, 24, 32, 48-bit color)

Different from 5.13, the pixel format mentioned here is the pixel format of the image finally obtained through the CameraImageProcess function, not the original RAW pixel format output by the camera.

Most models of industrial cameras can output 12bit or even 16bit original RAW images. For color cameras, if a color component uses 8bit, the low-level details of the image will be lost. Therefore, we provide the RGB48bit color image format to satisfy users for high-dynamic and high-precision visual analysis. The three color channels of red, green, and blue are represented by 16 bits for each channel, and a pixel requires 6 bytes. When the camera supports 12bit or 16bit original RAW format, 48bit color image format can be used. When RAW data in 12-bit format is converted into a 48-bit color image, the lower 4 bits of each channel will be filled with 0.

After the camera is initialized, it is set by calling CameraSetIspOutFormat.

for example:

```
CameraSetlspOutFormat(hCamera, CAMERA_MEDIA_TYPE_MONO8);
```

//After setting, the image output by CameraImageProcess is an 8bit grayscale image, and 1 pixel occupies 1 byte, arranged in sequence.

```
CameraSetlspOutFormat(hCamera, CAMERA_MEDIA_TYPE_MONO16);
```

//After setting, the image output by CameraImageProcess is a 16bit grayscale image, one pixel occupies 2 bytes, arranged in sequence.

```
CameraSetlspOutFormat(hCamera, CAMERA_MEDIA_TYPE_BGR8);
```

//After setting, the image output by CameraImageProcess is a 24bit BGR color image, and one pixel occupies 3 bytes, which are arranged in order of blue, green, and red.

```
CameraSetlspOutFormat(hCamera, CAMERA_MEDIA_TYPE_BGR16);
```

//After setting, the image output by CameraImageProcess is a 48bit BGR color image, and one pixel occupies 6 bytes, which are arranged in order of blue 16bit, green 16bit, and red 16bit.

```
CameraSetlspOutFormat(hCamera, CAMERA_MEDIA_TYPE_BGRA8);
```

//After setting, the image output by CameraImageProcess is a 32-bit BGRA color image. 1 pixel occupies 3 bytes, arranged in order of blue, green, red, and Alpha.

CameraSetIspOutFormat(hCamera, CAMERA_MEDIA_TYPE_BGRA16);

//After setting, the image output by CameraImageProcess is a 64-bit BGRA color image. 1 pixel occupies 8 bytes, followed by blue 16bit, green 16bit, red 16bit, and Alpha16bit.

CameraSetIspOutFormat(hCamera, CAMERA_MEDIA_TYPE_RGB8);

//After setting, the image output by CameraImageProcess is a 24bit RGB color image, and one pixel occupies 3 bytes, which are arranged in order of red, green, and blue.

CameraSetIspOutFormat(hCamera, CAMERA_MEDIA_TYPE_RGB16);

//After setting, the image output by CameraImageProcess is a 48-bit RGB color image, and one pixel occupies 6 bytes, followed by red 16bit, green 16bit, and blue 16bit.

CameraSetIspOutFormat(hCamera, CAMERA_MEDIA_TYPE_RGBA8);

//After setting, the image output by CameraImageProcess is a 32bit
RBGA color image. 1 pixel occupies 4 bytes, followed by red 8bit, green
8bit, blue 8bit, and Alpha8bit.

```
CameraSetIsppOutFormat(hCamera, CAMERA_MEDIA_TYPE_RGBA16);
```

//After setting, the image output by CameraImageProcess is a 64bit
RBGA color image. 1 pixel occupies 8 bytes, followed by red 16bit, green
16bit, blue 16bit, and Alpha16bit.

For 16bit (2 bytes per channel) data, we use little-endian storage, byte 1
is the low 8bit, and byte 2 is the high 8bit.

For the usage of this function, please refer to this routine:

\\Demo\\VC++\\ImageFormat&Saving

5.15 Process the original RAW image

In general, industrial cameras transmit raw RAW data. For
monochrome cameras, it is grayscale image data, and for color cameras,
it is image data in Bayer format.

The CameraGetImageBuffer function gets the original RAW data, which
is 8bit by default. If you need 12 or 16bit format, please refer to the
method in 5.13. Note that CameraGetImageBuffer needs to be used in
conjunction with CameraReleaseImageBuffer.

5.16 Ethernet port camera uses API to dynamically set IP, gateway, and subnet mask

When using the web camera, you must first set the camera to the same subnet segment as the connected network card, so that normal communication can take place. We provide a dedicated gigabit network IP setting tool for IP setting (located in the TOOLS folder of the software installation directory).

At the same time, the user can also dynamically modify the camera's IP parameters in the program through the CameraGigeSetIp function. For specific usage, please refer to the routine: Demo\VC++\GigeConfig.

5.17 Set the frame rate of the camera

Through the CameraSetFrameSpeed function, the output frame rate of the camera can be dynamically set.

CameraSetFrameSpeed is the gear for setting the speed. We divide it into high-speed, medium-speed, low-speed and other modes. For different models of cameras, the corresponding specific frame rates in high, medium and low speed modes are different.

CameraSetFrameSpeed (hCamera,0) ;//Set to low speed mode.

CameraSetFrameSpeed (hCamera,1) ;//Set to medium speed mode.

`CameraSetFrameSpeed (hCamera,2) ;//Set to high-speed mode.`

Generally, when the camera leaves the factory, the default is the highest speed. If you need to reduce the speed, you can call the `CameraSetFrameSpeed` function to reduce the speed. In addition, some models of cameras only support 2 speed modes, some support 4, the specific number of supported modes can be obtained through the `CameraGetCapability` function to get the description information of the camera `tSdkCameraCapbility` structure, The `iFrameSpeedDesc` member in the `tSdkCameraCapbility` structure indicates the number of speed modes supported by the current camera model.

5.18 Use soft trigger or hard (external) trigger function

- Set the camera to continuous image capture mode. The camera is in continuous image capture mode by default, and it can also be switched to continuous image capture mode by `CameraSetTriggerMode(hCamera,0);`
- Set the camera to soft trigger image capture mode. Through `CameraSetTriggerMode (hCamera, 1);` switch to soft trigger image capture mode. After entering this mode, the camera stops collecting and sending images. Only when the user calls `CameraSoftTrigger (hCamera)` once, the camera collects an image once and sends it up.

After sending, the camera will enter the waiting state again, until the next time the user calls `CameraSoftTrigger(hCamera)`.

- Set the camera to hard trigger image capture mode. Through `CameraSetTriggerMode (hCamera, 2)`; switch to hard trigger image capture mode. After entering this mode, the camera stops collecting and sending images. Only when the user inputs a pulse on the trigger terminal on the camera housing, the camera will collect an image and send it up. After sending, the camera will enter the waiting state again until it receives the trigger pulse next time.

If the user needs to trigger multiple frames of images at a time, they can be set by the `CameraSetTriggerCount` function. The default is to get one frame of image with one trigger. If you need to get N images in one trigger, you can call `CameraSetTriggerCount(hCamera,N)` to set.

5.19 Debounce the signal in external trigger mode

In order to avoid the influence of interference signals on external triggering, especially when mechanical signals are used for external triggering of the camera, the signal will have a lot of interference and jitter, which may easily cause misjudgment. Therefore, you can use the `CameraSetExtTrigJitterTime` function to set the debounce time to increase the system's ability to carry interference. The default time is 0,

which means no debounce, and no debounce work when the electronic switch signal is clean. Setting the debounce time will introduce a trigger delay. For example, if the debounce time is 10 milliseconds, the trigger delay will be delayed by at least 10 milliseconds.

5.20 External trigger mode set trigger delay time

If the camera needs to delay for a period of time after the trigger signal is sent out before taking pictures, you can use the trigger delay method. Typical applications, such as using a proximity switch as the trigger signal source, but when the proximity switch sends a signal, the object is not in place or is still in an unstable mode such as jitter. At this time, it is necessary to delay a period of time before triggering a photo. The `CameraSetTriggerDelayTime` function is used to set the external trigger delay in microseconds.

5.21 Detect camera disconnection and automatic reconnection

In some extreme environments, the camera may be at risk of disconnection, for example, when the power supply of the computer host is unstable, or the vibration is large, the USB port is loose and so on.

By default, our USB and Gigabit LAN cameras have the function of automatically disconnecting and reconnecting, and this function has also

been integrated in the SDK. If the user needs to monitor the disconnection in real time, he can call the CameraConnectTest function periodically to query.

5.22 Reading and writing of camera serial number

The camera comes with a three-level serial number. The first-level serial number is read-only and cannot be modified. It has been fixed when it leaves the factory. The secondary and tertiary serial numbers can be read and written freely. The serial number of each level is 32 bytes.

CameraReadSN, read the serial number.

CameraWriteSN, write the serial number.

For the usage of this function, please refer to the

\Demo\VC++\UserDataTest example in the installation directory.

5.23 Read and write custom data in the camera

All models of our cameras are equipped with a data storage space for users to read and write freely. And it will save even after power failure, no data will be lost. The size of the storage space is different for each model of camera. The specific size can be obtained through the CameraGetCapability function. The iUserDataMaxLen of the tSdkCameraCapbility structure represents the maximum storage bytes of the camera.

CameraSaveUserData, write a piece of data to the camera. It is still saved after power failure.

CameraLoadUserData, read a piece of data from the camera.

For the usage of this function, please refer to the

\Demo\VC++\UserDataTest example in the installation directory.

5.24 Get the string description information corresponding to the error code

CameraGetErrorString

For example, the CameraInit function returns an error code of -45, and then `char* s = CameraGetErrorString(-45)` will get a pointer to a string. If it is printed out with the `printf` function, it will display:

"Access is forbidden. When the specified camera is already occupied by other programs, if you apply to access the camera again, it will return to this state. (A camera cannot be accessed by multiple programs at the same time)";

6 GigE Vision and USB3 Vision XML file definition explanation

For customers who use the standard Vision protocol interface to develop, during the device enumeration stage, the XML configuration file corresponding to the camera needs to be parsed. In this part, we provide a special file for explanation. Please refer to "USB3, GigE Vision Interface Development and XML Analysis.pdf" in the Document folder under the installation directory.

7 Halcon development guide

Halcon is currently the most versatile machine vision software in the world. Users around the world benefit greatly from Halcon's flexible architecture for rapid development of image analysis and machine vision programs. Halcon provides more than 1,100 libraries with outstanding performance controllers, such as fuzzy analysis, morphology, pattern matching, 3D correction, etc. Halcon supports multiple operating systems.

Our camera interface is optimized specifically for Halcon and can support Halcon well. In the system where Halcon has been installed, run our camera software installation package ChinaVision Platform Setup (x.x.x.x).exe installation program, the installation program will automatically detect the installation path of Halcon and set it accordingly. Currently, our camera development kit supports the 32-bit and 64-bit versions of Halcon8, Halcon9, Halcon10, Halcon11, and Halcon12.

After connecting the camera to the PC with the data cable, you can start development. The following instructions take the MV-U300 model camera as an example. When developing other models of cameras, you only need to modify the model name.

7.1 Development in HDevelop

1. After starting Halcon, click on the "Assistant" menu and select "Open New Image Acquisition", as shown in Figure 7.1.

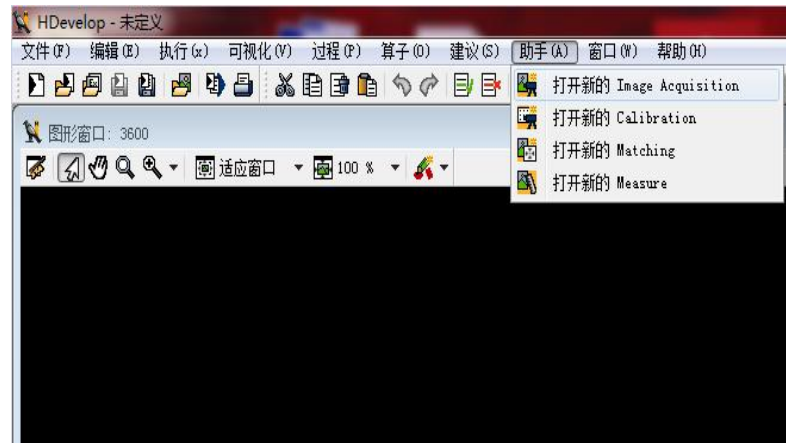


Figure 7.1 Halcon image acquisition interface

2. Click "Image Acquisition Interface (I)" and find ChinaVision in the drop-down list. As shown in Figure 7.2.
3. Click the "Connect" tab in Figure 7.2, and a window as shown in Figure 7.3 will appear. In Figure 7.3, we can see our MV-U300 camera. If multiple ChinaVision cameras are connected at the same time, please select the camera you want to access in the device list.

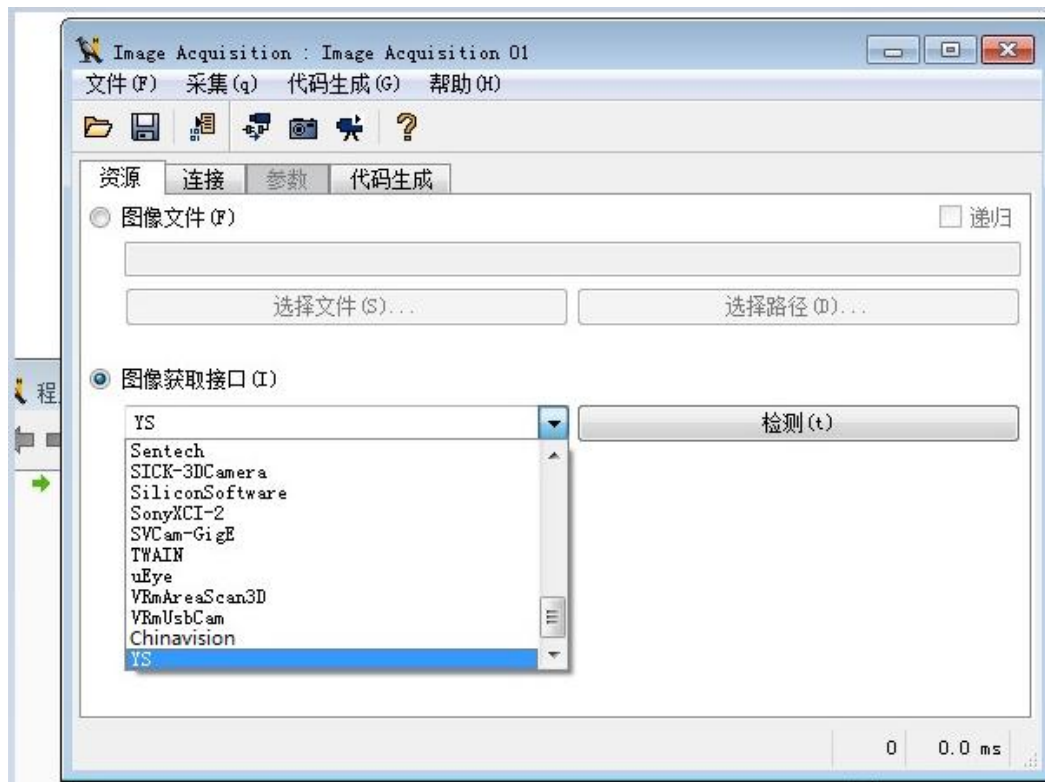


Figure 7.2 Find ChinaVision in the Halcon image acquisition interface list

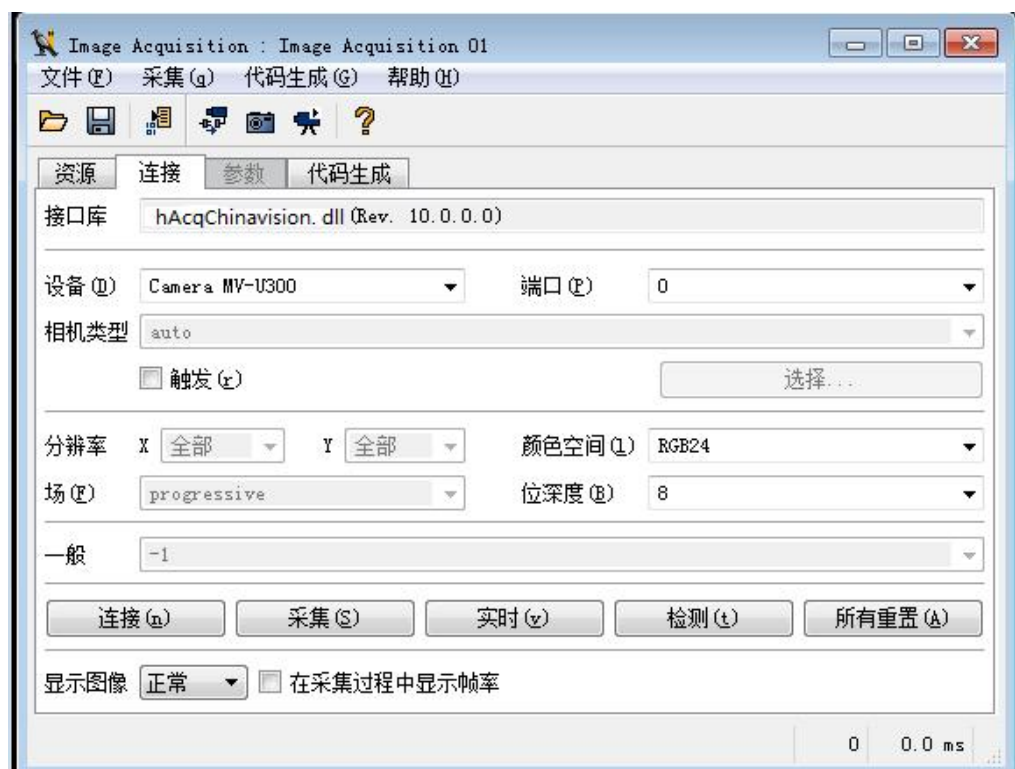


Figure 7.3 Camera acquisition interface window in Halcon

4. Click the "Connect (n)" button in Figure 7.3. After a successful connection, the "Parameters" tab will be launched, as shown in Figure 7.4. The color space and bit depth can be selected at this time.
- Our interface supports two color spaces, RGB24 and Gray. When RGB24 is selected, the output image is in RGB888 format, which is suitable for color cameras. When Gray is selected, the output image format is 8-bit grayscale, which is suitable for monochrome cameras.
 - Bit depth supports 8, 10, and 12 bit depth (only some cameras support 10 and 12 bit bit depth, and the non-frame memory series only support 8 bit bit depth).



Figure 7.4 Successfully connected to MV-U300 in Halcon

5. parameter setting. Click the "Parameter" tab page button to switch to the parameter configuration page, as shown in Figure 7.5.
- bits_per_channel can select the bit depth of each channel;

- `grab_timeout` is the capture timeout time, the unit is milliseconds, and the default is 200 milliseconds timeout; Note that if the image does not arrive within the timeout period, the capture thread will be blocked until the timeout period expires or a valid image frame is read. You can use `set_framegrabber_param (AcqHandle,'grab_timeout', 200)` to set the timeout period of image capture, in milliseconds.
- After clicking `camera_play`, start or continue the image capture function;
- After clicking `camera_pause`, pause the image capture function;
- After clicking `show_camera_settings`, the configuration page of the MV-U300 camera will be displayed, as shown in Figure 5.6. In the halcon program, you can use `set_framegrabber_param (AcqHandle,'show_camera_settings', 1)` to display the configuration page of the camera.
- After `software_trig` is clicked, if the camera is in soft trigger mode, it will trigger the camera to collect a frame of image and send it to the host. If the camera is not working in the soft trigger mode, this button is invalid. In the halcon program, you can send a soft trigger command to the camera via `set_framegrabber_param (AcqHandle,'software_trig', 1)`.
- `shutter`, set the camera's exposure time, in microseconds.
- `contrast`, set the contrast of the camera. The range is 0 to 200, and the default is 150.
- `gamma`, set the gamma value of the camera, the range is 0 to 1000, the default is 50, and the corresponding gamma coefficient is 0.5.

- `exposure_mode`, select the exposure mode. 0 means manual mode; 1 means automatic mode. Only when `exposure_mode` is set to 0, the adjustment of shutter and gain parameters will take effect.
- `trigger_mode`, select the camera working mode. 0 means continuous acquisition mode; 1 means soft trigger mode; 2 means external hardware trigger mode. In the program, you can use `set_framegrabber_param (AcqHandle, 'trigger_mode', 2)` to let the camera enter the external trigger mode. At this time, as long as the external input signal is to the camera trigger pin, the camera will start to collect a frame of image and transmit it to the PC. When there is no trigger signal, the `grab_image_async (Image, AcqHandle, -1)` function will return a timeout.
- `frame_speed`, select the acquisition speed of the camera, 0 means the slowest, 1 means medium, and 2 means the highest. Some cameras only have a medium speed mode. The slower the acquisition speed, the longer the maximum exposure time of the camera. If you need a long exposure, please set `frame_speed` to 0. In the halcon program, you can call `set_framegrabber_param (AcqHandle, 'frame_speed', 0)` to change the camera's acquisition speed.
- `resolution`, select the default resolution. 0 means select the first default resolution, 1 means select the second default resolution, and so on. Can be switched dynamically.
- `color_temperature`, select the color temperature mode (it is invalid for monochrome cameras). 0 indicates a color temperature of about 2700K, 1 indicates a color temperature of about 4200K, 2 indicates a color temperature of about 5500K, and 3 indicates a color temperature of about 6500K.

- `red_gain`, set the digital gain of the red channel. The range is 0 to 4, used for fine-tuning the color. For monochrome cameras, this parameter has no meaning.
- `green_gain`, set the digital gain of the green channel. The range is 0 to 4, used for fine-tuning the color. For monochrome cameras, this parameter has no meaning.
- `blue_gain`, set the digital gain of the blue channel. The range is 0 to 4, used for fine-tuning the color. For monochrome cameras, this parameter has no meaning.
- `saturation`, set the saturation of the camera color. Only valid for color cameras. The range is from 0 to 200. When it is set to 0, the image completely loses its color and becomes a monochrome image. When it is set to 200, the color is the most colorful, and the default is 100.
- `gain`, set the analog gain of the camera. Together with the shutter parameter, this parameter determines the brightness of the image. Only when `exposure_mode` is set to 0, the gain and shutter parameters can be adjusted manually.
- `color_space`, set the format of image output. If Gray is selected, 8 grayscale images will be output. When RGB24 is selected, 24-bit color images will be output. Monochrome cameras can also choose RGB24 format, but R=G=B; color cameras can also choose 8-bit grayscale images for output.
- `GPO0`, `GPO1`, `GPO2`, `GPO3` These 4 correspond to the control of the 4 output IO states on the camera,
`set_framegrabber_param (AcqHandle,'GPO0', 1)` sets the OUTPUT IO state numbered 0 to high level,
`set_framegrabber_param (AcqHandle,'GPO0', 0)` Set the status of OUTPUT IO with number 0 to low level;
`set_framegrabber_param (AcqHandle,'GPO1', 1)` Set the

OUTPUT IO status of number 1 to high level, `set_framegrabber_param (AcqHandle,'GPO1', 0)` Set the status of OUTPUT IO with number 1 to low level; the IO with other numbers can be deduced by analogy.

- GPIO, GPI1, these 2 correspond to the reading of the 2 input IO status on the camera, `get_framegrabber_param (AcqHandle,'GPIO')` reads the status of the IO numbered 0, and returns 0 for low level and 1 for high level; `get_framegrabber_param (AcqHandle,'GPI1')` reads the status of the IO numbered 1, and returns 0 for low level and 1 for high level.



Figure 7.5 Camera parameter setting page in Halcon

6. Image Acquisition. Image acquisition in Halcon is divided into two methods: single acquisition and real-time acquisition. Click the "Acquisition (S)" button in Figure 7.4 to perform a single acquisition. After the acquisition, the image is now in the Halcon window, as shown in Figure 7.7; click the "Real-time (v)" button in Figure 7.4 to perform continuous acquisition, Halcon will continuously display the collected images.

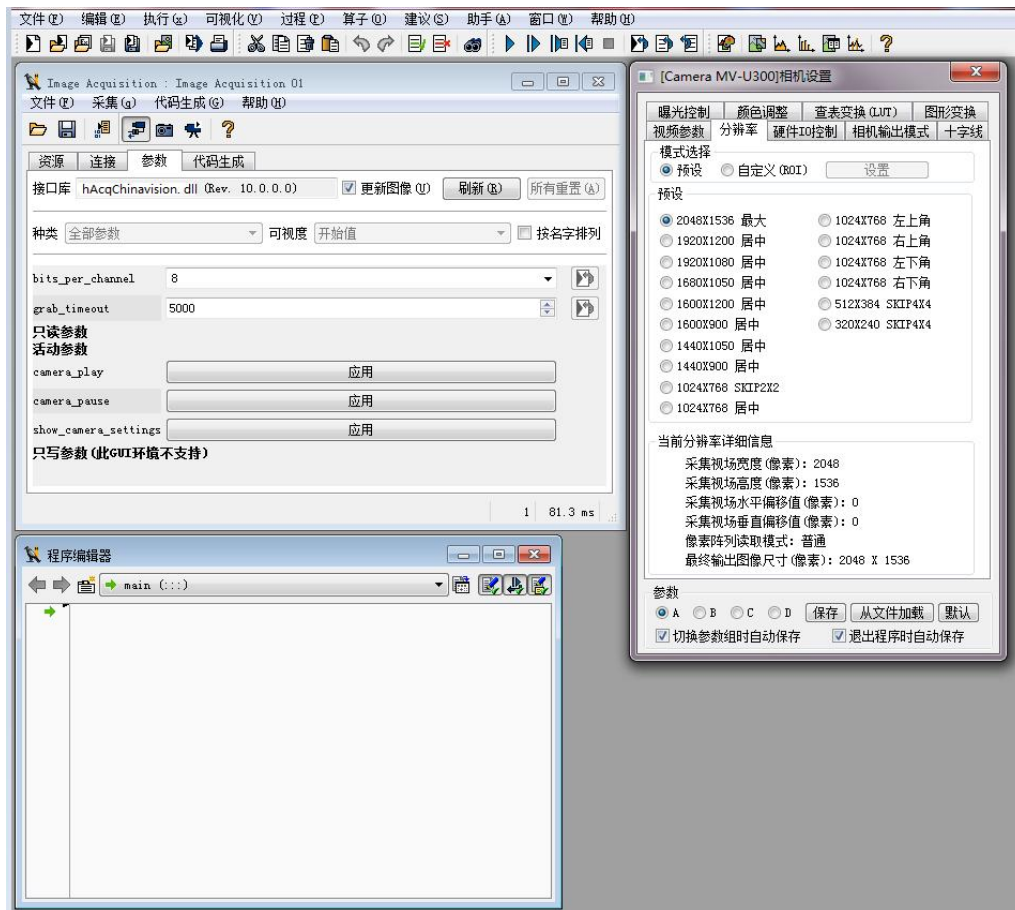


Figure 7.6 The setting window of the camera displayed in Halcon

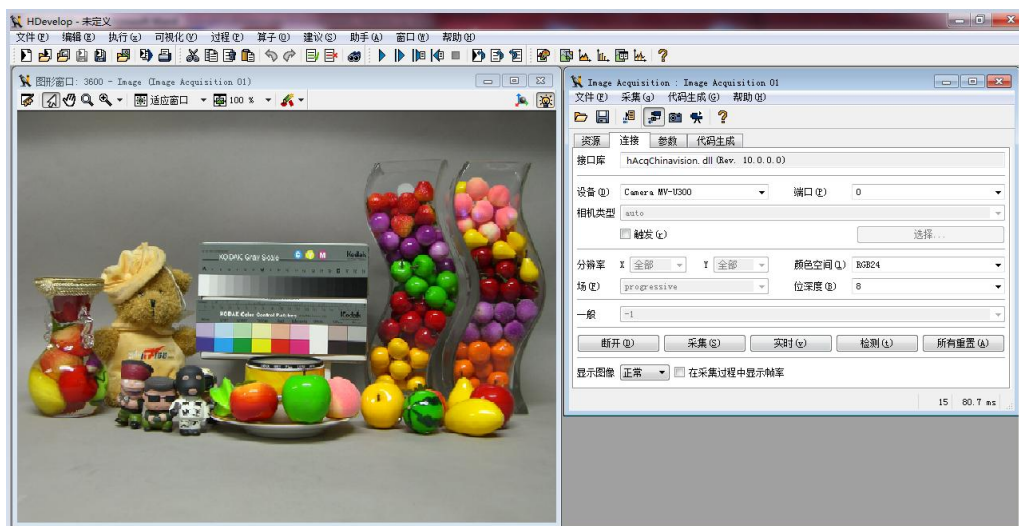


Figure 7.7 Images collected in Halcon

7.2 Halcon development in C/C++, VB, C#

If you need to use C/C++, VB, C# and other languages for Halcon development, you only need to generate the corresponding project and code in HDevelop. The specific method is: in Figure 7.5, click the "code generation" tab, as shown in Figure 7.8:

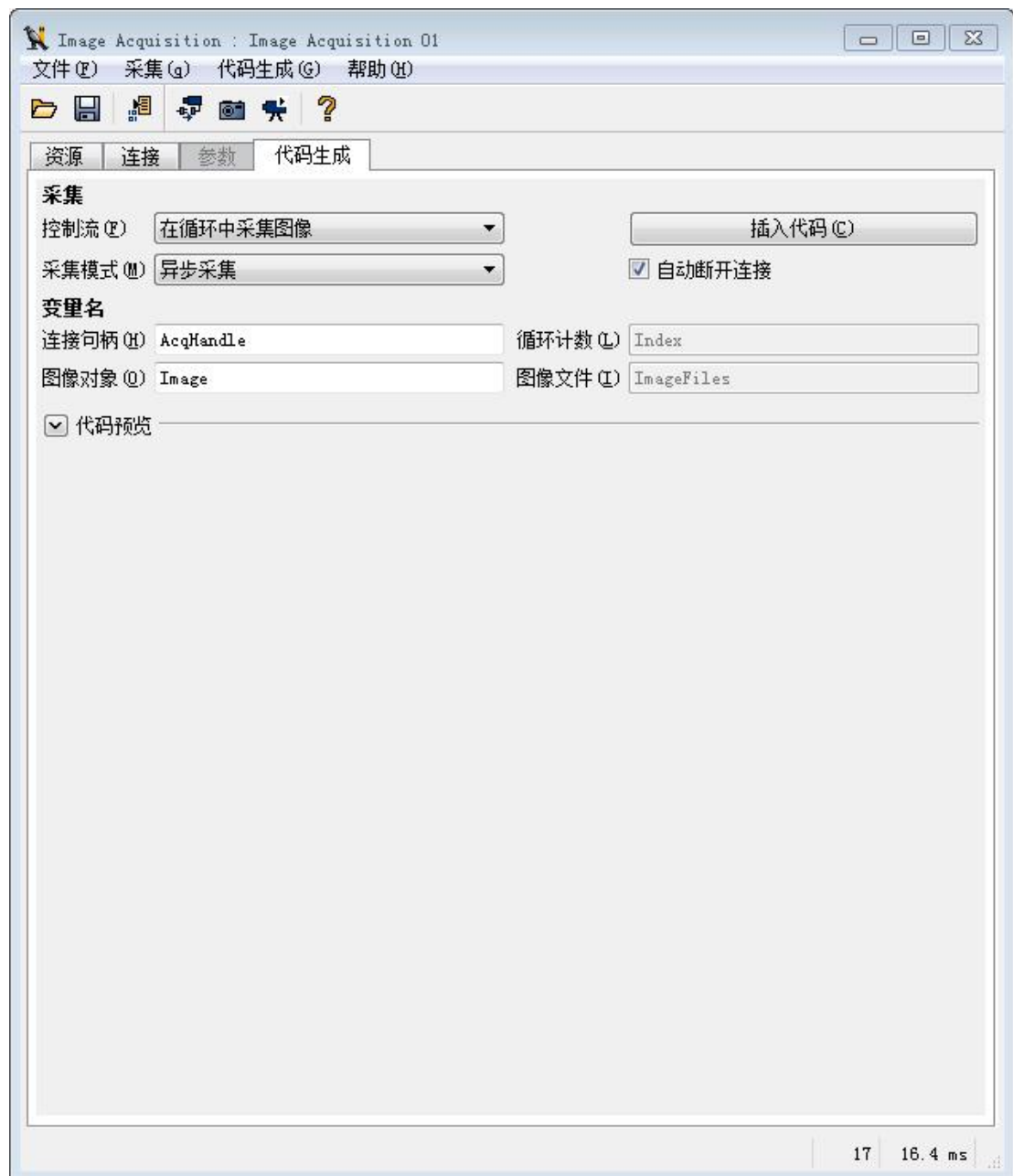
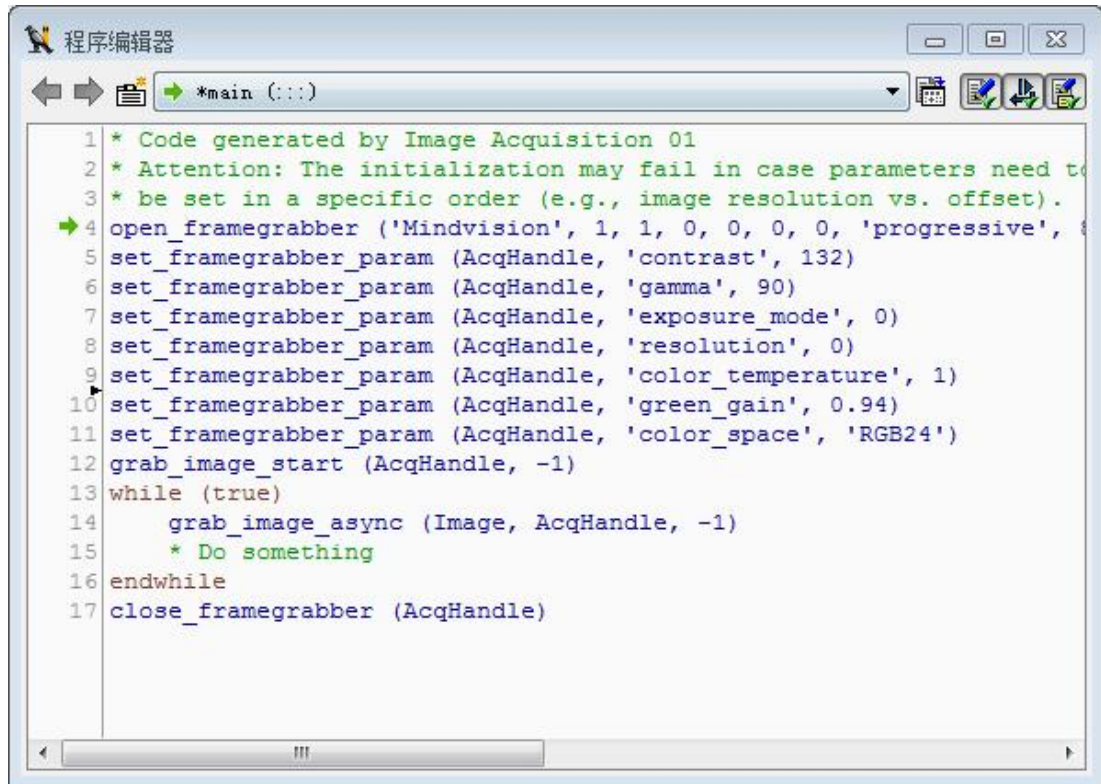


Figure 7.8 Code generation configuration in Halcon

In 7.8, you can perform the corresponding configuration, or directly click the "Insert Code" button to get the code that can be compiled and run in HDevelop, as shown in Figure 7.9.



```
1 * Code generated by Image Acquisition 01
2 * Attention: The initialization may fail in case parameters need to
3 * be set in a specific order (e.g., image resolution vs. offset).
4 open_framegrabber ('Mindvision', 1, 1, 0, 0, 0, 0, 'progressive', 8
5 set_framegrabber_param (AcqHandle, 'contrast', 132)
6 set_framegrabber_param (AcqHandle, 'gamma', 90)
7 set_framegrabber_param (AcqHandle, 'exposure_mode', 0)
8 set_framegrabber_param (AcqHandle, 'resolution', 0)
9 set_framegrabber_param (AcqHandle, 'color_temperature', 1)
10 set_framegrabber_param (AcqHandle, 'green_gain', 0.94)
11 set_framegrabber_param (AcqHandle, 'color_space', 'RGB24')
12 grab_image_start (AcqHandle, -1)
13 while (true)
14     grab_image_async (Image, AcqHandle, -1)
15     * Do something
16 endwhile
17 close_framegrabber (AcqHandle)
```

Figure 7.9 HDevelop code automatically generated

The next step is to convert the code shown in Figure 7.9 into C/C++, VB, Delphi, and C# projects. Click the "File" menu and select "Export", as shown in Figure 7.10.

Then in the pop-up export dialog box, set, as shown in Figure 7.11, in the drop-down list, select the language type you want to export, and then click the "Export" button. After the export is complete, you can find the project file of the corresponding language under the path you set, and directly open the compiler to run.

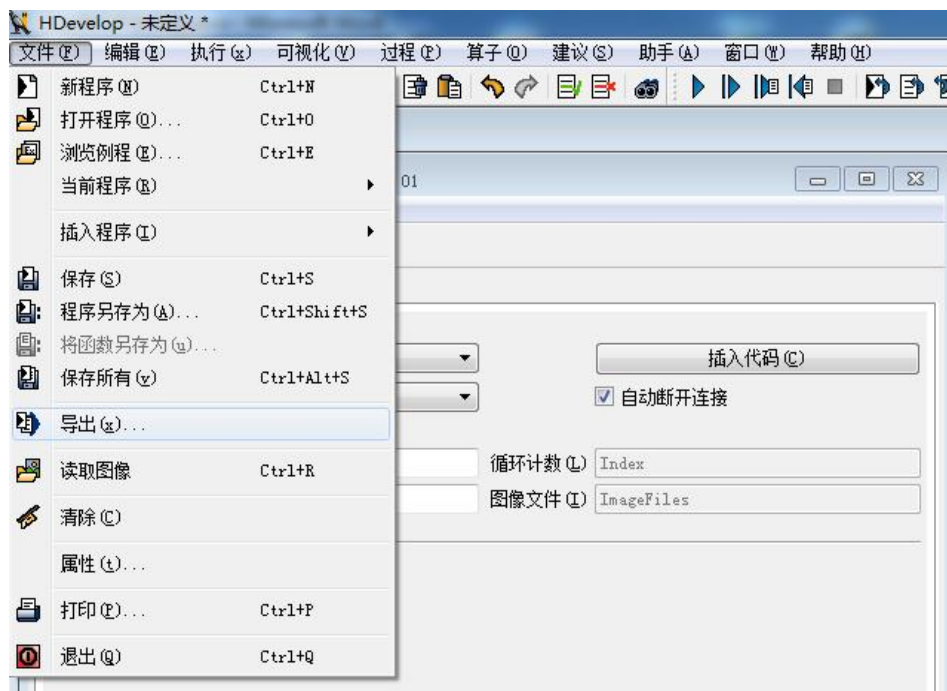
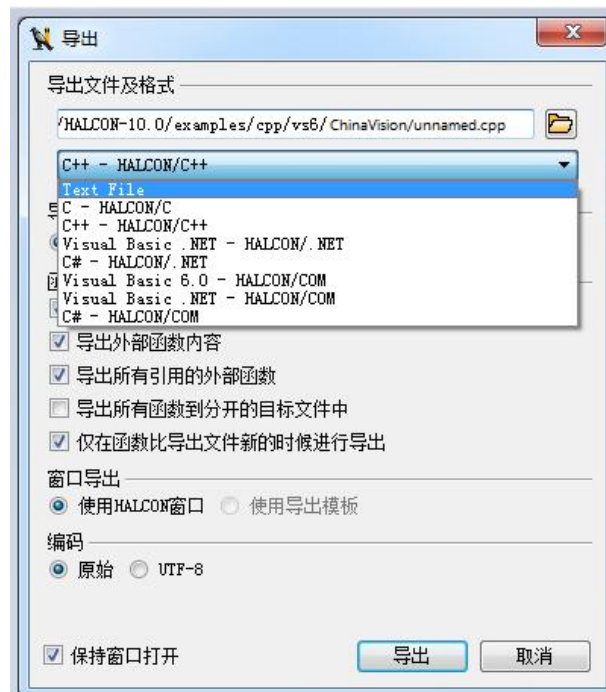


Figure 7.10 Export other development languages



7.11 Export settings

7.3 Develop with multiple cameras at the same time

When connecting two cameras at the same time, in order to better distinguish the corresponding relationship of each camera, the following process is recommended:

1, Use the demo software we provide to modify the camera's device nickname, For example, the name of the first camera is changed to Camera1, and the name of the second camera is changed to Camera2, After the modification is completed, no matter which USB or network interface the two cameras are connected to, their names will be Camera1 and Camera2, and they will still be valid after changing the computer (The modification of the camera device name is shown in Figure 5.12).

2, When initializing the Halcon device, fill in the device names "Camera1" and "Camera2" respectively, as shown below:

```
open_framegrabber ('ChinaVision', 1, 1, 0, 0, 0, 0, 'progressive', 8, 'Gray',  
-1, 'false', 'auto', 'Camera1', 0, -1, AcqHandle)
```

```
open_framegrabber ('ChinaVision', 1, 1, 0, 0, 0, 0, 'progressive', 8, 'Gray',  
-1, 'false', 'auto', 'Camera2', 0, -1, AcqHandle)。
```

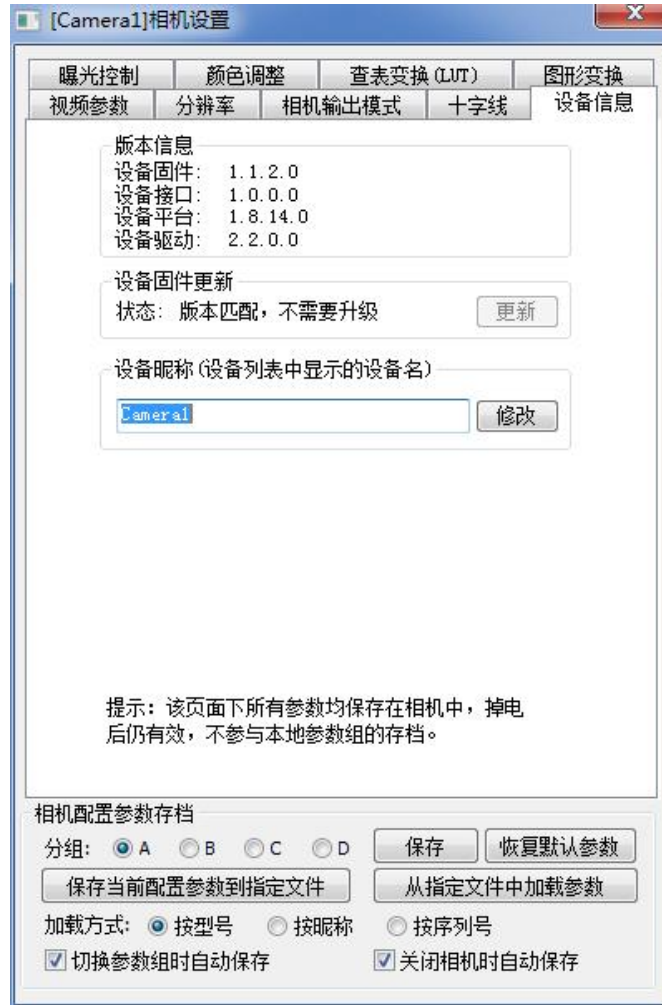


Figure 7.12 Modify camera name

8 LabView Development Guide

In Labview, development can be done in two ways: NI MAX tools and the routines we provide based on DLL calls.

8.1 Develop with NI MAX

1. Connect the camera first, and then run NI MAX. You can find our camera under NI-IMAQdx Devices under "Devices and Interfaces", as shown in Figure 8.1. After double-clicking, you can get the camera preview screen as shown in Figure 8.2 (you need to click the Grab button at the top of Figure 8.2 to preview).
2. In Figure 8.2, the resolution can be switched.
3. After clicking the Camera Attributes tab in Figure 8.2, you can set other camera parameters. As shown in Figure 8.3. Under this interface, due to the limitation of the protocol, only some parameters of the camera are allowed to be set, and the setting of other parameters. You can use our demo software, after adjustment, save it (The camera parameters can be saved as a file, no matter which method you use to develop the parameter file can be effectively automatically loaded), and then open with NI MAX, the same effect.



Figure 8.1 NI MAX device list

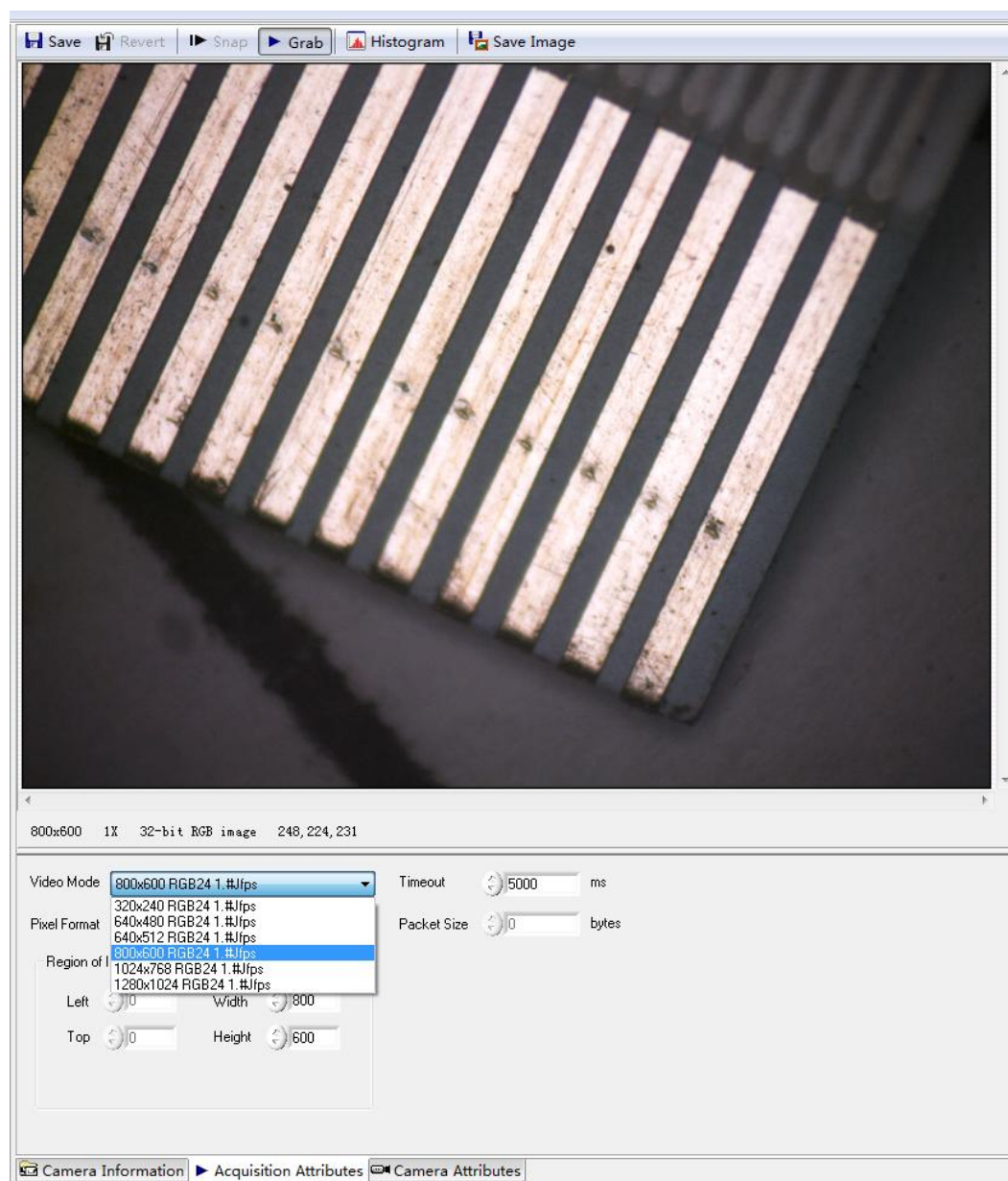


Figure 8.2 Preview interface

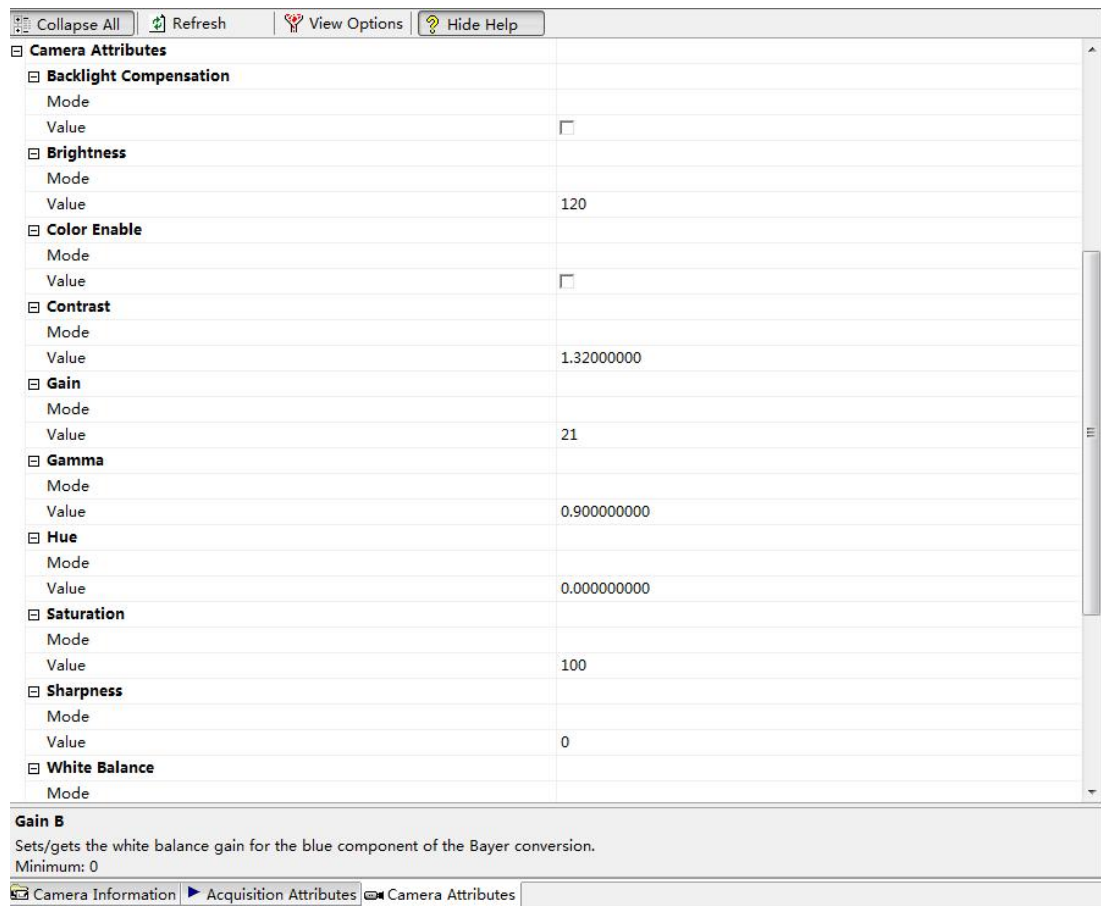


Figure 8.3 Parameter configuration of the camera in NI MAX

8.2 Development based on DLL file call method

The routine of this method is located in the DEMO/Labview/useDLL folder of the installation directory, and demonstrates how to develop by calling the DLL file (MVCAMSDK.dll) of the SDK. This routine has the following characteristics:

1. Support automatic recognition of monochrome and color cameras, monochrome cameras use U8 format grayscale images for display; color cameras use U32 format color images for display.

2. Supports simultaneous development and use of multiple cameras. You only need to increment the first parameter in the CameraInitEx interface call. Enter 0 to initialize the first camera, enter 1 to initialize the second camera, and the rest of the VI source code can be copied directly.
3. This routine can be applied to all models of our cameras. Regardless of the resolution, trigger mode, or transmission interface, the routine can be automatically identified without manually modifying any variables.
4. The camera parameters support saving and loading (file mode). Camera parameter files modified by other software can also be loaded by this routine, reducing the workload of manual code input.

8.3 Use multiple cameras in Labview

The example of this method is located in the DEMO/Labview/TwoCameras directory of the installation directory. This example is also based on the DLL method and demonstrates how to use multiple cameras simultaneously in Labview. The example shows how to use 2 cameras at the same time. It can be 2 cameras of the same model, different models, or even multiple cameras with different interfaces, such as 1 USB2.0, 1 USB3 .0, 1 GIGE camera, can use this method for

multi-camera development. When using more than two cameras to develop at the same time, please refer to the method in this example for expansion. The current number of multiple cameras in the SDK is limited to 64. When you need to use more than 64 cameras at the same time, please contact our technical support , Can further expand the number of multi-camera support.

8.4 The distinction of Labview multi-camera

When developing with multiple cameras, the corresponding relationship of each camera is often required. Different cameras will be used to complete different tasks. There are many ways to distinguish between multiple cameras, such as the unique serial number in the camera, the camera name, and the custom data in the camera. In Labview, we provide the following 2 ways to distinguish multiple cameras:

- The way to use custom data. We provide an interface to read and write custom data in the camera. You can distinguish different cameras based on these data. But this method must be after the camera has been initialized before the user-defined data can be read. Therefore, in the programming of Labview, after the camera initialization is completed, the user-defined data must be obtained through the interface to determine which camera it is. Then carry out

the corresponding branch processing.(The reading and writing method of camera custom data has been provided in the routine)

- Use the function of customizing the device name.The process is similar to the distinction between multiple cameras in Chapter 5.3 Halcon.In the first step, as shown in Figure 6.4, change the device names of the two cameras to Camera1 and Camera2 respectively. In the second step, in Labview, call the CameraInitEx2 interface to initialize the camera, and the first parameter is passed into the strings "Camera1" and "Camera2" respectively.After the modification, the name is solidified inside the camera, which is permanently valid and will not be affected by the interface or computer replacement. This routine is located in the DEMO/Labview/TwoCamerasEx directory of the installation directory.

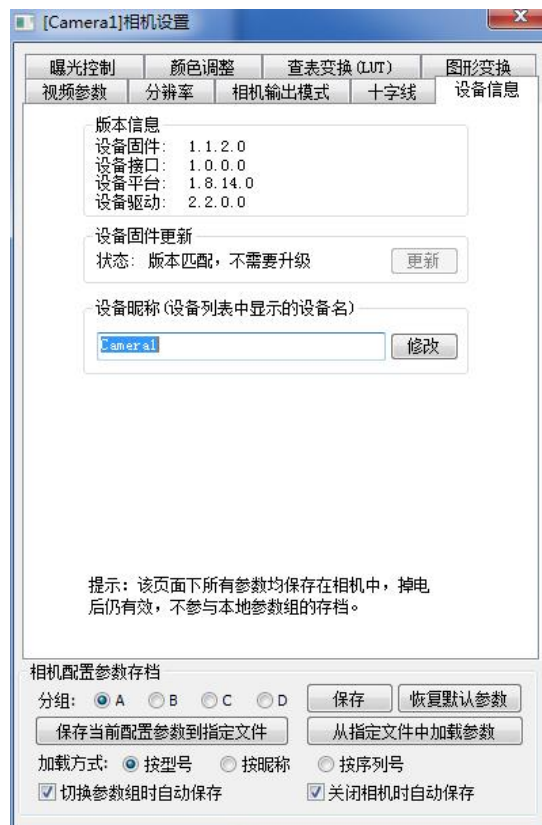


Figure 8.4 Modify camera nickname

9 Release the camera installation file after integration

When you integrate our camera and need to package and release the camera SDK files, you can follow the following methods:

1, The files that need to be packaged are in our camera program installation directory, there are the following folders:

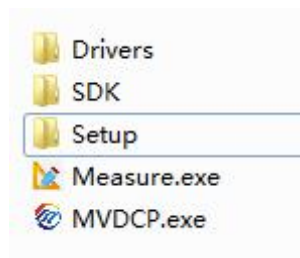
- ./SDK directory, which contains all SDK files.
- ./Drivers directory that contains the camera's kernel driver file.
- he ./Demo directory contains the development DEMO of the camera, you can publish it as needed, or not publish the Demo program at all.
- ./Camera/Configs directory, the corresponding camera configuration file. This directory saves the configuration parameters of the camera on your current computer, you can package them together and release them, and the parameter file will be automatically loaded. If the camera configuration file is not released, the camera will use the default parameters for the first time and generate a file in the Camera/Configs directory.

Please put the above files and the exe program you

developed in the same directory.

2, How to publish. You can choose the following 3 ways to publish.

- Use our platform installation package to install directly, ChinaVision Platform setup (version number).exe, but the installation package will contain some company information.
- Use the tools we provide. In the Setup folder, we provide a program called "Installer.exe", which is used to install all files in the SDK and Drivers directories. Copy the SDK, Drivers, and Setup folders to the target machine, and then run "Installer.exe" in the Setup folder to automatically complete the installation. As shown in the figure below (MVDCP.exe and Measure.exe are the demo software and measurement software provided by us, and it is not necessary to publish them. After running Installer.exe, the MVDCP and Measure software can run normally):



- Manually or write the installation program yourself to install. First copy the Drivers directory to the target machine, and then manually install the device kernel driver (When the USB camera is plugged into the computer, it will prompt to install the driver. Just select the corresponding INF file in the Drivers directory. Gige cameras need to manually run the MvDriverInstall.exe program under Drivers\Gige to complete the kernel driver installation); Then copy all the files in the SDK (if it is X64 system, please copy SDK/X64) folder (note: it is the file under the folder, not the folder) to any directory of the target machine (assuming it is C:\TEST directory), Then put the executable file that you have developed for the second time into the C:\TEST directory, and you can run it directly without other installations.

10 Technical Support

If you encounter any questions during the use and development process, please contact our technical support as soon as possible. We recommend that you try to communicate with us in written forms such as emails, faxes, etc., and attach a detailed description of the problem, screenshots and other information, which will help us solve your problem quickly.

Your success is our success, and your satisfaction is our satisfaction. Any suggestions you give back are affirmation and encouragement to us. We look forward to your feedback and we will work hard to do better!